Fondamenti di Comunicazioni e Internet

Casolino Ilaria, Giamberardini Matteo

a.a. 2023/2024

CAPITOLO 1

Internet si può definire in due modi, ovvero descrivendo i componenti di base hardware e software che la costituiscono, oppure attraverso le infrastrutture di rete che forniscono servizi ad applicazioni distribuite.

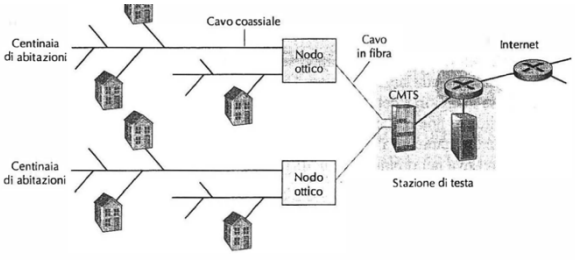
Si definisce come l'insieme di tutti i dispositivi connessi tra loro in rete, questi sono detti **host** o sistemi periferici (end system). Gli host sono definiti tali in quanto ospitano ed eseguono programmi applicativi come browser, web server o software di gestione e lettura di posta elettronica, ad esempio. Talvolta vengono ulteriormente suddivisi in **client** e **server**, i primi richiedono dei servizi, come PC e smartphone, i secondi li erogano e sono macchine più potenti, collocate in data center, in grado di memorizzare e distribuire pagine web o flussi video.

Gli host sono connessi tra loro attraverso una rete di **collegamenti e commutatori di pacchetti** (packet switch). Si definiscono commutatori perché ricevono i pacchetti e li smistano, infatti commutare significa scambiare. I collegamenti possono essere differenziati per il mezzo fisico del quale si servono che ne determina la velocità di trasmissione dati misurata in bit/secondo (bps). Affinchè i dati possano essere trasferiti tra due sistemi periferici è necessario vengano suddivisi in **pacchetti**, ovvero unità informative (payload) alle quali viene aggiunta un'intestazione (header). I pacchetti viaggiano attraverso reti di collegamento alle quali accedono grazie ai commutatori a livello di collegamento, nel mentre attraversano svincoli e incroci (router) nel nucleo della rete, fino a giungere a destinazione (host) dove vengono riassemblati per ottenere i dati originali. La sequenza di collegamenti e commutatori attraversati dai pacchetti viene definita percorso o **route**.

Gli host accedono ad Internet tramite gli **ISP**, Internet Service Provider, che forniscono svariate modalità di accesso alla rete: residenziale a larga banda (DSL), locale ad alta velocità, senza fili (wireless), dial-up (modem). Un **provider** è un insieme di commutatori di pacchetto e di collegamenti. Ciascuna rete di un ISP utilizza il protocollo IP per controllare l'invio e la ricezione di informazioni.

Cos'è un protocollo? I **protocolli** sono le modalità, ovvero insiemi di regole e procedure, per mezzo delle quali le diverse reti comunicano tra loro. Tutto ciò è standardizzato, ovvero non conta solo il messaggio che deve essere trasferito, ma anche il suo formato e l'ordine con cui i frammenti che lo costituiscono sono inviati. Così come accade per lo scambio di informazioni tra una persona e un'altra, lo stesso si verifica per lo scambio di informazioni tra diversi computer tramite la rete. In Internet esistono vari protocolli, i più conosciuti sono TCP e IP, ma esistono anche http e SMTP.

LA RETE



Una rete si riferisce a dispositivi di calcolo interconnessi che possono scambiare dati e condividere risorse tra loro. Il **nucleo della rete** (core network) è una maglia di commutatori di pacchetto e collegamenti, ed è definito come un sistema di comunicazione che consente il trasferimento di dati e informazioni tra dispositivi collegati tra loro. Il tempo necessario per inviare una richiesta e ricevere una risposta in rete è denominato latenza.

Reti di accesso

La **rete di accesso** (access network) è il tipo di rete che consente la connessione tra edifici, ad esempio case e uffici. Le abitazioni, in particolare, accedono ad internet attraverso l'access network denominato DSL (digital subscriber line). Generalmente lo scambio di dati sulla rete con questo tipo di accesso, avviene attraverso il cavo di rame della linea telefonica detto twisted-pair, infatti il modem DSL residenziale converte i dati in toni di frequenza che possono essere trasportati lungo i cavi di rame. L'utilizzo di questo materiale non è però efficiente poiché rallenta la velocità di trasmissione.

Un altro tipo di access network è quello via cavo, nel quale i dati viaggiano inizialmente su cavi coassiali, utilizzati dalle televisioni, e dopo aver raggiunto la giunzione del quartiere vengono inviati alla centrale attraverso cavi in fibra ottica. Questo standard, o tecnologia, è detto HFC (hybrid fiber coax). Negli ultimi anni si è diffusa la tecnologia FTTH, (fiber to the home), per cui le case sono collegate direttamente con la centrale ISP (Internet Service Provider) attraverso cavi in fibra ottica che permettono di raggiungere una trasmissione dati nell'ordine di Gigabit al secondo.

Mezzi di trasmissione

I dispositivi comunicano sfruttando i server che fanno da tramite tra i sistemi periferici e sono dislocati in varie parti del mondo. L'interconnessione di questi dispositivi avviene tramite i media che possono essere di due tipi:

i. con cavo o cablati (**wired**): cavi coassiali, fili di rame, fibra ottica;
ii. senza cavo (**wireless**): onde elettromagnetiche.

Questi mezzi hanno differenti velocità di trasmissione di dati, misurata in bit/secondo. I mezzi fisici sono divisi in due categorie: vincolati e non vincolati. Nei primi, le onde vengono contenute in un mezzo fisico, quale un cavo in fibra ottica, un filo di rame o un cavo coassiale. Nei secondi, le onde si propagano nell'atmosfera e nello spazio esterno, come avviene nelle LAN wireless o nei canali digitali satellitari.

i. Uno dei mezzi di trasmissione più diffusi è il cavo doppino di rame, costituito da due fili di rame intrecciati non schermati.
ii. Il cavo coassiale è costituito da due conduttori di rame concentrici. Con questa struttura, e grazie a uno speciale isolamento e schermatura, il cavo coassiale può raggiungere alte frequenze di trasmissione. Il suo impiego è piuttosto comune nei sistemi televisivi via cavo.
iii. La fibra ottica è un mezzo sottile e flessibile che conduce impulsi di luce, ciascuno dei quali rappresenta un bit. Una singola fibra ottica può supportare enormi velocità trasmissive, fino a decine o centinaia di Gigabit al secondo. Tale mezzo è immune all'interferenza elettromagnetica.

La Wireless access network è resa possibile grazie alla produzione di onde elettromagnetiche di diverse frequenze e lunghezze d'onda, che consentono la connessione tra i dispositivi. A seconda della frequenza e della lunghezza d'onda vengono prodotti diversi effetti. Tanto è più bassa la frequenza tanto più lontana può andare la trasmissione, tanto è più alta la frequenza tanto più vicina la trasmissione deve essere.

Ad esempio, la connessione degli smartphone 4G lavora tra frequenze di 800 Mhz, 1800 Mhz e 2600 Mhz, permettendo di raggiungere distanze di diversi chilometri. I canali radio trasportano segnali all'interno dello spettro elettromagnetico in grado di attraversare le pareti, non richiedendo

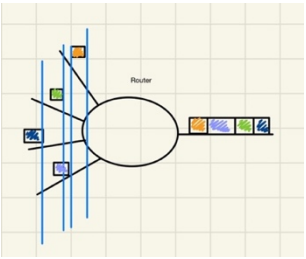
l'installazione fisica di cavi e fornendo connettività agli utenti mobili trasportando il segnale per lunghe distanze.

Metodi di trasmissione

Il **Packet Switching**, come anticipato dal nome che tradotto significa commutazione di pacchetto, ovvero scambio, è un meccanismo usato dai dispositivi per comunicare all'interno della rete. Il suo nome deriva dall'utilizzo di pacchetti contenenti le informazioni da scambiare tra i diversi host, tra le quali l'indirizzo di arrivo. Se un sistema periferico o un commutatore inviano un pacchetto di L bit su un canale con velocità R bps (capacità o **bit rate**), il **tempo di trasmissione** risulta pari a L/R secondi.

La maggior parte dei commutatori di pacchetto utilizza la trasmissione **store-and-forward**. Ciò significa che il commutatore prima di iniziare a trasmettere il pacchetto in uscita deve averlo ricevuto tutto. Immaginiamo di avere due host, sorgente e destinazione, che si vogliono scambiare un pacchetto che passa per un commutatore, ovvero un router. Questo non invia i bit man mano che li riceve, prima li immagazzina tutti nel buffer e solo dopo inizia a trasmetterli. All'istante L/R secondi la sorgente ha finito di trasmettere il pacchetto al router, all'istante 2L/R il pacchetto è arrivato a destinazione.

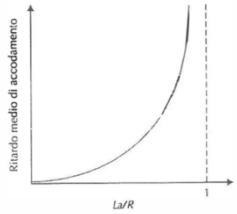
Ogni commutatore di pacchetto connette più collegamenti. Per ciascuno di questi, il commutatore ha un buffer di output, detto **coda di output**, per conservare i pacchetti che sta per inviare su quel collegamento. Se un pacchetto in arrivo richiede l'invio attraverso un collegamento, ma lo trova occupato dalla trasmissione di un altro, deve attendere nella coda di output. Di conseguenza, oltre ai **ritardi store-and-forward** dovuti all'attesa che i pacchetti vengano acquisiti nel buffer del router, i pacchetti subiscono anche **ritardi di accodamento**. Dato che la dimensione del buffer è finita, un pacchetto in arrivo può trovare il buffer riempito da altri pacchetti che attendono di essere trasmessi. In questo caso si verificherà una perdita di pacchetto (**packet loss**), in quanto verrà eliminato il pacchetto in arrivo o uno di quelli in coda (buffer overflow).



Tipicamente l'ordine con cui vengono spediti i pacchetti è **FIFO** ovvero first-in-first-out, il primo che entra è il primo ad uscire. Il pacchetto lungo il percorso da un host ad un altro può subire ritardi di diversa natura, i principali sono il ritardo di elaborazione, il ritardo di accodamento, il ritardo di trasmissione e il ritardo di propagazione, che complessivamente formano il ritardo totale di nodo (**nodal delay**).

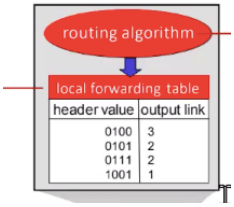
> *Ritardo di elaborazione (processing delay) –* Il tempo richiesto per esaminare l'intestazione del pacchetto e per determinare dove dirigerlo
>
> *Ritardo di accodamento (queuing delay) –* Una volta in coda, il pacchetto subisce un ritardo di accodamento mentre attende di essere trasmesso sul collegamento;
>
> *Ritardo di trasmissione (transmission delay) –* Il tempo richiesto per trasmettere tutti i bit del pacchetto in uscita dal router sul collegamento (L/R), dunque è funzione della lunghezza del pacchetto e della velocità di trasmissione, non della distanza tra i router;
>
> *Ritardo di propagazione (propagation delay) –* Il tempo impiegato dal bit, una volta immesso sul collegamento, per raggiungere il sistema periferico. È dato da d/v, dove d è la distanza

tra i due router, mentre v è la velocità di propagazione nel collegamento, che dipende dal mezzo fisico (fibra ottica, rame e così via), spesso pari a c = $3*10^8$ m/s (velocità della luce).



Il **carico**, ovvero il numero di pacchetti inviati, è compreso tra 0 e 1, 0 per indicare nessuno pacchetto ed 1 per il massimo numero di pacchetti memorizzati. Il ritardo cresce in maniera esponenziale, più pacchetti vengono inviati più il ritardo cresce. Con *La* si indica la velocità media di arrivo dei bit in coda, mentre *La/R* è l'intensità di traffico.

La comunicazione, come detto, avviene mediante i link e i router: i pacchetti percorrono i link incontrando diversi router, dai quali vengono smistati nei diversi collegamenti fino a giungere alla loro destinazione. Questa operazione è detta di **routing**. Il percorso di ogni pacchetto non è deciso da un solo router, bensì viene gestito man mano che questo percorre i collegamenti ed è il prodotto delle decisioni di tutti i router incontrati.

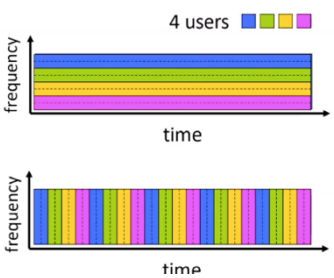


Ogni pacchetto è caratterizzato da un indirizzo di destinazione. Quando questo entra in un router, attraverso un algoritmo di routing e ad una tabella di routing (forwarding table), viene smistato nei diversi collegamenti di output del router stesso. Ogni router ha una tabella diversa, nonostante ciò, grazie all'algoritmo, le tabelle sono combinate tra loro per scegliere il percorso ottimale da far prendere al pacchetto.

L'alternativa al packet-switching è il **circuit-switching**, ovvero commutazione di circuito, che consiste in un circuito instaurato tra l'host di partenza e quello di arrivo. La comunicazione, quindi, non inizia prima di aver stabilito il circuito, ovvero una **connessione end-to-end**, solo dopo quel momento la velocità di trasmissione rimane costante per tutta la durata della comunicazione, poiché è stata riservata una certa larghezza di banda. Questa modalità di trasmissione dati viene utilizzata per la rete telefonica.

Confronto tra Packet-Switching e Circuit-Switching – Il Packet-Switching non è troppo efficiente se consideriamo servizi in tempo reale, in quanto la velocità di trasmissione (bit rate) dipende da molti fattori che influiscono sul ritardo complessivo di ricezione di un pacchetto. Il Circuit-Switching, d'altro canto, offre garanzie di trasmissione, infatti, la comunicazione avviene solo dopo aver stabilito un circuito tra i due host; dunque, *permette di avere risorse dedicate per la specifica comunicazione*. Nonostante ciò, la commutazione di circuito ha dei limiti, infatti, *la comunicazione contemporanea è limitata*: in base alla capienza della rete possono essere attivati diversi circuiti, ma una volta superato il limite, l'utente è impossibilitato ad accedervi.

Multiplexing significa mettere insieme diverse comunicazioni in una rete dedicata utilizzando un solo canale comunicativo.

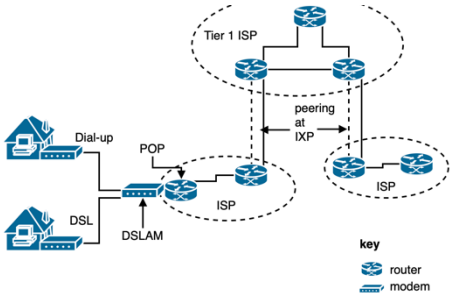


- **FDM:** Frequency Division Multiplexing, ovvero multiplazione a divisione di frequenza. Ad esempio, la radio trasmette il segnale su più stazioni contemporaneamente usando canali differenti, ovvero frequenze differenti;
- **TDM:** Time Division Multiplexing, ovvero Multiplazione a divisione di tempo. Agli utenti vengono dedicati dei time slot, perciò usano tutte le frequenze contemporaneamente in un intervallo di tempo determinato e limitato.

Nella modalità FDM ricavare un canale implica tagliarsi una porzione di banda (frequenza) che ha sempre la stessa dimensione, ciò che può cambiare è la "posizione" del canale durante la comunicazione in base ai nodi percorsi. Nella modalità TDM la frequenza rimane uguale per tutti e viene divisa la risorsa del tempo in modo equo tramite i time slot. In termini di bit-rate tutti trasmettono equamente. La TDM generalmente viene utilizzata per le comunicazioni digitali mentre la FDM per le comunicazioni analogiche.

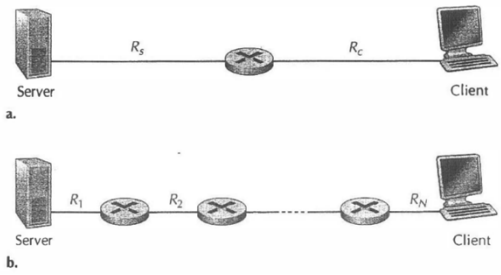
Una rete di reti

Un sistema periferico ha accesso alla rete attraverso gli ISP (Internet Service Provider). Affinché gli host possano scambiarsi informazioni è necessario che gli ISP siano collegati tra loro, si parla perciò di rete mesh. Questo tipo di rete permette l'interconnessione tra tutti i dispositivi, il che porta ad una bassa efficienza, in quanto stabilisce connessioni in numero pari a n^2. Questo significa che quando molti utenti sono connessi alla rete ci sono n^2 connessioni. È conveniente fare uso di reti mesh in contesti domestici o di altro genere, il cui numero di utenti non sia elevato.



Gli ISP sono connessi tra loro attraverso gli IXP (Internet Exchange Point) ovvero dei nodi a cui diversi ISP si connettono. I collegamenti dei diversi ISP sono detti **peering link**. Bisogna ricordare che fare uso di peering significa che gli ISP vicini possono connettere le loro reti in modo che tutto il traffico tra essi passi per una connessione diretta. Un ruolo fondamentale nella rete è ricoperto dai **POP** (Point Of Presence) che permettono agli host di collegarsi agli ISP, esistono in tutti i livelli della gerarchia tranne in quello degli ISP di accesso. I POP sono semplicemente i router in grado di instradare il traffico dei pacchetti nella rete. Una rete cliente si connette al POP del fornitore affittando un collegamento ad alta velocità. La Struttura di rete 5 aggiunge i *content provider networks*, ovvero le reti che si occupano di distribuire contenuti. Questi fornitori di contenuti creano la propria rete privata per gestire l'erogazione dei propri contenuti e per risparmiare i costi dovuti agli ISP di livello superiore.

Throughput



Il throughput è la velocità massima con la quale un router trasmette i pacchetti in uscita, o meglio è la quantità reale di bit trasmessi per secondo. Essa viene misurata in bit per secondo (bps) e si calcola effettuando il rapporto tra il numero totale di bit trasmessi e il lasso di tempo T in cui vengono trasmessi.

Si consideri il throughput per il trasferimento di file dal server al client, come in figura. Sia R_s la velocità del collegamento tra il server e il router e R_c quella del collegamento tra il router e il client.

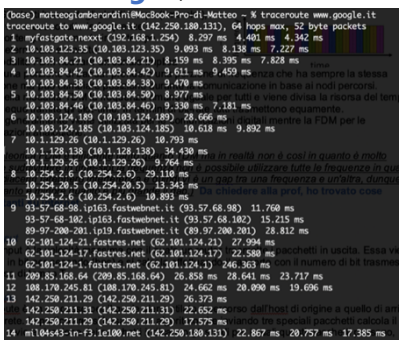
Per calcolare il throughput, dobbiamo pensare ai bit come a un fluido e ai collegamenti come a dei condotti. Chiaramente il server non può pompare nel suo collegamento dati a una velocità maggiore di R_s bps e il router non può inoltrare bit a una velocità più alta di R_c bps. Se $R_s < R_c$ allora i bit immessi dal server scorreranno attraverso il router e arriveranno al client a una velocità di R_c bps. Se $R_s > R_c$ allora il router non sarà in grado di inoltrare i bit alla stessa velocità alla quale li riceve. In tal caso, i bit lasceranno il router a una velocità di R_c. La quantità di bit accumulata al router in attesa di

trasmissione cresce indefinitamente. Quindi, per questa semplice rete con due collegamenti il throughput è il min (R_c, R_s).

Bottleneck

Il bottleneck è un processo per il quale la velocità di trasmissione di un sistema è fortemente vincolata alla velocità di trasmissione di un singolo componente. Se il router trasmettesse i dati in uscita ad una velocità inferiore rispetto a quella di entrata, si creerebbe il **collo di bottiglia**, dovuto al ritardo di accodamento dei pacchetti in uscita.

Traceroute



Il traceroute è un processo per il quale viene stilato il percorso dall'host di origine a quello di arrivo lungo la rete. In particolare, si basa su un algoritmo che inviando tre speciali pacchetti calcola il tempo di invio e ricezione di questi indicando gli snodi per i quali i pacchetti passano, registrando l'indirizzo IP di ogni router.

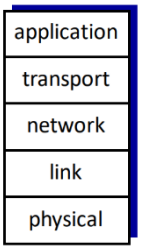
Architettura di livelli

La comunicazione che avviene tra due o più dispositivi in modo strutturato si chiama **pila protocollare** (*protocol stack*), poiché stiamo facendo riferimento all'organizzazione gerarchica in **livelli**, o **strati**, di oggetti chiamati protocolli di rete, che offrono dei servizi ai livelli superiori.



Per capire meglio è possibile analizzare il sistema di azioni compiute quando si vola con una linea aerea: acquistare il biglietto, consegnare i bagagli, andare al gate d'imbarco e infine raggiungere l'aeroplano. Il velivolo decolla e si dirige verso la propria destinazione. Dopo l'atterraggio, scendere e ritirare i bagagli. Questo esempio mostra diverse analogie con la rete di calcolatori, infatti, la linea aerea trasporta i passeggeri da una località di partenza a una di destinazione, così come in Internet un pacchetto viene trasportato da un host di partenza a uno di destinazione. La *Figura* presenta le funzionalità di una linea aerea divise in livelli. Si noti che ciascuno di essi, combinato con quelli inferiori, implementa dei **servizi**. Ogni livello fornisce il proprio servizio effettuando determinate azioni all'interno di questo (per esempio, a livello di gate, l'entrata e l'uscita dei passeggeri dall'aereo) e utilizzando i servizi del livello immediatamente inferiore (proseguendo nell'esempio, l'utilizzo del servizio di trasferimento dei passeggeri dalla pista di decollo alla pista d'atterraggio, servizio proprio del livello di decollo/atterraggio).

Stratificazione dei livelli



La **pila protocollare** di Internet consiste di cinque livelli: fisico, collegamento, rete, trasporto e applicazione.

Lo strato **fisico** è l'insieme di funzioni dei protocolli che servono a trasmettere i bit sul mezzo trasmissivo (*esempio fibre ottiche, rame, comunicazione wireless, ecc.*), ovvero trasferire i singoli bit del frame da un nodo a quello successivo.

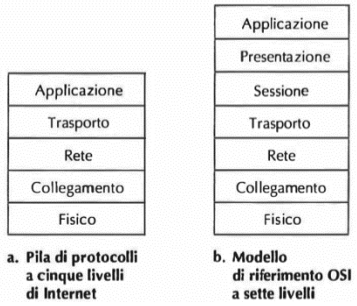
Lo strato di **collegamento** (*link layer*) si occupa del trasferimento dei frame in modalità affidabile (dà una garanzia che questi vengono consegnati correttamente).

Lo strato di **rete** si occupa di trasferire i pacchetti a livello di rete, detti datagrammi, da un host a un altro, attraverso due operazioni: l'indirizzamento (verso chi) e l'instradamento (in che modo vengono trasferiti).

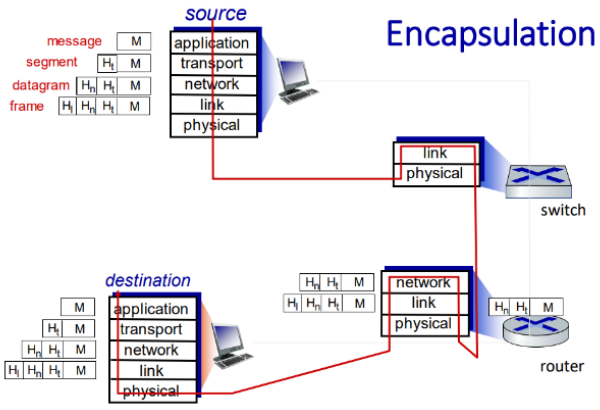
Lo strato di **trasporto** trasferisce i messaggi del livello di applicazione tra punti periferici gestiti dalle applicazioni.

Lo strato di **applicazione** è la sede delle applicazioni di rete (*WhatsApp, le mail, ecc.*) e dei relativi protocolli.

Modello OSI Il modello OSI (Open System Interconnection) proposto da ISO negli anni '70 prevedeva di rappresentare non gli specifici protocolli, che possono migliorare ed evolvere, quanto piuttosto il fatto che ogni strato ha delle funzioni specifiche organizzate gerarchicamente e offra un servizio allo strato superiore, il quale a sua volta offre un servizio avvalendosi di quello sottostante. Aggiungendo il livello di presentazione e sessione.



Incapsulamento



Consideriamo un **messaggio** a livello di applicazione (ad esempio, un'e-mail, un file, un video), che deve essere inviato da un host mittente ad un host destinatario. Questo messaggio viene passato al livello di trasporto, dove viene unito a delle informazioni aggiuntive chiamate **informazioni di intestazione a livello di trasporto**. Queste includono dettagli necessari per il corretto recapito del messaggio, come l'identificazione dell'applicazione destinataria o la rilevazione degli errori. L'insieme del messaggio a livello di applicazione e delle informazioni di intestazione a livello di trasporto costituisce ciò che viene chiamato **segmento a livello di trasporto**. Il segmento a livello di trasporto viene quindi passato al livello di rete, che aggiunge ulteriori informazioni di intestazione specifiche di questo livello. Queste includono gli indirizzi dei sistemi di origine e destinazione. L'insieme del segmento a livello di trasporto e delle informazioni di intestazione a livello di rete costituisce ciò che viene chiamato **datagramma a livello di rete**. Infine, il datagramma a livello di rete viene passato al livello di collegamento, che aggiunge le sue informazioni di intestazione per creare un **frame a livello di collegamento**. In ogni livello, il pacchetto è composto da **intestazione** (*header*, che contiene le informazioni necessarie per l'elaborazione del pacchetto) e **payload** (il contenuto effettivo del messaggio, tipicamente proveniente dal livello superiore).

Malware Il termine malware (software dannoso) indica qualunque software che agisca contro l'interesse dell'utente. Oltre al computer o al dispositivo infetto, il malware può colpire anche tutti i dispositivi con cui questo comunica.

RAPPRESENTAZIONE DIGITALE DELL'INFORMAZIONE – Addendum

Le informazioni che quotidianamente trasmettiamo possono presentarsi in due "forme": **informazione a blocchi** e **stream**. Nel primo caso consideriamo file, ovvero dati che naturalmente sono strutturati in unità indipendenti chiamate blocchi, come ad esempio messaggi di testo, immagini JPEG, ecc. con una dimensione ben precisa sottoforma di numero di bit per blocco. Nel secondo caso la modalità streaming, con la quale l'informazione viene prodotta e trasmessa in un sistema di telecomunicazioni, la porta a presentarsi in forma di flusso continuo di dati, ovvero bit. Quando si considera questa "forma" si parla di bit-rate, ovvero si misura la quantità di bit prodotti dalla sorgente in una unità di tempo.

Avendo parlato precedentemente di delay (ritardo) di trasferimento di un messaggio, è possibile allargare questo discorso, in particolar modo circa il delay minimo, alle tecniche di trasmissione e compressione dei dati. Infatti, quando si trasmettono le informazioni è necessario quantificare i dati che si sceglie di inviare in modo tale da preservare l'integrità delle informazioni stesse. Per tal motivo spesso si preferisce comprimere l'informazione originale e successivamente decomprimerla, una volta concluso l'invio. Detto ciò, è possibile introdurre il **rapporto di compressione** (Compression Ratio, R_c), ovvero il rapporto tra la lunghezza dei dati originali (bits file originale) e la lunghezza dei dati compressi. Ovviamente, tanto più io riesco a comprimere il blocco, tanto più aumenta il rapporto (inversamente proporzionale), tenendo presente che se riduco la quantità di dati trasferiti migliora il rapporto L/C.

Esistono principalmente due algoritmi di compressione dati, che ricordiamo avere il compito di ridurre il numero di bit necessari alla rappresentazione dell'informazione:

Senza perdita (Lossless): non perdo le informazioni relative ai dati compressi, questi una volta decompressi mi restituiscono l'informazione originale ricostruita esattamente com'era (zip, GIF, fax);

Con perdita (Lossy): di loro natura comprimono l'informazione ma quando questa viene decompressa qualcosa si è perso, il compressore ha fatto un lavoro di taglio laddove poteva farlo cercando di mantenere ragionevole l'operazione fatta. Tante volte si comprimono le immagini e si perde la loro qualità, conservandone però il soggetto (JPEG).

Esempi di informazione a blocchi

Tipo	Metodo	Formato	Originale	Compressed Ratio
Text	Zip	ASCII	Kbyte-Mbyte	$2<R_c<6$
Fax	CCITT Group 3	A4 page 200x100 pixel/in^2	256 kbyte	5-54 kbyte ($5<R_c<50$)
Immagine a Colori	JPEG	8x10 in^2 photo 400^2 pixel/in^2	38.4 Mbyte	1-8 Mbyte ($5<R_c<30$)

Per le informazioni stream abbiamo flussi di bit che si avvicendano nel tempo, l'esempio classico è quello relativo ad un segnale vocale che nella sua forma originale è di tipo analogico.

Segnale analogico, trasmette i dati sotto forma di onda continua che cambia nel tempo ed è di tipo sinusoidale, essa può eventualmente essere scomposta.

Segnale digitale, trasporta informazioni o dati in forma binaria, ovvero sotto forma di bit, e viene definito un segnale a tempo discreto non continuo.

I segnali analogici dipendono da un parametro, li trasformo in segnali binari e ottengo un flusso di bit. *Qual è il risultato?* Ho il segnale, lo leggo in certi istanti temporali e trasformo le letture in bit, ovviamente c'è un periodo fisso in cui si va a leggere questo segnale. Affinché questa digitalizzazione sia possibile, è necessario introdurre la tecnica del campionamento. Consiste nel convertire un segnale continuo nel tempo, oppure nello spazio, in un segnale discreto, valutandone l'ampiezza a intervalli temporali o spaziali solitamente regolari. In questo modo, a seguito di una successiva operazione di quantizzazione e conversione, è possibile ottenere una stringa digitale (discreta nel tempo o nello spazio e, nell'ampiezza) che approssimi quella continua originale.

Esempi di codifica

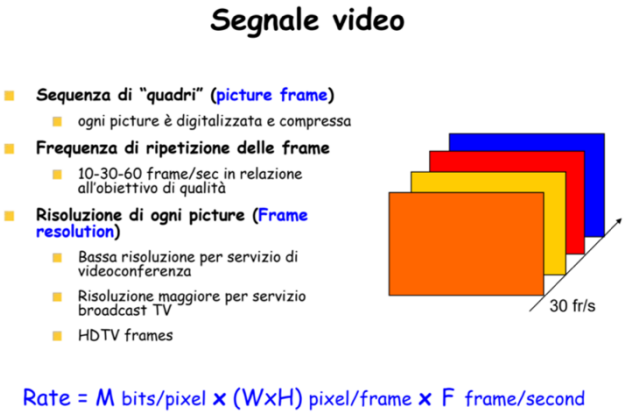
Codifica vocale (Telefonia), vale la pena trasmettere ogni volta che una persona non parla? No, per tal motivo è nato il rivelatore di pause, uno strumento che non trasmette quando ci sono silenzi. Nel corso degli anni sono nati nuovi codificatori di sorgente in grado di effettuare la "soppressione dei silenzi", riescono a captare la mancanza di attività vocale e di conseguenza non generano pacchetti di descrizione vocale. Sul canale vengono trasmessi pacchetti di descrizione del silenzio risparmiando così banda e riducendo l'interferenza. I silenzi vanno soppressi senza dare la sensazione di linea caduta. Sulla base costante aggiungo le differenze, così da migliorare il bit-rate, questa esigenza di compressione è emersa con la presenza dei telefoni cellulari che hanno una banda molto ridotta. Quando si sente la voce metallica il sintetizzatore vocale non ha informazioni a sufficienza per trasmettere l'informazione raffinata con la nostra vera e propria voce.



CD Audio, i numeri sono diversi perché devo leggere 44000 volte al secondo, a differenza della voce, 8000 volte al secondo, perché? Ho necessità di cogliere molte più peculiarità ed effetti. Tipica compressione sul segnale audio: mp3.

Segnale video. Il segnale video digitale è costituito da una sequenza di fotogrammi o frame, ognuno dei quali può essere un'immagine in scala di grigi o a colori.

Cosa succede nel mondo video? Abbiamo una lettura e successiva cattura delle immagini, ovvero la cosiddetta sequenza di "quadri" (picture frame), ognuna delle quali deve essere codificata in digitale. I frame si muovono con una certa velocità che tiene conto del tempo di permanenza dell'immagine sulla retina (esempio libretto di immagini che "scorro" tra le dita), dunque si sceglie l'obiettivo di qualità delle immagini e si parla di 10/30/60 fr/s (frame al secondo).



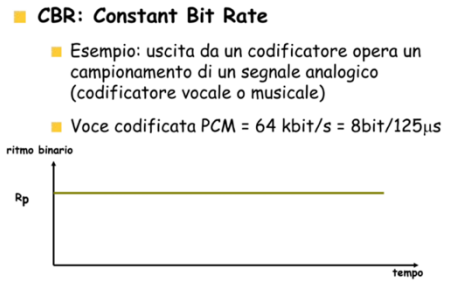
La decodifica avviene in orizzontale e verticale, tenendo conto di larghezza e altezza dell'immagine, la necessità è quella di leggere tutti i pixel (WxH). Quello che si nota è che più aumenta il numero di pixel per frame (risoluzione) più aumenta il bit-rate. Trasmetto un'immagine, un frame portante, ovvero quello principale e successivamente una serie di bit non completi e dopo un po' ricorreggo l'immagine con un frame portante.

Digital Video Signals

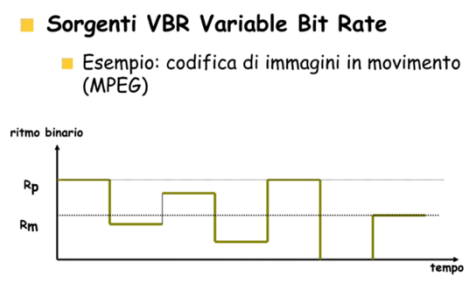
Tipo	Metodo	Formato	Originale	Compresso
Video Confer-enza	H.261	176x144 or 352x288 pix a 10-30 fr/sec	2-36 Mbit/s	64-1544 kbit/s
Full Motion	MPEG2	720x480 pix a 30 fr/sec	249 Mbit/s	2-6 Mbit/s
HDTV	MPEG2	1920x1080 a 30 fr/sec	1.6 Gbit/s	19-38 Mbit/s

Classificazione delle sorgenti

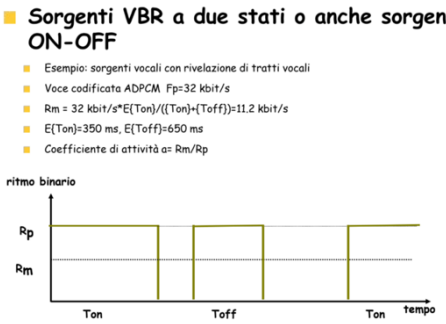
CBR, Constant Bit Rate I flussi informativi hanno bit-rate costante, la rete deve fornire un canale con banda almeno uguale al bit-rate della sorgente. Ad esempio, il segnale vocale ha un flusso pressoché costante, il che implica nel grafico ritmo binario-tempo una retta orizzontale



VBR, Variable Bit Rate I flussi informativi hanno bit-rate variabile nel tempo, la rete deve supportare in modo efficiente la variabilità del bit-rate. Ad esempio, le immagini in movimento, i video, hanno un flusso variabile che tiene conto del ritmo di picco e di quello medio. Posso dire qualcosa in più, ovvero il massimo contenuto informativo, il ritmo di picco, se nel video una scena è ricchissima di contenuti e informazioni, la codifico in tanti bit, nel tempo o varia di poco o perde di informazioni)



VBR a due stati o anche sorgenti ON-OFF Periodi in cui si trasmette al massimo e periodi in cui non si trasmette, anche qui ritmo di picco e ritmo medio, quest'ultimo misurando nel tempo quanto la sorgente è stata accesa e quanto spenta



CAPITOLO 3

Il transport layer, o livello di trasporto, è lo strato quattro della pila protocollare e si trova tra il livello di applicazione e quello di rete, stabilisce una comunicazione logica tra processi applicativi che vengono eseguiti su host differenti. Per far ciò si avvale delle unità informative (campo utile e intestazione). Il destinatario non entra nel payload, ovvero il campo utile, svolge le istruzioni dell'intestazione (header). Grazie a questo tipo di comunicazioni è possibile scambiare messaggi senza preoccuparsi dell'infrastruttura fisica utilizzata per il trasporto. È uno strato da estremo a estremo (**end to end**), il che vuol dire che viene implementato (esiste) solo agli estremi del collegamento, ovvero nei sistemi periferici. Ad esempio nel client e nel server con cui questo vuole comunicare.

Lato mittente Il livello di trasporto (transport layer) converte i messaggi che riceve da un processo applicativo in pacchetti noti come segmenti (transport-layer segment). Spezzando, se necessario, i messaggi in parti più piccole e aggiungendo a ciascuna di esse un'intestazione di trasporto. Il livello di trasporto, quindi, passa il segmento al livello di rete, dove viene incapsulato all'interno di un pacchetto a livello di rete (datagramma) e inviato a destinazione.

Lato ricevente Il livello di rete estrae il segmento dal datagramma e lo passa al livello superiore, quello di trasporto. Quest'ultimo elabora il segmento ricevuto, rendendo disponibili all'applicazione destinataria i suoi dati.

Esistono due protocolli messi a disposizione a livello di trasporto da Internet: **UDP** (user datagram protocol), che fornisce alle applicazioni un servizio non affidabile e non orientato alla connessione, infatti, non garantisce che i dati inviati arrivino intatti al destinatario, né tanto meno che arrivino; **TCP** (transmission control protocol), che offre un servizio affidabile e orientato alla connessione, infatti, evita che le connessioni intasino i collegamenti e i router, dividendo equamente la larghezza di banda del collegamento, ovvero controlla la congestione. Tutto parte dal servizio di consegna di IP tra i sistemi periferici, host-to-host, per arrivare a quello process-to-process, tra processi in esecuzione su sistemi periferici, passaggio definito multiplexing e demultiplexing.

L'**IP** è il protocollo a livello di Internet, il quale fornisce comunicazione logica tra host. IP è un modello di servizio *best effort*, ovvero fa del suo meglio per consegnare i messaggi tra host comunicanti, ma non assicura la consegna dei segmenti, né il rispetto del loro ordine, né tanto meno l'integrità dei dati.

Quando due processi su host differenti devono scambiarsi reciprocamente messaggi non lo fanno in modo "diretto" ma usano un intermediario, ovvero una socket, che si muove da un processo all'altro tramite la rete. Possono essere presenti contemporaneamente molteplici socket nell'host di ricezione, per questo è bene differenziarne il tipo: UDP o TCP. Una **socket** è un oggetto software che permette l'invio e la ricezione di dati tra host remoti tramite una rete o tra processi locali. È l'insieme degli indirizzi di livello tre (IP) e l'indirizzo di porta. Rappresenta un punto di accesso tra il processo e lo strato di trasporto. Viene distinta grazie ad una combinazione di informazioni che si trovano nell'header formato da righe di 32 bit, la prima costituita da 16 bit rappresenta il numero di porta della sorgente, i successivi 16 il numero di porta del destinatario.

Multiplexing Questa tecnica ha il compito di radunare frammenti di dati da diverse socket sull'host di origine, e incapsulare ognuno di essi in segmenti, aggiungendo intestazioni a livello di

trasporto, per poi passarli al livello di rete. *Esempio:* Anna raccoglie le lettere dai mittenti e le imbuca.

Demultiplexing Ha il compito di trasportare i dati dei segmenti a livello di trasporto verso la giusta socket. Viene effettuato a livello dell'host ricevente per individuare la giusta socket di ricezione. *Esempio:* Andrea riceve le lettere dal postino, legge il nome riportato sopra la busta e consegna la missiva al destinatario.

UDP Per un singolo trasporto, da A a B, o viceversa, servono solamente indirizzo IP e porta di destinazione, ecco perché se due segmenti hanno lo stesso indirizzo e lo stesso numero di porta di destinazione utilizzeranno la stessa socket.

TCP Una socket di questo tipo è identificata da quattro parametri: indirizzo IP di origine, numero di porta di origine, indirizzo IP di destinazione e numero di porta di destinazione. In ogni caso è impossibile accada una situazione simile a quella di UDP, ovvero che una socket venga utilizzata per due segmenti, proprio per la presenza di molteplici parametri.

Vantaggi di UDP

- *Controllo più fine a livello di applicazione su quali dati sono inviati e quando.* UDP dopo aver impacchettato i dati, trasferisce direttamente il segmento in rete, mentre TCP controlla la congestione del traffico e ritarda l'invio. Le applicazioni moderne spesso richiedono invio di dati in tempo reale e sono disposti a perderne alcuni.
- *Nessuna connessione stabilita.* UDP garantisce la mancanza di ritardi di connessione, proprio perché non c'è bisogno di stabilirla.
- *Nessuno stato di connessione.* A causa della necessità di effettuare a monte una connessione, TCP ne mantiene lo stato nei sistemi periferici (buffer, controllo congestione, parametri numero di sequenza e acknowledgment).
- *Minor spazio usato per l'intestazione del pacchetto.* TCP 20 byte, contro gli 8 di UDP.

Oggi la maggior parte dei distributori di servizio streaming usa HTTP/3 che garantisce affidabilità a livello applicativo e non necessita perciò di TCP ma di UDP.

Struttura segmenti UDP

L'intestazione contiene solo quattro campi di 2 byte ciascuno: le *porte* di sorgente e destinazione, la *lunghezza*, che specifica il numero di byte del segmento, infine il *checksum*, ovvero un metodo a disposizione dell'host ricevente per verificare se sono avvenuti errori nel segmento. Il **checksum** è un numero che si ottiene sommando i primi 16 bit della porta sorgente con i 16 bit della porta destinatario, si rifà la somma quando il pacchetto arriva, se le somme corrispondono non ci sono stati errori di trasmissione, altrimenti ci sono stati **flip**. Quando un bit partito in uno stato arriva in un altro (un 1 arriva uno 0, uno 0 arriva 1) allora si parla di flip. Tuttavia, UDP non fa nulla per risolvere l'errore, inoltre lungo un collegamento si potrebbe verificare un errore non rintracciabile mentre il segmento si trova nella memoria del router.

Trasferimento di dati con protocollo rtd2.0 (reliable data transfer)

Analizziamo come viene dettato un lungo messaggio al telefono. In uno scenario ipotetico, chi raccoglie il messaggio potrebbe dire "OK" dopo ogni frase che ha sentito, compreso e memorizzato. Se la persona che prende nota non capisce una frase, chiede di ripeterla. Questo

protocollo di dettatura dei messaggi usa notifiche (**acknowledgement**) positive ("OK") e notifiche negative ("Per favore, ripeti") NAK. Tali messaggi di controllo permettono al ricevitore di far sapere al trasmettitore che cosa sia stato ricevuto correttamente e cosa no, chiedendone quindi la ripetizione. Nel contesto di una rete di calcolatori, i protocolli di trasferimento dati affidabili basati su ritrasmissioni sono noti come protocolli **ARQ** (automatic repeat request). Per gestire la presenza di errori nei bit i protocolli ARQ devono avere tre funzionalità aggiuntive.

> *Rilevamento dell'errore:* innanzitutto, è richiesto un meccanismo che consenta al destinatario di rilevare gli errori sui bit, tale tecnica necessita dell'invio di bit extra tra mittente e destinatario.
>
> *Feedback del destinatario:* le risposte di notifica positiva (**ACK**) e negativa (**NAK**). In linea di principio, tali pacchetti potrebbero essere costituiti da un solo bit: 0 per NAK e 1 per ACK.
>
> *Ritrasmissione:* un pacchetto ricevuto con errori sarà ritrasmesso dal mittente.

Il mittente non invia nuovi dati finché non è certo che il destinatario abbia ricevuto correttamente il pacchetto corrente. È proprio per questo comportamento che i protocolli quali rdt 2.0 sono noti come protocolli **stop-and-wait**. Nel caso il cui il mittente riceva un pacchetto NAK è sufficiente inviare nuovamente il pacchetto di dati corrente. In realtà non è una soluzione, infatti il feedback è a tutti gli effetti un messaggio che può perdersi, frammentarsi o metterci troppo tempo ad arrivare, rappresentando una fonte di inaffidabilità, in questo modo il destinatario potrebbe ricevere pacchetti duplicati. Il destinatario non sapendo se l'ultimo ACK o NAK inviato sia stato ricevuto correttamente dal mittente, non può sapere a priori se un pacchetto in arrivo contenga dati nuovi o duplicati.

Una soluzione consiste nell'aggiungere un campo al pacchetto, un **numero di sequenza** col quale il mittente è obbligato a numerare i propri pacchetti. Al destinatario sarà sufficiente controllare questo numero per sapere se il pacchetto ricevuto rappresenti o meno una ritrasmissione. Infatti, l'ACK è numerato con l'unità informativa successiva, ovvero quella che il destinatario si aspetta di ricevere.

Trasferimento di dati con protocollo rtd3.0

Supponiamo ora che il canale di trasmissione, oltre a danneggiare i bit, possa anche smarrire i pacchetti. L'approccio adottato nella pratica è trovare un valore di tempo durante il quale è probabile i pacchetti vengano persi. Se non si riceve un ACK in questo lasso di tempo, il pacchetto viene ritrasmesso. Dunque, si implementa un meccanismo di ritrasmissione basato sul tempo di un contatore (countdown timer o **time out**) in grado di segnalare al mittente l'avvenuta scadenza del pacchetto nel lasso di tempo stabilito. È necessario stare attenti al RTT (round trip time = il ritardo di propagazione andata e ritorno), ovvero il tempo impiegato dal pacchetto per essere inviato e ricevuto dal destinatario, sommato al tempo necessario affinché il mittente riceva il feedback.

Trasferimento di dati con il metodo pipeline

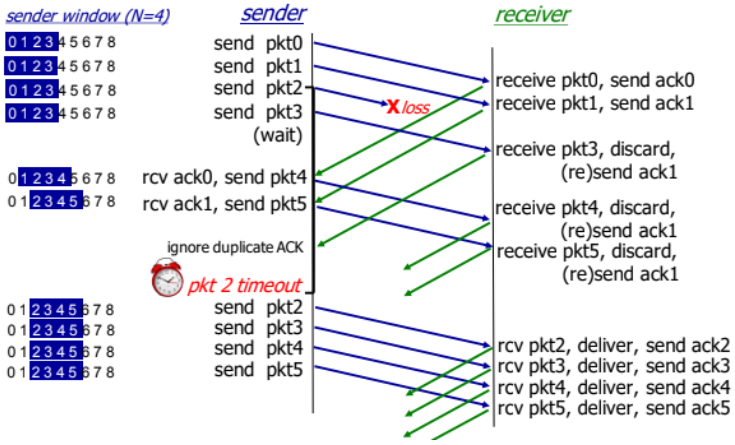
Permette di inviare più pacchetti insieme senza aspettare il feedback. Quindi è un metodo opposto al protocollo stop-and-wait. I protocolli che utilizzano questo metodo sono il Go-Back-N e Selective Repeat.

Protocollo Go-Back-N

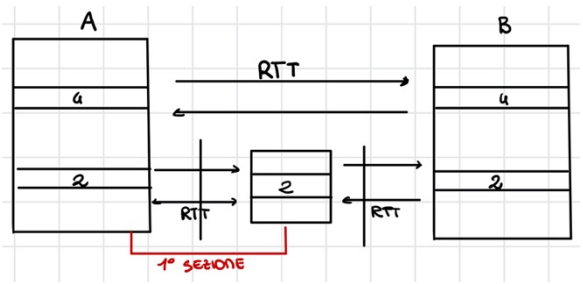
Il protocollo Go-Back-N, utilizzando il metodo pipeline, invia più pacchetti senza attendere per ciascuno di essi il pacchetto risposta. Viene detto protocollo a finestra scorrevole (**sliding-window protocol**), in quanto fa uso di un buffer di invio di pacchetti di dimensione N, il quale dopo aver spedito ogni pacchetto trasla la finestra di un'unità, fino a inviare tutti i pacchetti del buffer. Il GBN invia pacchetti solo in maniera ordinata, se uno si perde, quelli successivi vengono rifiutati e successivamente rinviati a partire dal pacchetto perso. Per capire se un pacchetto è stato perso il GBN è dotato di un temporizzatore, impostato con il tempo di RTT, se un pacchetto supera il tempo RTT viene considerato perso. Questo processo che caratterizza il GBN se da un lato migliora notevolmente la sicurezza con la quale i dati vengono inviati, dall'altro rallenta molto il processo di trasmissione, rendendo il protocollo non troppo efficiente.

Se venisse ricevuto un ACK dal trasmettitore prima dello scorrimento della finestra, quello certificherebbe l'arrivo e la ricezione da parte del destinatario di un certo numero di pacchetti, allora la finestra si aggiornerebbe in corrispondenza di quell'ACK. Ecco spiegato il concetto di **ACK cumulativo**.

Esempio La slider window ha dimensione 4, il GBN inizia ad inviare i pacchetti e la finestra scorre. Il pacchetto 2 si perde, allora tutti i pacchetti della finestra successivi al 2, ovvero 3-4-5, vengono rifiutati e il trasmettitore per ognuno di essi riceve l'ACK dell'ultimo pacchetto inviato e ricevuto con successo, quindi ACK1. Il GBN va in time-out quando la finestra termina i pacchetti da inviare. Infatti, dopo aver rifiutato il 3-4-5, i pacchetti nel buffer sono esauriti. Allora il GBN rinvia in ordine il 2-3-4-5.



Il meccanismo di consegna affidabile di pacchetti, nello strato quattro avviene tramite il protocollo Stop&Wait. Quando la comunicazione si svolge nello strato 4, gli host scambiano direttamente le informazioni tra di loro. Se invece la comunicazione avviene al livello 2, lo scambio non è diretto bensì i pacchetti devono passare per diversi nodi. Il RTT nello strato quattro dipende dal tempo impiegato per spostarsi da un sistema terminale ad un altro.



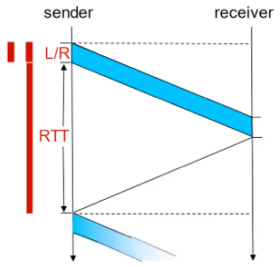
Nel caso dello strato 2, nella prima sezione, il RTT (round trip time) è uguale a 2d/v, dove d/v = τ, ovvero il ritardo di propagazione, d è la distanza tra due estremi della sezione e v è la velocità di trasmissione del pacchetto nel mezzo.

Considerando il calcolo dell'**efficienza** dei protocolli (gli elementi della tabella in percentuale), in base al bit rate R, ovvero 1 Mbit/s o 1 Gbit/s e al ritardo di 1ms, che è pari al RTT di 200km, ecc. si vede che aumentando il RTT l'efficienza diminuisce, stessa cosa accade aumentando il bit-rate.

Esempio: Impatto del prodotto banda-ritardo

- n_f=1250 byte =10000 bits, n_a=n_o=25 byte = 200 bit

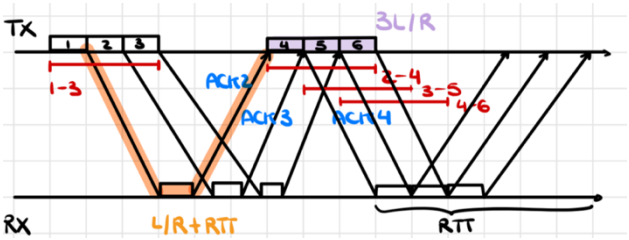
2xDelayxBW Efficiency	1 ms 200 km	10 ms 2000 km	100 ms 20000 km	1 sec 200000 km
1 Mbit/s	10^3 88%	10^4 49%	10^5 9%	10^6 1%
1 Gbit/s	10^6 1%	10^7 0.1%	10^8 0.01%	10^9 0.001%



Quando aumenta il bit-rate significa che il pacchetto viene inviato molto più velocemente e dunque, nel disegno, lo spessore della striscia celeste è molto minore. La formula per calcolare l'**efficienza** di una comunicazione nello **Stop&Wait** è la seguente:

$$U = \frac{\frac{L}{R}}{RTT + \frac{L}{R}} <=> E = \frac{L}{(RTT * R) + L}$$

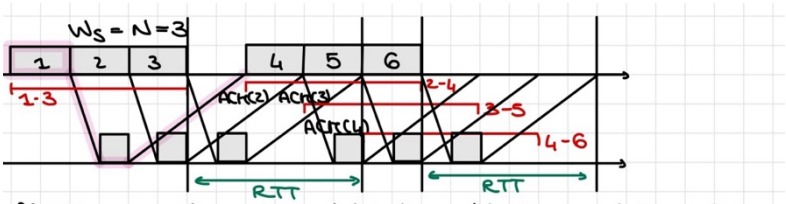
Essendo inversamente proporzionale al prodotto banda-ritardo (RTT*R), per R grandi, quindi per grandi larghezze di banda, il protocollo Stop&Wait non risulta efficiente. Per migliorare l'efficienza bisogna usare il protocollo Go-Back-N, poiché piuttosto che mandare un pacchetto alla volta, il protocollo in questione, usa il seguente schema con trasmettitore e ricevitore.



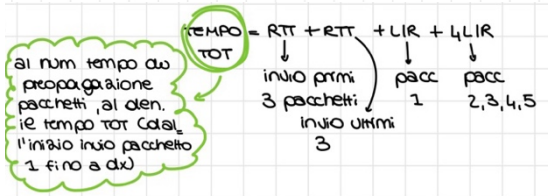
Si propone di mandare N pacchetti uno dopo l'altro, in questo caso tre, con i relativi ACK, ricordando che questi sono numerati in base al pacchetto che il RX si aspetta di ricevere. In questo caso avendo una finestra lunga 3 (1-3), una volta ricevuto il primo ACK si fa scorrere la finestra aggiornando l'estremo inferiore con il numero dell'ACK, ovvero il pacchetto che il ricevitore aspetta. Una volta arrivato l'ACK (3), intuitivamente, la finestra scorrerà e avremo 3-5.

L'efficienza, in questo caso, si trova ponendo al numeratore 6L/R e al denominatore il tempo totale.

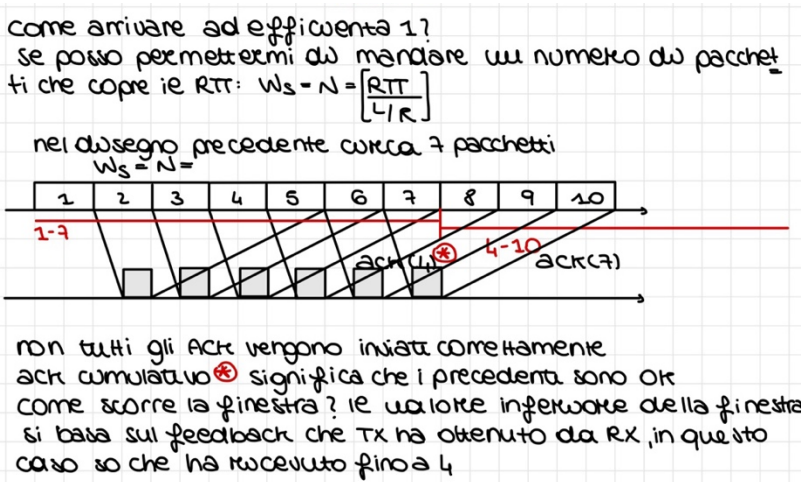
$$T = \text{tempo trasferimento} = \frac{\text{pacchetti}\ \frac{6L}{R}}{RTT + L/R + RTT + 4L/R}$$



finestre a scorrimento perché 2 l'ho già inviato, così come 3, sto dando per buono che il nuovo pacchetto sa che 2 e 3 sono stati trasmessi, allora sarà il 4. Quelle regole rosse sono le finestre corrispondenti alle ACK!

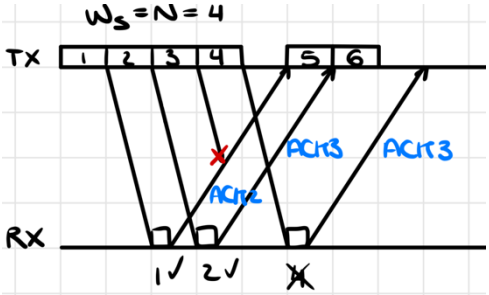


Quanto deve essere estesa la finestra, ovvero di quanti N pacchetti deve essere composta, affinché la trasmissione avvenga senza soluzione di continuità, ovvero l'efficienza è massima (=1)? Dovrei inviare il maggior numero di pacchetti in modo da riempire il tempo secondo la regola **NT>=RTT**, ovvero N>= RTT/(L/R), se il numero non è un intero si approssima per eccesso.



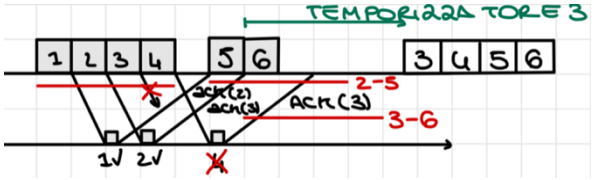
Non è necessario ogni volta inviare ACK al sender, talvolta è possibile fare uso di un ACK cumulativo che notifica al sender l'invio corretto di un tot di pacchetti.

Tutto ciò è garantito se la trasmissione è affidabile, nel caso contrario può succede che un pacchetto non arrivi, immaginiamo di avere una finestra di quattro pacchetti dove il terzo viene perso. Nonostante ciò, l'ACK (2) fa scorrere la finestra, così come l'ACK(3), in questo modo vengono inviati pacchetti fino a 5. Il protocollo Go Back N accetta pacchetti solo in sequenza, per tal motivo una volta compreso che manca un pacchetto, rifiuta il 4 perché non in sequenza con il numero 3. Continua a richiedere quello che manca all'appello e il protocollo fallisce.



Per risolvere questo problema è sufficiente implementare un temporizzatore per ognuno dei pacchetti, impostati con il tempo RTT per ogni pacchetto, così che, se uno di essi supera il tempo prestabilito, i successivi vengono rifiutati e rinviati tutti, sia quello perso che i successivi.

Nell'esempio sopra trattato, il temporizzatore del pacchetto 1 si annulla quando arriva l'ACK (2), nel caso del pacchetto 3, il temporizzatore ha un ruolo perché allo scadere di questo, se non si è ricevuto l'ACK, si rimanda il pacchetto.



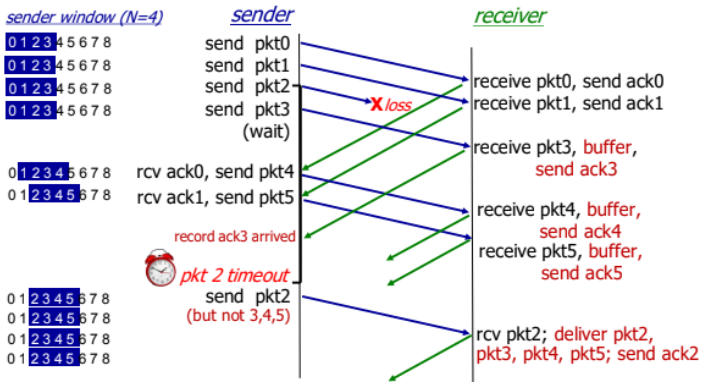
Da qui il nome Go Back N, una volta scaduto il temporizzatore 3 vengono reinviati 4 pacchetti, ovvero 3, 4, 5 e 6. Perché è indicato dalla finestra 3-6, ovvero le unità informative non riscontrate.

Questo processo che caratterizza il Go Back N se da un lato migliora notevolmente la sicurezza con la quale i dati vengono inviati, dall'altro rallenta molto il processo di trasmissione, rendendo il protocollo non troppo efficiente.

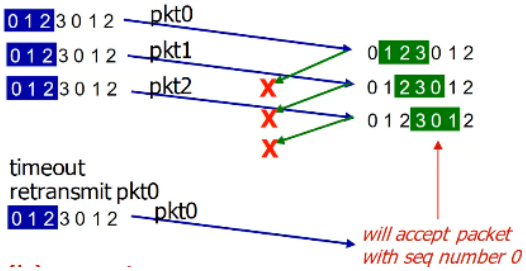
Protocollo Selective Repeat

Il GBN ha il problema che, quando la finestra ha dimensioni molto grandi, se un pacchetto si dovesse perdere, significherebbe inviare nuovamente tanti pacchetti, portando a numerosi ritardi e alla saturazione del processo.

Una soluzione è il protocollo **Selective Repeat**, che funziona sempre con una slider window e un temporizzatore, ma quando un pacchetto si perde, continua ad inviare i successivi e i rispettivi acknowledgement. I pacchetti che devono essere ricevuti in ordine non possono però essere conservati direttamente in questo modo, allora vengono inseriti in un buffer di arrivo nel ricevitore. Quando il pacchetto perso va in time-out, il mittente lo invia nuovamente dopo lo scadere del temporizzatore. Quando è arrivato al ricevitore, questo svuota man mano il buffer (in ordine).



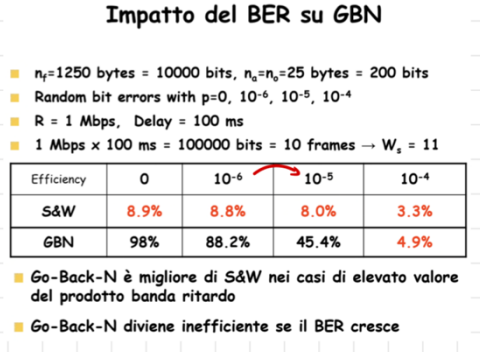
Dilemma – Se ad esempio scegliessi di numerare i pacchetti con una sequenza in base 4, dunque utilizzando due bit e i numeri da 0 a 3, nel caso seguente mi accorgerei di un problema. Il mittente invia il pacchetto 0 che viene accettato dal ricevitore, il quale fa scorrere la finestra di ricezione, quella verde, da qui posso capire che si aspetta di riceve i pacchetti da 1 a 3. Non parte l'ACK e dunque non posso far scorrere la finestra lato mittente. Mando il pacchetto 1 e, come prima, viene accettato dal ricevitore che fa nuovamente scorrere la finestra di ricezione, ma anche qui si perde l'ACK e la finestra lato mittente rimane la stessa. Stessa cosa accade con il pacchetto 2, dopodiché il mittente prova a inviare nuovamente il pacchetto 0 e, poiché 0 è presente nella finestra del ricevitore, viene accettato ma questo è un errore perché avrò un pacchetto duplicato. Lo 0 che il destinatario si aspetta di ricevere non è lo stesso che il mittente ha spedito.

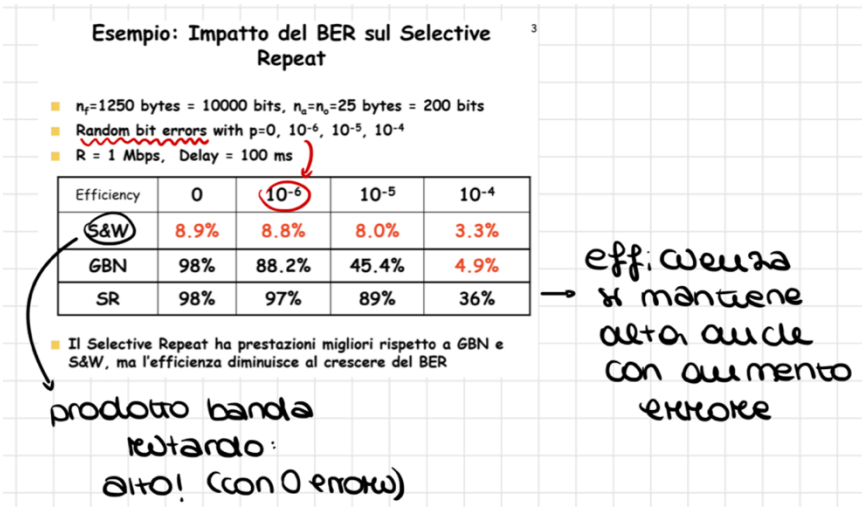


La **soluzione** è la seguente: la finestra in ricezione deve essere di dimensione inferiore rispetto a quella della finestra del mittente, dunque N-1 numeri, dove N è la dimensione della finestra di trasmissione. Prima anziché una finestra di 3 elementi, avrei dovuto averne una di dimensione 2.

Ricapitolando l'efficienza dei tre protocolli

Nel protocollo Stop&Wait c'è un'efficienza elevata quando il prodotto banda-ritardo è molto piccolo. Il Go Back N ha un'efficienza che dipende dal numero di unità informative che posso inviare una volta e dal prodotto banda-ritardo. Dunque, dipende dalla lunghezza della finestra e da quante volte rinvio un'unità.



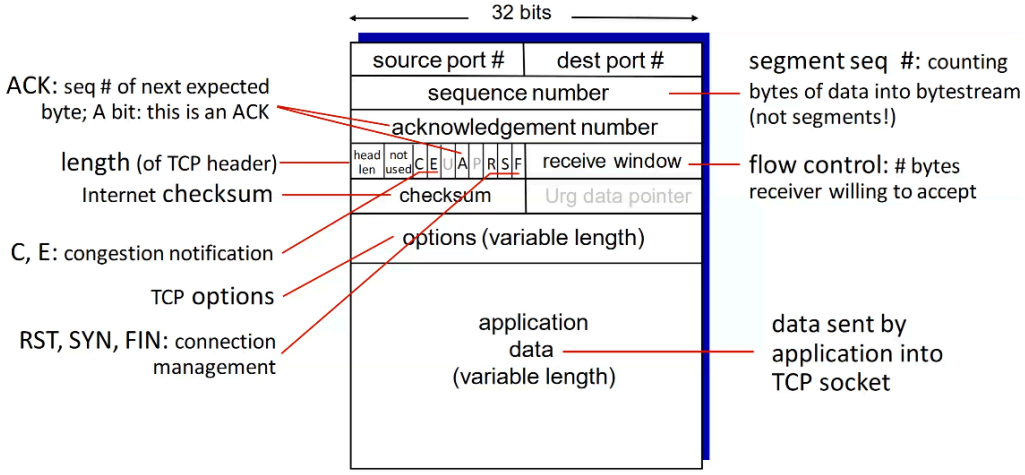


Nel caso di un errore pari a 0: lo Stop&Wait ha un'efficienza molto bassa, pari all'8%, se il prodotto banda-ritardo è alto, per avere efficienza 1, sia per il protocollo Go Back N che per il Selective Repeat dimensioniamo il valore della finestra esattamente maggiore o uguale al numero di pacchetti che entrano nel RTT, e non interessa più il prodotto banda-ritardo.

Se l'errore aumenta, vuol dire che ho una maggiore probabilità di rimandare l'unità informativa, per il protocollo Stop&Wait decresce il valore dell'efficienza. Per il Go Back N avrò uno spreco di tempo e dunque la percentuale decresce in modo abbastanza veloce, per il Selective Repeat c'è una diminuzione dell'efficienza, ma la velocità di diminuzione è minore rispetto al Go Back N perché dovrò reinviare solo l'unità o le unità informative perse, non tutta la finestra.

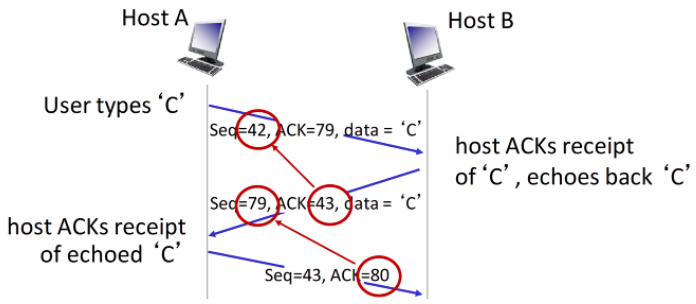
PROTOCOLLO TCP

Due host che si parlano da estremo a estremo della rete e mandano contemporaneamente le unità informative nelle due direzioni utilizzano un tipo di comunicazione definita **full duplex** e una connessione stabilita da TCP che si avvale del protocollo **Selective Repeat**.



Le unità dati hanno un'intestazione composta da molti campi distribuiti in righe da 32 bit ciascuna, l'header ha una grandezza variabile che al massimo arriva a 32 bit. Il **numero di sequenza** serve a numerare le unità informative; infatti, nel protocollo i pacchetti sono numerati, così come il numero di **ACK**. Le unità dati del TCP, ovvero i segmenti, non sono numerati, quanto piuttosto il primo byte di quest'ultimo lo è. Il flusso di dati che riguarda TCP non è un flusso di segmenti quanto piuttosto un flusso di byte, ecco perché l'ACK viene "numerato" contenendo il numero del byte che il destinatario si aspetta di ricevere.

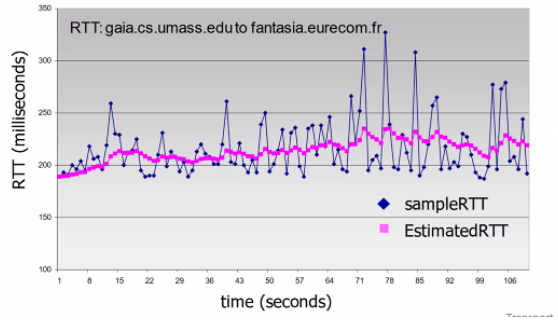
Osservando questo esempio di scambio di byte tra due host, A e B, si traduce nel seguente procedimento: l'host A invia 42 e si aspetta di ricevere 79 da B, B invia 79 e si aspetta di ricevere 43 da A, e così via…



Tra quello che invia A e quello che invia B non c'è nulla di correlato, infatti, quando si dice che le numerazioni devono essere progressive si intende rispetto al singolo host.

Se il time-out fosse sovradimensionato rallenterei la trasmissione, viceversa impedirei una corretta trasmissione dei pacchetti, dunque per dimensionarlo bene è necessario conoscere il RTT, ovvero il round trip time, tempo di andata e ritorno. TCP non fissa il numero di N pacchetti da inviare, né tanto meno il RTT, quanto piuttosto stima in tempo reale il RTT dei segmenti e successivamente calcola l'**EstimatedRTT**, ovvero una media variabile con una pesatura esponenziale.

Ottenendo il seguente risultato: `EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT`



Si può concludere che l'EstimatedRTT segue l'andamento del RTT istante per istante con valori smussati.

La **deviazione standard** serve per capire quanto è affidabile l'EstimatedRTT ed è anch'essa pesata in modo esponenziale. Si utilizza per calcolare il time-out come somma dell'EstimatedRTT e di quattro volte la deviazione standard.

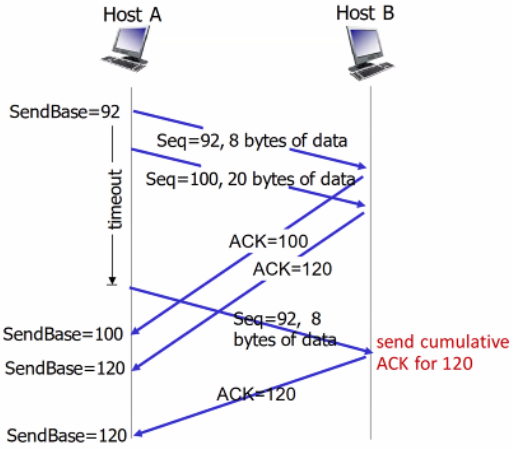
`TimeoutInterval = EstimatedRTT + 4*DevRTT`

estimated RTT — "safety margin"

- **DevRTT**: EWMA of **SampleRTT** deviation from **EstimatedRTT**:

`DevRTT = (1-β)*DevRTT + β*|SampleRTT-EstimatedRTT|`

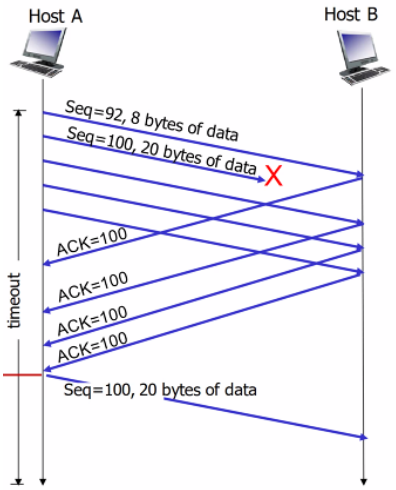
(typically, β = 0.25)



TCP fa uso dell'**acknowledgement cumulativo**, ovvero quando viene mandato un ACK significa che tutti i pacchetti precedenti sono arrivati con successo, ad esempio l'ACK (4) indica che il pacchetto 3, che la stazione trasmittente ha inviato, è stato ricevuto correttamente; implicitamente però l'ACK(4) cumulativo sta ad indicare che anche i pacchetti 3, 2, 1, 0 sono stati ricevuti e non sono andati persi.

Questo indica un valore di timeout troppo basso. Infatti, il pacchetto 100= 92+8 e il 120= 100+20 sono arrivati però l'ACK è arrivato ad A dopo il timeout. Nonostante ciò, grazie l'ACK cumulativo, B rileva che il pacchetto 92 è già stato ricevuto e per questo rinvia l'ACK 120, ovvero il valore dell'ultimo pacchetto ricevuto.

Supponendo di avere un time-out abbastanza lungo, l'host A non può reinviare l'unità informativa prima dello scadere di questo. Tuttavia, il TCP si avvale del **fast retransmit**, in modo da accelerare il processo di ritrasmissione del pacchetto mancante, dopo aver ricevuto un elevato numero di ACK (=3) rispettivi ad uno specifico segmento.

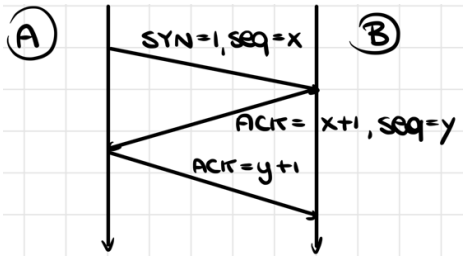


Il pacchetto 100 arriva a B e viene inviata l'ACK 100, che in realtà sarebbe 120 perché questo è il segmento mancante. Il pacchetto 120 si perde prima di raggiungere B. Dopodiché A continua ad inviare i pacchetti successivi al 120. Grazie all'ACK cumulativo l'host B mette nel suo buffer i pacchetti successivi al 120 e continua ad inviare l'ACK 100 (ovvero il numero dell'ultimo pacchetto ricevuto, per quanto studiato, avrei dovuto avere 120, ovvero il numero del segmento mancante). Dopo 3 tentativi e dopo 3 ricezioni dell'ACK 100, l'host A capisce che qualcosa non va e invia il pacchetto successivo al pacchetto 100, ovvero il pacchetto 120 il quale si era perso. Dopo di che B "ricostruisce il puzzle" e invia gli ACK dei pacchetti successivi.

Il TCP implementa due funzioni, una delle quali è il **flow control** (controllo di flusso). Chi riceve le unità dati potrebbe non avere la capacità di processarli velocemente, di conseguenza può permettersi di chiedere a chi trasmette di rallentare la trasmissione. Nell'header è presente un campo, la **receive window** (rwnd), la quale indica il numero di bit che il destinatario è disposto a ricevere. Ciò dipende dal buffer di ricezione, infatti questo ha una certa capacità e in base a quella è possibile chiedere al trasmettitore di rispettare un numero contenuto nella receive window, ciò vuol dire che il destinatario regola la trasmissione. Tanto più alto è il valore della receive window tante più informazioni posso ricevere.

Essendo TCP un protocollo con connessione, è necessario che i due host prima di inviare dati si mettano d'accordo attraverso la **3-way handshake**, a differenza di ciò che accade nella realtà. Infatti, se due persone voglio parlare si avrà la situazione di 2-way handshake. La connessione è a 3 vie (3 ways handshake): **(1)** A chiede a B di stabilire una connessione, **(2)** B accetta la richiesta di connessione e, **(3)** A invia i dati.

C'è un flag nell'header, ovvero un campo del segmento TCP che è composto da un solo bit, se è pari ad 1 "*leggi questo messaggio come un messaggio SYN*", se 0 no. **(1)** *SYN* (Synchronize Sequence Number) è un numero casuale estratto da chi vuole parlare, ovvero A, per stabilire la connessione con B, **(2)** SYN/ACK, B risponde con un ACK e con un SYN e, **(3)** ACK, A risponde con un ACK.



La chiusura della connessione avviene in due passaggi attraverso il flag FIN e la comunicazione si interrompe separatamente sui due lati: **(1)** A invia FIN a B e A chiude, **(2)** B invia FIN ad A e B chiude.

Controllo di congestione – In una rete attraversiamo molteplici router, per poter comprendere le cause ed i costi di congestione è bene analizzare tre diverse situazioni:

1. Due mittenti e un router con buffer illimitati.
   Abbiamo due host, A e B, connessi grazie ad un singolo router intermedio, immaginiamo che un'applicazione nell'host A stia inviando dati ad una frequenza di λ_in in byte/s. I pacchetti dall'Host A e dall'Host B passano attraverso un router e un collegamento uscente condiviso di capacità R. Se il buffer è infinito non ci sarà mai perdita di segmenti e, due connessioni che viaggiano in rete con protocollo TCP arrivano a dividersi in modo equo la banda (fair). Di conseguenza λ_in e λ_out saranno sempre uguali, ovvero ciò che entra nel buffer esce. Tuttavia, il throughput, ovvero λ_out arriverà a stabilizzarsi al valore R/2, ciò vuol dire che il ritardo di accodamento di questo router raggiunge R/2.
   Il grafico a destra mostra le conseguenze di operare al limite della capacità di collegamento: quando il tasso di arrivo dei pacchetti si avvicina alla capacità del collegamento, si ottengono lunghi ritardi di accodamento.

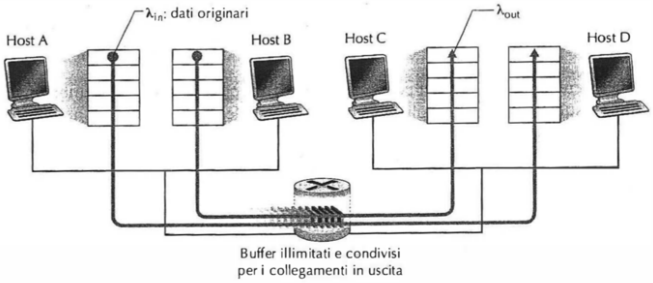
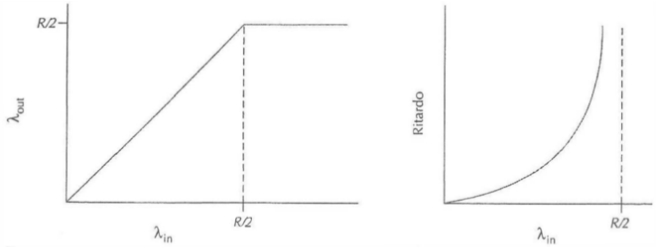
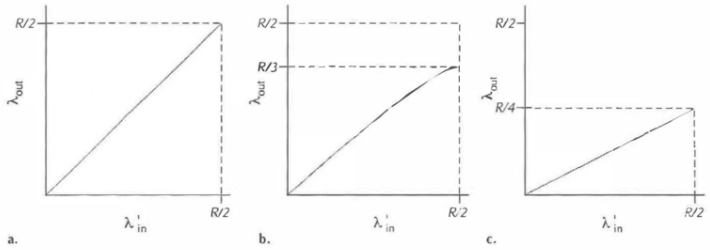


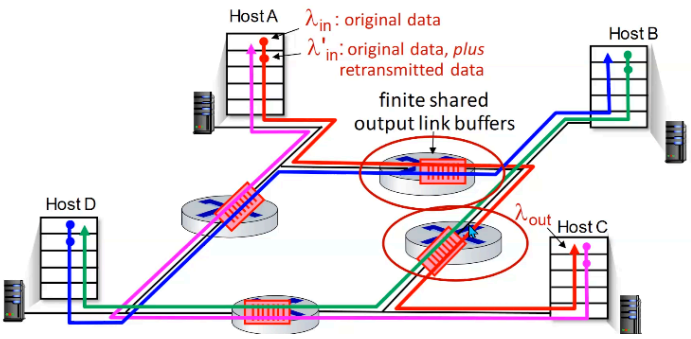
Figura 3.43 Scenario di congestione 1: due connessioni che condividono un hop con buffer illimitato.



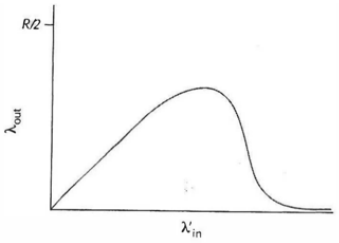
2. Due mittenti e un router con buffer limitati. Se il buffer ha dimensione finita esiste un momento in cui inizieranno a perdersi dei segmenti, avendo visto che, quando TCP comincia a perdere pacchetti si avvale della proprietà del protocollo Selective Repeat circa la ritrasmissione dei pacchetti mancanti, il λ_in sarà complessivamente composto da quello originale sommato ai segmenti ritrasmessi. Se l'host A trasmette solo quando il buffer ha spazio, le lamba sono identiche e non c'è smarrimento, allora la velocità di invio media dell'host non supera R/2 (situazione surreale, figura 3.46 a). Nella realtà il mittente ritrasmette solo quando è certo il pacchetto sia stato perso impostando un timeout lungo abbastanza per essere certo della perdita, come in figura 3.46 b, il costo della congestione si rispecchia nella necessità di ritrasmissioni. Oppure, col timeout, vengono ritrasmessi i pacchetti che hanno subito ritardi in coda ma non sono stati persi. In questo caso il costo di congestione è relativo alla ritrasmissione di pacchetti a causa di lunghi ritardi, ciò costringe il router ad usare la larghezza di banda per instradare copie inutili. Come in figura 3.46 c, il throughput assume il valore R/4 quando il carico offerto tende a R/2.



3. Quattro mittenti, router con buffer finiti e percorsi composti da più collegamenti. Consideriamo la comunicazione dall'host A verso l'host C e, quella dall'host B verso l'host D. A immette un certo λ_in, quando questo comincia a crescere, nel primo router cominciano ad esserci delle perdite di pacchetti, come nel caso b. precedente. Quando arriva nel buffer del secondo router, troviamo un λ_in meno aggressivo, ovvero quello proveniente da B che avrà maggiore banda rispetto a B. Osservando il grafico relativo al λ_in di A si comprende che vi è un effetto doppio rispetto al b. del caso precedente. Infatti, si perde il bit-rate, tende a 0, in corrispondenza del secondo buffer. Non poter controllare la congestione porta ad avere un abbassamento di throughput.

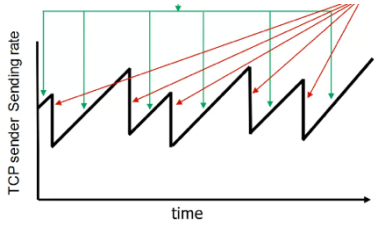


Nello scenario di traffico intenso considerato, ogni qualvolta un pacchetto viene scartato sul router del secondo hop (il termine "hop" si riferisce al passaggio di un pacchetto dati da un nodo o dispositivo di rete all'altro), il lavoro effettuato dal router del primo hop nell'instradamento del pacchetto verso il secondo router finisce per essere "sprecato".

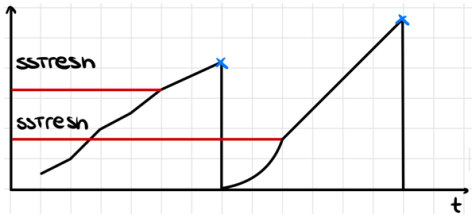


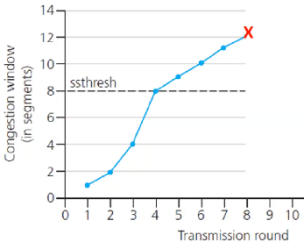
Pertanto, si parla di **congestione di rete** quando troppe sorgenti cercano di trasmettere una notevole quantità di dati ad una velocità talmente elevata che la rete non è in grado di gestirli. Andiamo incontro a due "sintomi": pacchetti smarriti per l'overflow nei buffer dei router, lunghi ritardi di accodamento nei buffer. TCP cerca di controllare questi costi di congestione con due metodi: end-to-end e assistito dalla rete. Nel primo caso la congestione è dedotta osservando le perdite e i ritardi nei sistemi terminali, nel secondo caso i router forniscono un feedback ai sistemi terminali, ovvero mandano dei messaggi di controllo che i router dovrebbero inoltrare alle applicazioni.

Questo secondo metodo non è stato implementato, a differenza del controllo end-to-end che si avvale dell'approccio **AIMD**, ovvero un modo attraverso cui TCP trasmette i propri dati in rete cercando di sondarne lo stato. Quando vengono inviati pacchetti, non conoscendo il bit-rate, TCP parte da un certo valore di bit-rate e lo aumenta, prova a mandare sempre più bit al secondo fino a raggiungere un valore che non va più bene. Quando c'è un evento che fa capire al mittente di aver causato una congestione TCP abbassa il bit-rate (verde aumento, rosso diminuisco). L'aumento additivo dipende dal **maximum segment size**, ovvero MSS, se c'è un evento di perdita il bit-rate viene dimezzato.



In realtà il protocollo prevede due parti, la prima è denominata **slow start** che consiste nell'aumento esponenziale del numero di pacchetti inviati ogni volta che viene ricevuto un ACK. L'aumento diventa lineare quando si raggiunge la soglia di throughtput denominata **ssthresh**, e si ferma una volta registrato un evento di perdita di pacchetti. Si torna al punto di partenza e si dimezza la ssthresh e il procedimento ricomincia. All'inizio del processo il valore di ssthresh è settato a metà della cwnd, ovvero **congestion window**, ogni volta che si raggiunge un picco con successiva perdita, la cwnd sarà pari al picco e la ssthresh alla sua metà. Ricorda che la velocità di invio del mittente è approssimativamente *cwnd/RTT* byte/s.





Ad esempio, in questo caso abbiamo la cwnd iniziale posta a 16 e dunque la ssthresh a 8, quando si ha perdita di pacchetti, ovvero in corrispondenza della x rossa, la cwnd sarà pari a 12 e, per ricominciare il protocollo, la ssthresh sarà posta a 6.

Fast recovery Abbiamo detto che esistono due motivi che causano la perdita di segmenti: errori di trasmissione e congestione, che provoca la perdita di più segmenti consecutivi. Quando viene perso un solo segmento continua a valere il principio per cui la ssthresh viene resettata a metà della cwnd ma, anziché ricominciare da capo, si parte direttamente dall'altezza pari al nuovo valore della ssthresh, evitando la fase slow start. In questo modo si ricomincia direttamente dalla fase additiva. Questa è la seconda parte dell'algoritmo.

Riassunto

MSS (Maximum Segment Size) – Rappresenta la dimensione massima di un segmento di dati che può essere inviato in un'unica trasmissione TCP ed è espressa in byte. I dati vengono divisi in segmenti di dimensioni non superiori a MSS prima di essere trasmessi sulla rete.

Slow Start Threshold (ssthresh) – Ssthresh è una soglia utilizzata per gestire il controllo di congestione in TCP. Durante la fase di slow start, la finestra di congestione di TCP cresce esponenzialmente fino a quando non raggiunge ssthresh. Dopo aver raggiunto questa soglia, la sua crescita diventa lineare.

TCP Tahoe – TCP Tahoe utilizza **slow start** e timeout per gestire la congestione. Quando si verifica una perdita, la finestra di congestione si riduce drasticamente, tornando a un valore iniziale (tipicamente 1 MSS) e successivamente inizia il processo di slow start. Il time-out di Tahoe è più aggressivo, aspettando di meno rispetto a Reno.

TCP Reno – TCP Reno estende le funzionalità di TCP Tahoe introducendo la "**fast recovery**" e migliorando la gestione delle perdite. Infatti, quando si verifica una perdita la cwnd è pari al picco e la nuova ssthresh alla sua metà. Anziché tornare ad 1MSS e salire come per slow start, si scende fino al nuovo valore di ssthresh e da lì parte la fase additiva lineare.

Oggigiorno esistono molte versioni di TCP come ad esempio TCP **cubico**, in questo caso, al posto di un incremento lineare ne avrò uno cubico man mano che ci si avvicina alla congestione.

*Com'è possibile che due host riescano a dividersi in modo equo, ovvero **fair**, la banda per mettere a punto le due connessioni?* Si dice che un meccanismo di controllo di congestione sia equo (**fair**) se la velocità trasmissiva media di ciascuna connessione è approssimativamente R/K, ovvero ciascuna connessione ottiene la stessa porzione di banda del collegamento, dove K sono le connessioni TCP, ciascuna con differente percorso end-to-end, ma che passano tutte attraverso uno stesso collegamento con capacità trasmissiva di R bps che costituisce il collo di bottiglia del sistema.

CAPITOLO 4

Analizzando lo strato di rete è possibile capire cosa c'è dietro il servizio di comunicazione host-to-host e che cosa lo fa funzionare; infatti, una parte del livello di rete è presente in tutti gli host e router della rete. In questo modo si completerà la panoramica relativa ai servizi che questo livello può offrire e i due approcci alla consegna dei pacchetti. Tra le funzioni principali ci sono quelle di inoltro, ovvero **forwarding**, e instradamento, ovvero **routing**. La prima implica il trasferimento di un pacchetto da un collegamento in ingresso a un collegamento in uscita all'interno di un singolo router. La seconda coinvolge tutti i router di una rete, le cui interazioni collettive, attraverso i protocolli di instradamento, determinano i percorsi seguiti dai pacchetti nel loro viaggio dal nodo di origine a quello di destinazione.

I pacchetti che viaggiano nella rete contengono differenti tipologie di **dati**: dati utente e le informazioni di controllo. Nel primo caso si fa riferimento al contenuto informativo che viene trasferito attraverso la rete, ad esempio quando si invia un'e-mail il testo rappresenta i dati utente nei pacchetti. Nel secondo caso si fa riferimento alle informazioni utili per guidare l'inoltro e l'instradamento dei pacchetti, ad esempio i protocolli di rete.

Un router ha un'**architettura** divisa come segue:

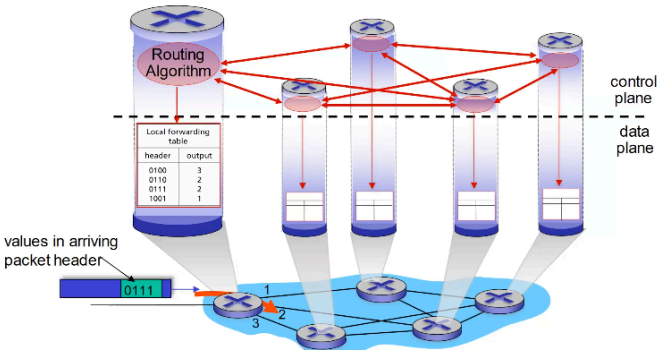
Porte di ingresso (input port): svolgono funzioni a livello fisico di terminazione di un collegamento in ingresso al router, funzioni a livello di collegamento e quella cruciale di ricerca, in modo che il pacchetto inoltrato nella struttura di commutazione esca sulla porta di uscita corretta. Con il termine "porta" si intendono le interfacce fisiche di input e output del router.

Struttura di commutazione (switching fabric): è quella struttura che connette fisicamente le porte di ingresso a quelle di uscita ed è interna al router.

Porte di uscita (output port): memorizzano i pacchetti che provengono dalla struttura di commutazione e li trasmettono sul collegamento in uscita.

Processore di instradamento (routing processor): esegue i protocolli di instradamento, gestisce le tabelle di inoltro e le informazioni sui collegamenti attivi.

Esistono due modelli di router, uno obsoleto che prevede la presenza dell'intelligenza che prende decisioni per le due funzioni principali del livello di rete all'interno dei router stessi e, quella moderna che separa due livelli, facendo nascere il *control plane*. Le funzioni relative alla struttura di controllo del router (control plane) sono implementate (realizzate) via software ed eseguite sul processore di instradamento.



In questo modo si parla di **SDN**, ovvero "Software Defined Networking" (Rete Definita dal Software), infatti, il controllo delle risorse di rete avviene in modo separato: piano di controllo, che prende decisioni, e piano di dati, che trasmette effettivamente i dati. Tra i vantaggi c'è una maggiore flessibilità, una migliore gestione delle risorse di rete e la presenza di un'interfaccia aperta separata da quella del piano di dati. Tuttavia, in questo modo, *potrebbe* subentrare un ritardo nell'elaborazione dati.

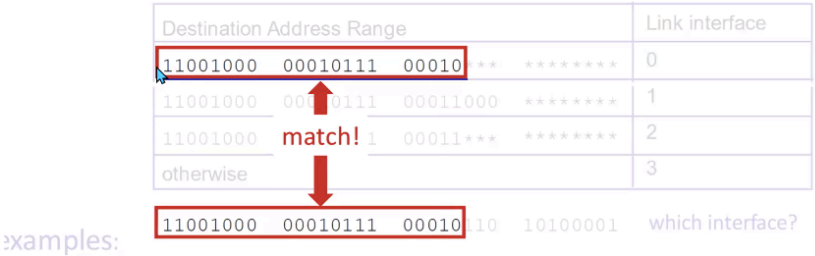
Il livello di rete di Internet mette a disposizione un solo servizio, noto come servizio *best-effort*, ossia "col massimo impegno possibile". A differenza di reti come ATM, la quale offriva servizi in grado di dar vita ad un'architettura ben controllata, in questo caso la locuzione sembrerebbe un eufemismo per indicare "nessun servizio". Infatti, non garantisce di preservare la temporizzazione dei pacchetti (tempi di trasferimento efficienti), né la loro ricezione nel medesimo ordine di invio e nemmeno che vengano consegnati, non ci sono garanzie per l'invio dei datagrammi, ovvero dei pacchetti.

La funzione di **forwarding** inizia con il processing delle informazioni che arrivano al router. Le informazioni in questione si chiamano indirizzi IP, le quali vengono indirizzate verso la linea di uscita del router tramite un'intelligenza, la quale consulta una **tabella di routing**. Trovandoci nello strato di rete le unità informative sono chiamate **pacchetti**.

Destination Address Range	Link interface
11001000 00010111 00010*** ********	0
11001000 00010111 00011000 ********	1
11001000 00010111 00011*** ********	2
otherwise	3

La tabella di routing, o forwarding table, è costituita da due colonne: la prima contenente i prefissi di rete che si esprimono con, al massimo, 32 bit rappresentati in 4 byte, letti da sinistra verso destra, ovvero dal più significativo al meno significativo; mentre la seconda contenente l'interfaccia verso cui sarà mandato il pacchetto.

Il processo del forwarding avviene quindi in questo modo: inizialmente arriva un pacchetto, il quale ha un prefisso e nell'header ha due indirizzi, uno di sorgente e uno di destinazione, quello di destinazione è una stringa di 32 bit che serve come elemento per consultare la tabella di routing. Infatti, in questa vi sono i prefissi, ovvero gli indirizzi delle sottoreti (es numeri di telefono che iniziano con 06). L'indirizzo di destinazione viene confrontato dall'intelligenza del router con i prefissi della tabella e tramite questi il pacchetto viene indirizzato verso il link di uscita.

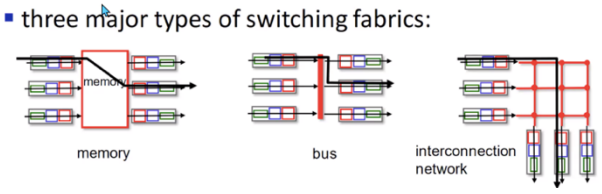


Es: l'indirizzo viene confrontato con il prefisso e se combaciano ottengo il match; quindi, il pacchetto sarà inviato verso l'uscita 0 del router.

A volte può accadere che più prefissi combacino con l'indirizzo di destinazione, in tal caso *si sceglie il prefisso con più bit in comune*, quindi quello che ha più numeri in comune con l'indirizzo (**longest prefix matching**).

In questo meccanismo di aggregazione conviene mantenere degli indirizzi di gerarchia più alta (conviene mantenere nella tabella il prefisso di Roma piuttosto dello specifico quartiere di Roma, infatti, quando ci servirà specificare il quartiere saremo arrivati nel dettaglio). Questa aggregazione è importantissima per le tabelle di routing perché devono essere di dimensioni compatte per poter essere accessibili in tempi molto rapidi, di fatti, il router deve confrontare la sequenza di bit con tutte le righe della tabella, questo tipo di analisi si chiama **ternary content addressable memories**.

Una volta decisa la linea di uscita da prendere inizia la messa in opera del forwarding, che avviene in modo differente per ogni **architettura interna della switching fabric**.



Commutazione in Memoria (Store-and-Forward)

Nella commutazione in memoria, il dispositivo di commutazione (spesso un router) riceve l'intero pacchetto, lo memorizza temporaneamente in memoria, ne esamina l'header per prendere decisioni di inoltro e quindi lo inoltra. Questo approccio consente una maggiore flessibilità nelle decisioni di routing, ma introduce una latenza legata al tempo di memorizzazione e alla verifica dell'header.

Commutazione tramite Bus (Bus Switching)

Nella commutazione tramite bus, i dispositivi nella rete condividono un singolo bus di comunicazione. Quando un dispositivo desidera inoltrare un pacchetto, lo trasmette attraverso il bus comune. Questo metodo è comune nei dispositivi di rete più piccoli ed è semplice, ma può diventare un collo di bottiglia in reti più grandi a causa delle limitazioni di larghezza di banda del bus.

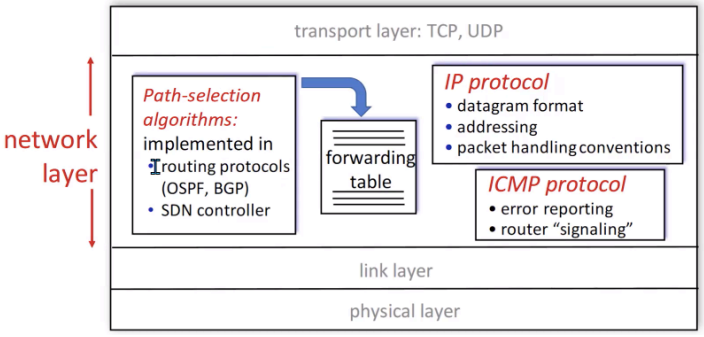
Commutazione attraverso Rete di Interconnessione (Network Switching)

La commutazione attraverso una rete di interconnessione coinvolge l'utilizzo di una matrice di connessioni interne o una rete di interconnessione che collega i diversi moduli di commutazione. Questo approccio è spesso utilizzato in switch di grandi dimensioni e in dispositivi di commutazione ad alte prestazioni.

La switching fabric di per sé è molto più veloce in bit/sec della velocità della linea d'uscita quindi un eventuale accodamento nel router avviene sulle porte di uscita. I pacchetti in ingresso vengono memorizzati e tramite l'intelligenza viene decisa la linea di uscita. Essendo la switching fabric molto più veloce delle linee d'uscita, i pacchetti diretti verso la stessa linea di uscita cominciano ad accodarsi, il ritardo di accodamento è chiamato **output port queuing**. Questo processo viene gestito secondo diverse politiche definite politiche di **scheduling**. Tra i meccanismi di scheduling più utilizzati vi sono:

1. FIFO, first in first out, il più semplice, a seconda dell'ordine di arrivo, verranno serviti.
2. Un'altra politica di attraversamento è quella di creare delle classi di priorità. Rispetto alla stessa linea di uscita si possono prevedere due (o più) buffer: uno ad alta priorità ed uno a bassa priorità. Questa modalità si può utilizzare a patto che il pacchetto abbia un'informazione riguardante la propria priorità.
3. Round Robin. Come nella seconda modalità abbiamo più code (per cui è necessario anche in questo caso che il pacchetto contenga l'informazione sulla propria categoria) che vengono gestite in modo equo, ovvero in modo ciclico vengono processati i pacchetti appartenenti alle varie code.
4. Weighted Fair Queuing. Esistono tre code con il tempo di servizio pesato; quindi, il tempo che viene dedicato ad ognuna di queste code è differente.

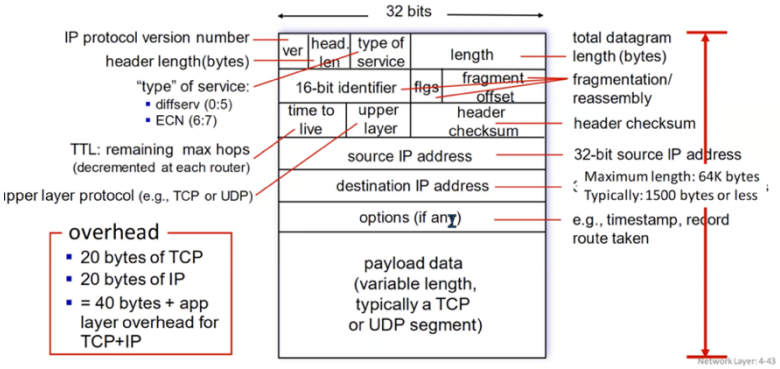
Protocollo IP – Il protocollo IP è un protocollo di strato di rete; quindi, risiede tra lo strato di trasporto e lo strato due. La prima cosa che deve fare è specificare il formato dell'unità di dati, in questo caso l'unità è chiamata **datagramma** o pacchetto. Il suo secondo compito è la modalità di indirizzamento e infine la gestione dei pacchetti.



Nello strato tre non esiste solo la parte dedicata al protocollo IP ma anche quella dedicata alle politiche di routing attraverso le quali poter completare le tabelle di routing. Di fatti questa tabella non è preconfigurata ma deve essere riempita, la si può riempire manualmente ma ciò viene fatto solo nei laboratori, mentre di consueto viene riempita dai protocolli di routing.

Dunque, esistono due mondi dentro lo strato tre: il data plane che è l'IP Protocol e il control plane che è tutto quello che ha a che vedere con il riempimento delle tabelle di Router, comprende anche l'ICMP protocol, ovvero il protocollo che, in caso di errore, invia dei segnali o feedback.

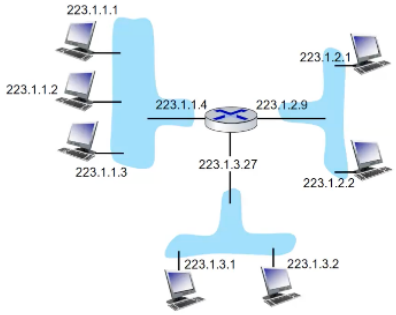
IP Datagram format



Un pacchetto IP, come tutte le unità di dati, presenta una parte di intestazione nota anche come *header* e una parte di *payload*, entrambe possono avere lunghezza variabile. L'header viene letto per righe ed ognuna di queste è composta da 32 bit, 5 sono obbligatorie, mentre le altre opzionali.

Prima riga	**Seconda riga**	**Terza riga**	**Quarta riga**	**Quinta riga**
ver (version): questi quattro bit, che specificano la versione del protocollo IP del datagramma (i più comuni sono IPv4 e IPv6)	16 bit di questa riga sono utilizzati per dare un identificativo al pacchetto, ovvero un numero che identifica i vari pacchetti	time to live (il tempo di vita del pacchetto): un contatore di 8 bit incluso nel router per assicurare che i datagrammi non restino in loop per sempre nella rete. Questo campo viene decrementato di un'unità ogni volta che il datagramma è elaborato da un router. Quando raggiunge 0, il datagramma deve essere scartato	source IP address: l'indirizzo dell'host sorgente (non del router).	destination IP address: l'indirizzo dell'host destinazione e poi vi sono le altre righe opzionali.
head len: la lunghezza dell'Header, è necessaria la sua presenza dato il carattere variabile di questa quantità	flag	upper layer: specifica il protocollo di trasporto che sta gestendo il pacchetto (UDP o TCP)		
type of service: il campo dove eventualmente si possono mettere delle classi di servizio servono all'host mittente per specificare il modo in cui il destinatario deve trattare il datagramma (precedenza, latenza, throughput, affidabilità)	fragment offset	header checksum: consente ai router di rilevare gli errori dell'header		
lenght: rappresenta la dimensione in byte del datagramma IP (header + dati), tipicamente di 20 byte				

Indirizzamento – Nella versione 4 l'indirizzo di destinazione e di sorgente sono composti da 32 bit mentre nella versione 6 da 128. Dato che la versione più comune è la 4, ci focalizziamo su di essa. Un indirizzo IP identifica un'interfaccia di rete, se un host è connesso a più di una rete avrà un indirizzo IP per ogni interfaccia, un router ha tanti indirizzi IP quante sono le interfacce da lui gestite.



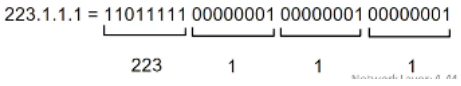
Es. Il router nell'immagine gestisce tre interfacce, sia di ingresso che di uscita, ognuna di esse fa riferimento ad un oggetto che chiamiamo sottorete, o rete IP; quindi, questo router essendo interconnesso a tre sottoreti ha tre indirizzi IP.

Tutti gli host che sono interconnessi alla stessa sottorete hanno una parte di indirizzo IP identica che si chiama **indirizzo di sottorete**.

*Una **sottorete**, o subnet, è una porzione di una rete di computer che è stata suddivisa in reti più piccole per motivi di gestione, efficienza e sicurezza e, per organizzare gli indirizzi. Le sottoreti vengono solitamente create partendo da un unico blocco di indirizzi IP che viene suddiviso in segmenti più piccoli. Ciascuna sottorete ha il proprio range di indirizzi IP e rappresenta un'entità logica separata all'interno della rete più ampia. Come detto precedentemente, ciò consente di migliorare la gestione del traffico, di applicare politiche di sicurezza specifiche a ciascuna sottorete e di semplificare la manutenzione. Una sottorete è identificata da un prefisso, per cui tutti gli indirizzi IP delle interfacce di una stessa sottorete sono caratterizzati dallo stesso prefisso. L'operazione che permette di suddividere una rete IP in reti più piccole chiamate sottoreti o subnet è denominata **subnetting**.*

Ognuno degli oggetti della sottorete deve avere un indirizzo univoco. La notazione di un indirizzo IP è duplice: **notazione numerica** sottoforma di stringa di 32 bit, o **"dotted"**, ovvero ogni gruppo di 8 bit della notazione numerica è sostituito dall'equivalente numero decimale. Generalmente i 32 bit sono divisi in due parti:

1. **net ID:** identificativo della sottorete (prefisso)
2. **host ID:** identificativo dell'host all'interno della sottorete



In questo caso possiamo assumere che la parte net-ID sia rappresentata dai i primi tre byte (223.1.1), mentre la parte host-ID dal quarto byte (.1 .2 .3…)

Le **maschere di sottorete** (subnet masks) sono utilizzate nei protocolli di rete per definire la suddivisione dei bit di un indirizzo IP nelle due parti sopra specificate: net-ID, i bit riservati alla rete, host-ID, i bit riservati agli host. Una maschera è composta da una sequenza di bit 1 seguita da una sequenza di bit 0.

L'indirizzo IP e la maschera di sottorete vengono combinati tramite l'operazione logica AND per determinare la *rete* a cui appartiene l'indirizzo IP. Per trovare la *maschera* di sottorete appropriata per una rete bisogna considerare il numero di host che questa dovrà supportare. A seconda di ciò, si può scegliere una maschera che fornisca un numero sufficiente di indirizzi IP per gli host e definisca la dimensione della rete. Ad esempio, dovendo rappresentare 50 host si può scegliere una maschera che fornisca almeno 64 indirizzi IP (2^6 = 64), ovvero /26 (6 bit per la parte host-ID e i restanti 26 per la parte net-ID). La formula per calcolare il numero di host possibili in una rete è $\mathbf{2^{(numero\ di\ bit\ per\ host)} - 2}$, si fa -2 perché non si possono assegnare agli host l'indirizzo di rete e l'indirizzo di broadcast. Nell'IPv4, l'indirizzo di rete è il primo del range ed ha tutti i bit dell'host-ID impostati a 0, l'indirizzo

di broadcast di rete, l'ultimo del range, ha tutti i bit dell'host-ID impostati ad 1. In una sottorete, l'indirizzo di rete è riservato per identificare la rete stessa, mentre l'indirizzo di broadcast per inviare pacchetti a tutti i dispositivi della sottorete contemporaneamente.

Esempio maschera di sottorete

> Indirizzo IP: 192.168.1.1
>
> Maschera di sottorete: 255.255.255.0 (o scritta in notazione CIDR come /24). In binario, la maschera di sotto rete sarà 11111111.11111111.11111111.00000000, il che indica che i primi 24 bit sono dedicati alla rete e gli ultimi 8 bit agli host.

In origine, nel 1981, le sottoreti erano divise in classi, ognuna di esse era individuata dai bit iniziali dell'indirizzo e i prefissi di sottorete avevano lunghezza fissa.

Classe	Bit iniziali	Net_Id	Host_Id	"Reti" disponibili	"Host" disponibili
A	0	7 bit	24 bit	128	16.777.216
B	10	14 bit	16 bit	16384	65.536
C	110	21 bit	8 bit	2.097.152	256
D	1110	Indirizzo multicast: 28 bit Indirizzi possibili: 268.435.456			
E	11110	Riservata per usi futuri: 27 bit Indirizzi possibili: 134.217.728			

La **classe A** prevedeva 1 byte per la parte net-ID e 3 per la parte host-ID, la **classe B** 2 byte per la parte net-ID e 2 per la parte host-ID, la **classe C** 3 byte per la parte net-ID ed 1 per la parte host-ID.

Riassunto Subnetting----------

Maschera di Sottorete (Subnet Mask): Una maschera di sottorete è una serie di bit che indica quali porzioni di un indirizzo IP sono riservate per la rete e quali sono riservate per gli host all'interno dell'indirizzo. Inoltre, sono spesso rappresentate in formato decimale puntato o in notazione CIDR.

CIDR (Classless Inter-Domain Routing): CIDR è un metodo di rappresentazione degli indirizzi IP e delle relative maschere di sottorete. Invece di utilizzare le classi fisse (Classe A, Classe B, Classe C), CIDR consente di specificare una lunghezza di prefisso variabile. Ad esempio, "/24" indica una maschera di 24 bit, mentre "/16" indica una maschera di 16 bit.

Sottoreti: Le sottoreti sono reti più piccole create all'interno di una rete principale. Ognuna di queste avrà il proprio intervallo di indirizzi IP per rappresentare i propri host e la propria maschera.

Vantaggi – Efficienza nella gestione degli indirizzi (l'assegnazione degli indirizzi IP è resa efficiente riducendone gli sprechi), miglioramento della sicurezza (isolando la rete in sottoreti è più facile individuare per ciascuna di esse una specifica politica di sicurezza), migliore gestione del traffico.

Esempio Pratico – Supponiamo di avere un indirizzo IP di classe C, 192.168.1.0, con maschera di sottorete 255.255.255.0 (/24 in notazione CIDR). Utilizzando il subnetting possiamo dividere questa rete in sottoreti più piccole, ad esempio 192.168.1.0/26, 192.168.1.64/26, 192.168.1.128/26, 192.168.1.192/26, creando così quattro blocchi di classe C, ovvero quattro sottoreti con 64 indirizzi IP ciascuna.

Nelle tabelle di routing va inserito l'**indirizzo IP di default** corrispondente all'indirizzo 0.0.0.0 con maschera 0.0.0.0.

Protocolli per gestire e risparmiare indirizzi IP: DHCP e NAT

DHCP – Il DHCP (Dynamic Host Configuration Protocol) è un protocollo che si utilizza per gestire in modo automatico gli indirizzi IP e le maschere all'interno della rete. Può essere configurato in modo che ogni qualvolta un host acceda alla rete, riceva sempre lo stesso indirizzo IP, oppure in modo tale che l'IP ricevuto dall'host sia temporaneo e diverso ogni volta che si collega alla rete.

Funzionamento di un DHCP Client-Server

1. Il client ricerca il DHCP server in rete attraverso il messaggio *DHCP Discover*
2. Il client richiede al DHCP server un indirizzo IP
3. Il server fornisce al client l'indirizzo IP temporaneo

Essendo l'IP temporaneo, quando il client termina il tempo di utilizzo dell'indirizzo, se ha ancora necessità di navigare in Internet gli verrà fornito dal server un indirizzo IP, non necessariamente lo stesso. **Ricorda:** DHCP è un servizio plug and play che prevede sempre l'utilizzo dell'indirizzo broadcast come destination address per quanto riguarda i messaggi inviati dalle parti, ovvero DHCP Discover e DHCP Request.

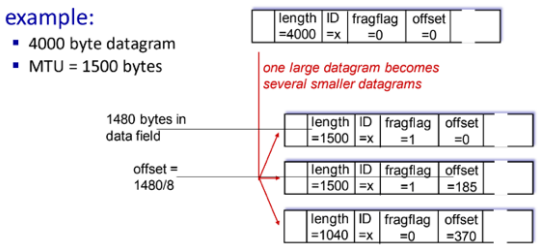
NAT – Il NAT (Network Address Traslation) è un altro metodo che insieme al DHCP ci consente di risparmiare indirizzi IP. È utilizzato per nascondere dietro un unico indirizzo pubblico decine e decine di indirizzi privati. In particolare, viene utilizzato un router NAT che si serve di un'apposita tabella di traduzione NAT. Il router assegna ad ogni host collegato alla rete una porta sorgente specifica che viene inserita all'interno della tabella. Così facendo quando un host esterno richiede una comunicazione con un host privato in questa rete NAT, il router instrada la comunicazione verso l'host specifico utilizzando la porta sorgente all'interno della tabella di NAT.

Frammentazione IP – Durante il passaggio dal livello 3 (livello di rete) al livello 2 (livello di collegamento) i datagrammi IP vengono incapsulati in frame. La dimensione dei frame non è arbitraria, questi non possono superare una determinata soglia detta **MTU** (Maximum Transmission Unit). Se i datagrammi IP la superassero sarebbe necessario avviare una procedura di frammentazione per suddividere il pacchetto originale in due o più frammenti più piccoli. La frammentazione avviene nei router e il riassemblamento del datagramma avviene negli host.

Come possono gli host percepire la frammentazione? Nell'header del datagramma IP la seconda riga, composta da 32 bit, è adibita proprio alla frammentazione; infatti, esistono tre campi che si definiscono come segue: Indentifier di 16 bit, FragFlag e Fragment offset.

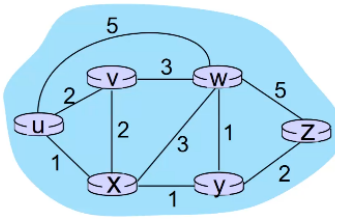
I pacchetti frammentati hanno tutti lo stesso identificatore. Il **FragFlag**, composto da un solo bit, indica se sono presenti altri pacchetti frammentati dopo il pacchetto in questione; quindi, se posto ad 1 significa che ci sono altri pacchetti frammentati che l'host deve attendere, altrimenti se posto a 0 significa che quello ricevuto è l'ultimo pacchetto. Il **Fragment offset**, espresso in byte, indentifica la posizione del frammento all'interno del pacchetto originale. Banalmente un pacchetto non frammentato avrà FragFlag e Fragment offset posti a 0.

Esempio – In questo caso il pacchetto originale è di dimensione 4000 bytes (20 bytes di header e 3980 bytes di dati). L'MTU è di 1500 bytes, il che implica che i 3980 bytes di dati devono essere suddivisi in frammenti. I primi due pacchetti trasportano 1500 bytes (20 byte di header e 1480 di dati) e l'ultimo i bytes rimanenti. Il primo pacchetto frammentato ha il flag posto ad 1 e offset a 0. L'ultimo pacchetto ha la flag posta a 0, infatti, non è seguito da nessun altro.



CAPITOLO 5 – Algoritmi e protocolli nello strato di rete

Ci sono due componenti che permettono di identificare il cammino migliore per giungere ad un host di destinazione a partire da uno di origine, di solito si sceglie il **shortest path**, ovvero il cammino più breve, quello col minor numero possibile di router, oggi può essere anche letto come il meno congestionato possibile.



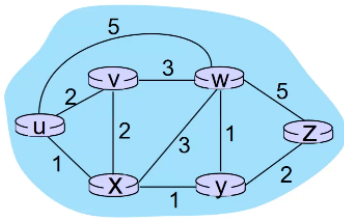
Un **grafo** è un insieme di nodi e rami, il che equivale a trattare vertici ed edge (collegamenti diretti tra un vertice ed un altro), in questo caso si intendono i router e i collegamenti logici, ma ricorda, anche se non ci interessa, i vertici possono essere host.

Osservando un grafo è possibile stabilire il costo degli edge, ovvero dei link diretti, si potrebbe voler associarlo al throughput, sarà rispetto ad esso inversamente proporzionale, oppure inversamente proporzionale allo stato di congestione. I collegamenti possono essere statici o dinamici, nel primo caso i router cambiano molto lentamente nel tempo.

Algoritmo Dijkstra link-state

L'algoritmo di Dijkstra calcola il percorso a costo minimo da un **nodo origine** che chiameremo u, a tutti gli altri nodi della rete, ed è un metodo iterativo. Si definisce con **D(v)** il costo minimo del percorso dal nodo di origine alla destinazione v, **p(v)** è il predecessore di v lungo il percorso a costo minimo dall'origine a v, e **N'** è il sottoinsieme di nodi contenente tutti e soli i nodi v per cui il percorso a costo minimo dall'origine a v è noto. L'algoritmo si compone di due parti: l'**inizializzazione** e il **ciclo**, eseguito una volta per ogni nodo del grafo.

```
1  Inizializzazione:
2    N' = {u}
3    per tutti i nodi v
4      se v è adiacente a u
5        allora D(v) = c(u,v)
6      altrimenti D(v) = ∞
7
8  Ciclo
9    determina un w non in N' tale che D(w) sia minimo
10   aggiungi w a N'
11   aggiorna D(v) per ciascun nodo v adiacente a w e non in N':
12            D(v) = min ((D(v), D(w) + c(w,v))
13   /* il nuovo costo verso v è il vecchio costo verso v oppure
14      il costo del percorso minimo noto verso w più il costo da w a v */
15    ripeti in ciclo finché non si verifica che N' = N
```



Ad esempio, guardando questo grafo si può notare che ogni arco è associato ad un numero che indica il costo del collegamento. Nel passo di inizializzazione, una volta scelto u come nodo origine, i valori dei percorsi a costo minimo noti da u ai suoi nodi adiacenti v, x e w, sono posti rispettivamente a 2, 1 e 5. Per ora consideriamo verso w il percorso diretto con un solo salto, ovvero quello che vale 5. I costi di y e z sono posti a infinito dato che tali nodi non sono adiacenti ad u.

Nella prima iterazione del ciclo si prendono in considerazione i nodi che non fanno parte di N' per determinare il nodo a costo minimo, tale nodo è x, di costo 1, che viene aggiunto all'insieme ed eseguita la riga 12 dell'algoritmo per aggiornare D(v) per tutti i nodi.

Il costo del percorso verso v non è cambiato. Il costo del percorso verso w (che era 5 al termine dell'inizializzazione) passando per il nodo x è diventato 4. Viene quindi selezionato questo percorso a costo inferiore e il predecessore di w lungo il percorso minimo da u diventa x. Analogamente, il costo verso y (attraverso x) viene calcolato essere 2, e la tabella viene aggiornata di conseguenza.

Nella seconda iterazione si trova che i nodi v e y hanno percorsi a costo minimo (2), ne scegliamo arbitrariamente uno e aggiungiamo y all'insieme N' che ora conterrà u, x e y. I costi verso i nodi rimanenti non ancora in N', ossia v, w e z sono aggiornati dalla riga 12 dell'algoritmo, ottenendo i risultati mostrati nella terza riga della tabella. E così via.

Tabella 4.3 Esecuzione dell'algoritmo LS sulla rete della Figura 4.27.

Passo	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					

In questo modo quando l'algoritmo sarà terminato, sarà evidente il percorso a costo minimo poiché per ciascun nodo sapremo qual è il precedente.

Qual è il costo computazionale dell'algoritmo? Ovvero, dati n nodi, senza contare l'origine, quanti calcoli occorre fare in totale, nel caso peggiore? La complessità è di ordine quadratico $O(n^2)$.

Osservazioni importanti – Il grafo si costruisce in modo unico per ciascun router, ciò significa che ogni nodo ha un suo grafo, soprattutto l'assegnazione del peso per ciascun arco è determinato attraverso dei messaggi.

Quando i nodi sono router, tengono conto anche di quanto traffico grava su un certo link, ovviamente un link ha un peso maggiore quando c'è più traffico. Questa situazione genera delle oscillazioni nell'algoritmo che causano un loop infinito per il quale è costretto a cambiare cammino continuamente.

Esiste un altro tipo di algoritmo, ovvero il **distance vector**, che porta ad ottenere lo stesso risultato ma non è interesse di studio. Un generico nodo decide quale sia il percorso minimo sulla base della distanza comunicatagli dai nodi vicini, non accade la stessa situazione precedente, ogni nodo non conosce tutto il grafo. Dunque, è come se, anziché partire dall'origine, si partisse dalla destinazione.

Nei sistemi autonomi l'instradamento interno viene gestito da **OSPF**, che sta per Open Shortest Path First, è un protocollo di routing adatto a reti IP basato sulla tecnologia link-state. È ampiamente utilizzato nelle reti aziendali e nelle reti di Internet come un protocollo di routing interno. Inoltre, utilizza il **flooding**, ovvero una tecnica di trasmissione di dati in cui un messaggio viene inviato a tutti i nodi della rete, indipendentemente dalla destinazione. I dati che vengono trasmessi riguardano lo stato dei collegamenti e l'algoritmo di Dijkstra per la determinazione del percorso a costo minimo. Con questo protocollo viene costruito un grafo dell'intero sistema autonomo e viene mandato in esecuzione l'algoritmo per determinare i percorsi minimi verso tutte le sottoreti.

Ogni qualvolta si verifichi un cambiamento nello stato di un collegamento, ad esempio una variazione di costo o un cambiamento di disponibilità, il router manda informazioni di instradamento via broadcast a tutti gli altri router. Inoltre, invia periodicamente lo stato dei collegamenti, almeno ogni 30 minuti, anche se non è cambiato. I messaggi che vengono inviati sono pacchetti denominati **LSA**, ovvero link state advertisement, inondano la rete ciò significa che i router inviano questi pacchetti verso gli adiacenti, contengono informazioni dettagliate sullo stato dei link adiacenti al router mittente. Questo include informazioni come l'identificatore del router adiacente, lo stato del link, il costo del link, e altri parametri rilevanti. *L'unica regola è non reinviarli se sono già stati ricevuti, per questo vengono numerati.*

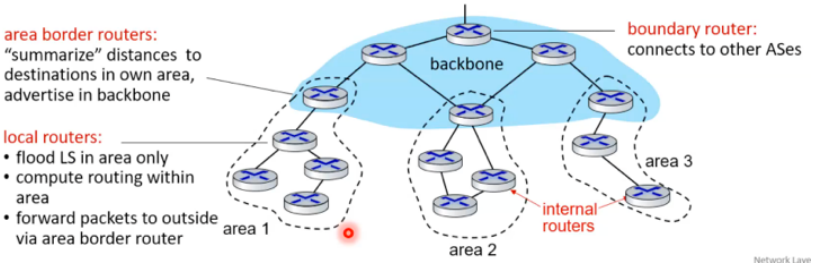
Instradamento di Internet – Il compito dei protocolli di instradamento di Internet è determinare il percorso del datagramma per arrivare alla destinazione partendo dalla sorgente. In Internet OSPF viene utilizzato in modo gerarchico introducendo i sistemi autonomi. Nel contesto del networking, un **Sistema Autonomo** è un insieme (o un gruppo) di router e reti che lavorano sotto il controllo di una singola e ben definita autorità amministrativa (ad esempio un Provider). Il routing può essere di due tipologie:

intra-domain, quando si considera il protocollo di instradamento che funziona all'interno di ciascun sistema autonomo

inter-domain, ovvero il protocollo di instradamento usato dai router per indirizzare i pacchetti verso i diversi sistemi autonomi.

Possiamo immaginare un router generico di un sistema autonomo, questo ha due identità, infatti, è capace di implementare sia le politiche di routing del sistema autonomo 1 del quale fa parte, che quelle del sistema autonomo 2 sul quale si interfaccia, ecco perché intra e iter.

Esistono tre protocolli, ad esempio OSPF viene usato sia all'interno che all'esterno del sistema autonomo. *Come si fa a parlare di gerarchia?* Ci sono router "di bordo", quelli interni, ecc.



Come si fa il routing tra sistemi autonomi, ovvero come viene gestito l'instradamento tra sistemi autonomi? **BGP**, acronimo di Border Gateway Protocol, è un protocollo di routing che permette lo scambio di messaggi tra sistemi autonomi attraverso i router di bordo, che sono quelli più "intelligenti" perché hanno una duplice visione del sistema autonomo, sia interna che esterna.

In BGP, coppie di router si scambiano informazioni di instradamento su connessioni TCP, una connessione di questo tipo per ciascun collegamento connette direttamente due router in due diversi sistemi autonomi, ma anche appartenenti allo stesso. Nel caso in cui una sessione BGP colleghi due sistemi autonomi viene detta esterna, ovvero eBGP, mentre se avviene tra router dello stesso sistema si dice interna, ovvero iBGP.

CAPITOLO 6

Lo strato di collegamento, o Livello 2, è responsabile della trasmissione affidabile dei dati attraverso un collegamento fisico o logico tra dispositivi di rete. Questo strato si occupa di questioni legate al collegamento diretto tra due nodi adiacenti in una rete e fornisce l'accesso al mezzo fisico, il quale può essere cablato (ad esempio, Ethernet su cavi in rame) o wireless (ad esempio, Wi-Fi). Lo strato di collegamento gestisce il **controllo del flusso** per garantire che il mittente non sovraccarichi il destinatario con più dati di quanti ne possa gestire (flow control), include meccanismi per il rilevamento degli errori durante la trasmissione dei dati (**error detection**).

Le unità dati dello strato di collegamento sono i **frame**, l'operazione che porta alla "nascita" di questi si chiama **framing**, ovvero i datagrammi del livello di rete vengono incapsulati dai protocolli all'interno dei frame per trasferirli attraverso i collegamenti. La struttura del frame è specificata dal protocollo, ognuno di essi include un'intestazione (header) con informazioni di controllo, l'indirizzo MAC del mittente e del destinatario e il payload dei dati. Ogni dispositivo di rete, come una scheda di rete, ha un **indirizzo MAC** (Media Access Control) univoco assegnato allo strato di collegamento. Gli indirizzi MAC vengono utilizzati per identificare i dispositivi sulla stessa rete locale.

I protocolli a livello di collegamento che forniscono un servizio di consegna affidabile garantiscono il trasporto senza errori di ciascun datagramma. Abbiamo già visto che anche alcuni protocolli di trasporto (come TCP) forniscono un servizio di consegna affidabile. Analogamente a quello del livello di trasporto, anche il servizio di consegna affidabile del livello di collegamento può essere realizzato attraverso acknowledgment e ritrasmissioni. Il servizio di consegna affidabile è spesso utilizzato per i collegamenti soggetti a elevati tassi di errore (per esempio i collegamenti wireless), allo scopo di correggere l'errore localmente sul collegamento dove questo è avvenuto.

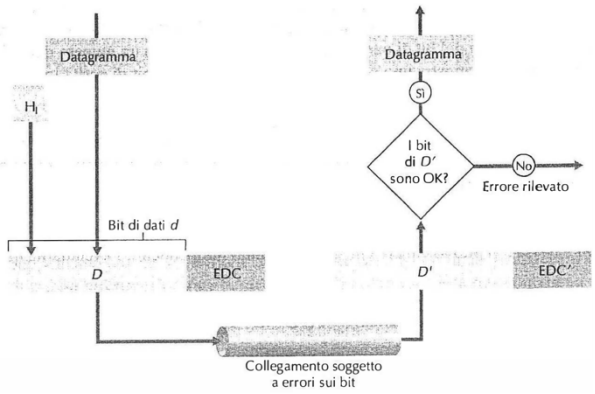
I concetti di "**half-duplex**" e "**full-duplex**" si riferiscono alle modalità di trasmissione dei dati su un canale di comunicazione. In una comunicazione half-duplex i dati possono essere trasmessi in entrambe le direzioni, ma non contemporaneamente. Questo significa che un dispositivo può inviare dati mentre l'altro dispositivo riceve, e viceversa, ma non possono farlo simultaneamente. È simile a un "turno di conversazione" in cui i partecipanti si alternano tra l'ascoltare e il parlare. In una comunicazione full-duplex i dati possono essere trasmessi in entrambe le direzioni simultaneamente. Entrambi i dispositivi possono inviare e ricevere dati contemporaneamente senza bisogno di turni. Questo consente una comunicazione bidirezionale più efficiente e continua.

Dove è implementato il livello di collegamento? Per un dato collegamento, il protocollo del livello di collegamento è realizzato da una **scheda di rete** (NIC, network interface card). Il cuore della scheda di rete è un chip dedicato che implementa molti servizi a livello di collegamento, come framing, accesso al collegamento, rilevazione di errori. La maggior parte delle funzionalità è implementata in hardware.

Tecniche di rilevazione e correzione degli errori – Quando si trasferisce un frame, ovvero un insieme di bit, attraverso un canale si potrebbe andare incontro ad un'inversione di bit (flip). Ciò significa che il nodo ricevente può decidere, erroneamente, che un bit in un frame sia 0 quando questo era stato trasmesso come 1, e viceversa. Gli errori di bit sono causati dall'attenuazione di segnale e dai disturbi elettromagnetici. Molti protocolli del livello di collegamento forniscono un meccanismo per rilevarne la presenza. Ciò è possibile grazie all'inserimento, da parte del nodo trasmittente, di **bit di controllo di errore** all'interno del frame e all'esecuzione di un semplice controllo da parte del nodo ricevente. Anche il livello di trasporto e di rete di Internet forniscono una, seppur limitata, rilevazione degli

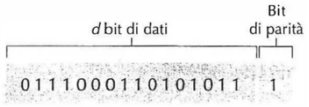
errori: il checksum. Il rilevamento degli errori a livello di collegamento è però solitamente più sofisticato, in quanto implementato in hardware.

In corrispondenza del trasmittente, ai dati D, che devono essere protetti da errori, vengono aggiunti dei bit detti **EDC** (error detection and correction). Generalmente, i dati che devono essere protetti non includono solo i datagrammi trasferiti dal livello di rete, ma anche le informazioni relative agli indirizzi del livello di collegamento, i numeri di sequenza e altri campi nell'intestazione. I dati D e i bit EDC sono inviati in un frame al nodo ricevente. Questo legge una sequenza di bit D' ed EDC' che può essere diversa dall'originale, come risultato della modifica dei bit in transito. Il nodo ricevente deve determinare se D' coincida con D, potendo contare soltanto su D' e su EDC'.

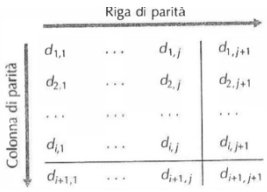


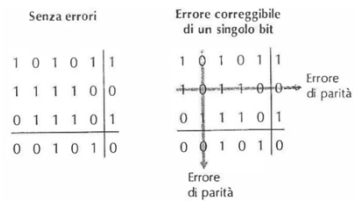
Controllo di parità – La **parity checking** (verifica di parità) è una tecnica utilizzata per il rilevamento degli errori nei dati trasmessi o memorizzati. Viene aggiunto un bit di parità (parity bit) ai dati per creare un valore che ne rifletta la parità degli uni.

Parity Bit di Parità Singola: la forma più semplice di rilevamento degli errori è quella che utilizza un unico bit. Supponiamo che le informazioni da inviare, D nella Figura, siano costituite da d bit. In uno schema di parità pari, il mittente include un bit addizionale e sceglie il suo valore in modo da rendere pari il numero totale di bit 1 nei d + 1 bit trasmessi (l'informazione originale + il bit di parità). Con un solo bit di parità ciò che il ricevente deve fare è molto semplice: contare il numero di bit a 1 tra quelli ricevuti. Se trova un numero dispari di bit 1, sa che si è verificato almeno un errore in un bit. Nello schema di parità dispari, il valore del bit di parità è scelto in modo che ci sia un numero dispari di bit 1.

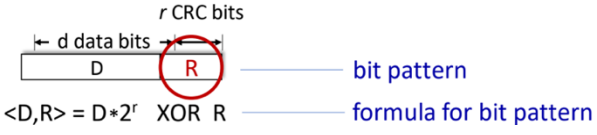


Parity Bit di Parità Doppia: in alcuni casi, si utilizza una configurazione più avanzata chiamata "Hamming Code" o "parity bit di parità doppia". Questo schema aggiunge più bit di parità in posizioni specifiche per consentire la rilevazione e la correzione di errori. Ad esempio, una stringa di 16 bit può essere riscritta sotto forma di matrice, per poi assegnare ad ogni riga e colonna un bit di parità. Gli schemi come questo di correzione degli errori sono progettati per individuare e correggere errori su un singolo bit. Se si verificassero errori su più bit nello stesso blocco di dati, il sistema potrebbe non essere in grado di correggerli tutti.

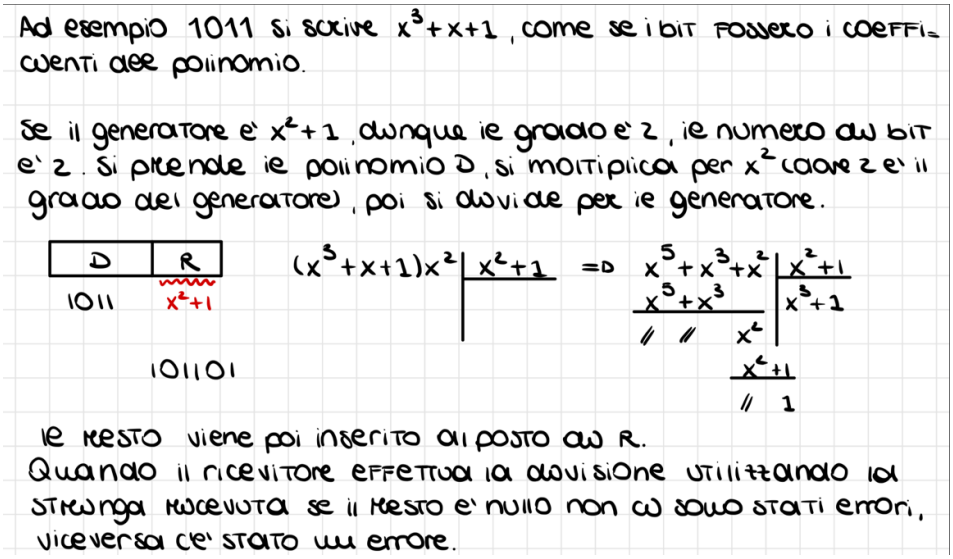




Controllo a ridondanza ciclica (CRC) – I codici CRC sono anche detti **codici polinomiali**, in quanto è possibile vedere la stringa di bit da trasmettere come un polinomio i cui coefficienti sono i bit della stringa. Le operazioni sulla stringa di bit, in questo tipo di controllo, sono interpretate come aritmetica polinomiale. Chi trasmette il codice polinomiale, ovvero i dati D costituiti da d bit, conosce anche il polinomio generatore G, una stringa di r + l bit. Dato un blocco di dati D, il mittente sceglierà r bit addizionali, R, e li unirà a D in modo da ottenere una stringa di d + r bit che, interpretata come numero binario, sia esattamente divisibile



per G nell'aritmetica modulo 2. Il processo di controllo del CRC controlla se la divisione (d+r) /G ha un resto diverso da 0, così che il ricevente sappia che si è verificato un errore; altrimenti i dati sono accettati come corretti.



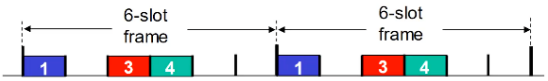
Protocolli e collegamenti ad accesso multiplo – Esistono due tipi di collegamento di rete: **punto a punto** e **broadcast**. Nel primo caso c'è un trasmittente ad un'estremità del collegamento e un unico ricevente all'altra. Il secondo, invece, può avere più nodi trasmittenti e riceventi connessi allo stesso canale broadcast condiviso, si parla, cioè, di mezzo condiviso. Il termine broadcast indica che, quando un nodo trasmette un frame, il canale lo diffonde e tutti gli altri nodi ne ricevono una copia. Ethernet e Wireless LAN (ricorda che le LAN sono reti geograficamente confinate in un singolo edificio, in un'azienda o campus universitario) sono esempi di tecnologie con collegamenti broadcast. Di conseguenza è importante controllare e coordinare l'accesso di più nodi (host, router, switch) trasmittenti e riceventi in un mezzo condiviso, ossia il problema dell'**accesso multiplo**. Il problema principale è determinare chi e quando debba parlare, ovvero trasmettere.

Tutti i nodi sono in grado di trasmettere frame, è possibile che due o più lo facciano nello stesso istante, per cui tutti i nodi riceveranno contemporaneamente più frame. Fisicamente si traduce nella presenza, sul mezzo, di due o più segnali che si sovrappongono e diventano indistinguibili, non è un vero e proprio errore, bensì i bit diventano incomprensibili. Tra questi nodi si genera una **collisione** a causa della quale nessuno dei nodi riceventi riuscirà a interpretare i frame. Con la collisione si verifica una perdita di dati e il canale rimane inutilizzato, se la situazione si dovesse ripetere più volte si arriverebbe a sprecare gran parte della banda.

Esistono tre modalità per la **condivisione di frame**, ovvero tre differenti protocolli: **divisione a priori**, **ad accesso random** e **a rotazione**, ovvero a turni. Idealmente un protocollo ad accesso multiplo per un canale broadcast con velocità di R bit/s dovrebbe avere le seguenti caratteristiche: gli M terminali, o nodi, devono avere il loro spazio 1/M e disporre di un throughtput pari a R/M bps, il che implica che ciascun nodo avrà velocità media di trasmissione pari a R/M bps; il protocollo è decentralizzato, ciò vuol dire che non ci sono nodi principali che, qualora non funzionassero correttamente, potrebbero rendere inattivo l'intero sistema.

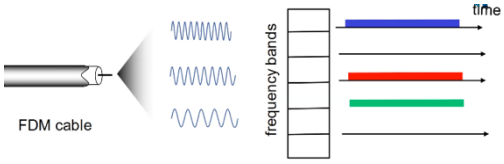
Protocolli a suddivisione del canale (divisione a priori) – Esistono due tecniche che possono essere utilizzate per suddividere la larghezza di banda di un mezzo condiviso fra i nodi che vi accedono: TDM e FDM. Ad esempio, supponiamo che il canale supporti M nodi e che la sua velocità di

trasmissione sia di R bps, nel caso di **TDMA**, ovvero accesso multiplo divisione nel tempo, si suddivide il tempo in intervalli (time frame) che sono macrostrutture chiamate trame, per evitare di chiamarle come le unità dati del livello 2 e, poi si divide ciascun intervallo di tempo in M slot temporali (time slot).



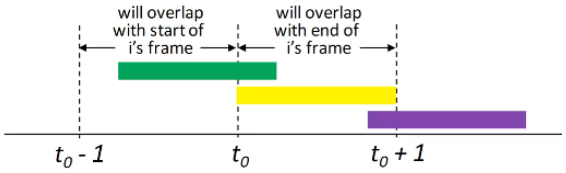
TDMA riesce a evitare le collisioni ed è perfettamente imparziale: ciascun nodo ottiene, durante ciascun intervallo di tempo, un tasso trasmissivo di R/M bps. Tuttavia, presenta due specifici inconvenienti: se durante il suo time slot un utente non lo usa o lo uso parzialmente, c'è uno spreco di risorse, e inoltre non può superare il limite medio di R/M bps.

Altro caso è quello relativo a **FDM**, ovvero questa tecnica divide il canale in frequenze differenti, ciascuna con una larghezza di banda di R/M e assegna ciascuna frequenza a un nodo, si immagina che ogni stazione propagherà il proprio segnale lungo la sottostazione che gli è stata assegnata. Anche FDM evita le collisioni e divide equamente la larghezza di banda tra gli M nodi, il che si traduce, anche con questo protocollo, nel dedicare a ciascun nodo una porzione di banda limitata a R/M, anche quando vi è un solo nodo che deve trasmettere.

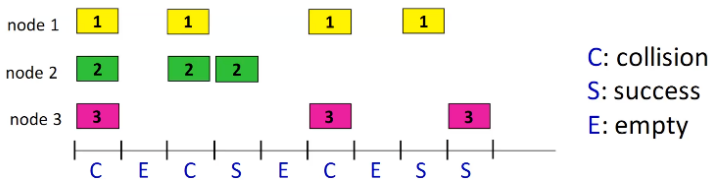


Protocolli ad accesso casuale (accesso random) – I protocolli precedentemente analizzati non sono efficienti nella stragrande maggioranza dei casi perché non si è certi di avere sempre flussi continui di bit, perciò si può andare incontro a spreco di risorse.

Una soluzione potrebbe essere rappresentata da un protocollo ad accesso casuale che si chiama **Pure ALOHA**, un utente generico se deve trasmettere un frame prende e lo fa, ovviamente ciò porta l'eventualità di collisioni. Appena arriva un frame, ovvero un datagramma del livello di rete raggiunge la scheda del nodo trasmittente, questo lo trasmette immediatamente, se il frame va in collisione allora il nodo lo ritrasmette immediatamente. Durante la trasmissione ci potrebbe essere qualcuno che comincia a parlare prima o mentre il nodo 1 trasmette. L'efficienza di questo modello è pari al 18%, la restante parte del nostro parlare è da buttare.



Tra le prime specificazioni e soluzioni alle problematiche presenti per il Pure ALOHA c'è lo **Slotted ALOHA**. Si assume che tutti i frame consistano esattamente di L bit, il tempo sia suddiviso in slot di L/R secondi, ovvero che ciascuno slot equivalga al tempo di trasmissione del pacchetto, i nodi comincino la trasmissione dei frame solo all'inizio degli slot, i nodi siano sincronizzati in modo che tutti sappiano quando iniziano gli slot e, qualora in uno slot due o più frame collidano, tutti i nodi della rete rilevino l'evento prima del termine dello slot.



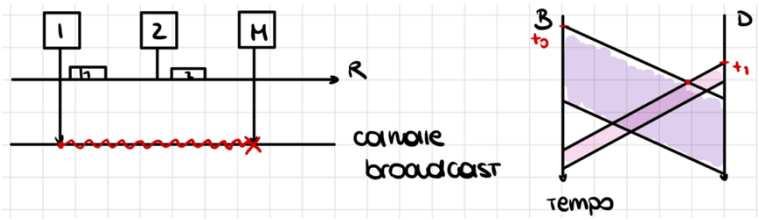
Ci possono essere collisioni perché più terminali potrebbero scegliere lo stesso time slot per trasmettere i frame. Con questo piccolo accorgimento si è dimezzata la vulnerabilità e dunque il picco di efficienza sarà pari al 36%.

Come ci si accorge della collisione? In entrambi i protocolli chi trasmette deve essere avvisato circa l'esito della trasmissione inserendo un'acknowledgement. L'efficienza si calcola tempo utile/tempo totale, nel caso dell'ultimo grafico avrò 3/9 perché 3 sono i nodi e 9 gli slot temporali.

Nei due protocolli ALOHA i nodi prendono la decisione di trasmettere indipendentemente dall'attività degli altri nodi collegati al canale broadcast. In particolare, un nodo non presta attenzione al fatto che ve ne sia un altro che sta trasmettendo, né arresta la trasmissione se un altro nodo inizia a interferire con la sua trasmissione. Il modo per **moderare la trasmissione** dei frame, al pari di ciò che avviene in una conversazione reale tra due o più persone, si avvale di due regole importanti: ascoltare prima di parlare e, se qualcun altro comincia a parlare insieme a voi, smettere di parlare. Il che si traduce, nel mondo delle reti, nei seguenti comportamenti: (rilevamento portante, **carrier sensing**) un nodo ascolta il canale prima di trasmettere e, se questo sta già trasmettendo un frame, aspetta finché il canale è libero; (rilevamento della collisione, **collision detection**) il nodo che sta trasmettendo rimane contemporaneamente in ascolto del canale, se osserva che un altro nodo sta trasmettendo un frame che interferisce col suo, arresta la propria trasmissione, aspetta un intervallo di tempo casuale e poi ripete il processo.

Queste due regole sono rispettivamente alla base dei protocolli **CSMA** (carrier sense multiple access) e **CSMA/CD** (CSMA with collision detection).

Perché, se tutti i nodi eseguono il rilevamento portante, si verificano collisioni? Dopo tutto, i nodi rinunciano alla trasmissione quando rilevano che qualcun altro sta trasmettendo. Si può rispondere osservando un diagramma spazio-tempo, dove l'asse orizzontale mostra la posizione dei nodi sul canale condiviso e quello verticale rappresenta il tempo. La limitazione è fisica perché all'istante t0, ad esempio, il nodo B rileva che il canale è inattivo, ovvero nessun altro nodo sta trasmettendo, allora comincia la trasmissione e i suoi bit si propagano in entrambe le direzioni lungo il mezzo trasmissivo. Al tempo t1>t0 il nodo D ha un frame da spedire, sebbene al tempo t1 il nodo B stia ancora trasmettendo, i suoi bit non hanno ancora raggiunto D e quindi questo ritiene il canale libero. Dunque, il ritardo di propagazione da un estremo all'altro di un canale broadcast, ovvero il tempo richiesto da un segnale per propagarsi da un nodo all'altro, sarà fondamentale: maggiore è il ritardo, maggiore sarà la possibilità che il secondo nodo non si accorga che è già cominciata la trasmissione da parte di un altro.



Introdurre il rilevamento di collisione in un protocollo ad accesso multiplo ne migliorerà le prestazioni evitando l'inutile trasmissione dell'intero frame danneggiato dall'interferenza con un altro frame (CSMA/CD). Riassumiamone le operazioni:

1. La scheda ottiene direttamente un datagramma dal livello di rete, prepara un frame a livello di collegamento e lo sistema in un suo buffer.

2. Quando riscontra che il canale è libero inizia la trasmissione del frame. Se il canale risulta occupato, resta in attesa sino al momento in cui non rileva più il segnale e solo allora inizia l'invio del frame.

3. Durante la trasmissione verifica la presenza di eventuali segnali provenienti da altre schede di rete sul canale broadcast.

4. La scheda di rete, se trasmette l'intero frame senza rilevare energia di segnale proveniente da altre schede, ha finito il suo lavoro; altrimenti, se riscontra energia di segnale durante la trasmissione, interrompe immediatamente la trasmissione del frame.

5. Dopo aver annullato la trasmissione, la scheda di rete aspetta per un tempo casuale e poi ritorna al passo **2**.

Il RTT, ovvero round trip time, forza i frame ad avere una **lunghezza minima**, per far sì che, nel caso peggiore, ovvero quando il primo terminale trasmette senza problemi e un epsilon prima della fine della trasmissione si sovrappone ad un altro, posso accorgermi della sovrapposizione. Avrò pertanto un frame di lunghezza L per un tempo pari o maggiore al RTT: L/R >= RTT. Sapendo che il RTT è pari a due volte il ritardo di propagazione τ (=d/v), si calcola 2d/v, dove d è la distanza percorsa dal pacchetto e v è la sua velocità.

Se avendo progettato la scheda di rete in un certo modo cambiassi il mezzo trasmissivo, anziché utilizzare un mezzo cablato di un certo tipo, come il rame, utilizzassi la fibra ottica che ha una velocità di trasmissione maggiore, ad esempio R=100Mbit/s anziché i precedenti 10Mbit/s, il protocollo non funzionerebbe più perché la disuguaglianza non è detto venga garantita.

Guardando il disegno precedente, D è il primo ad accorgersi della collisione; infatti, B è quello che termina prima la trasmissione RTT e "tocca" fisicamente D, allora quest'ultimo raccoglie informazioni e continua a mandare bit sporchi per far cogliere a B la collisione, perciò non interrompe subito la trasmissione. Dunque, se c'è stata una collisione bisogna ritrasmettere i frame aspettando per un intervallo di tempo casuale piuttosto che fisso, ciò perché se due nodi trasmettono frame simultaneamente e quindi aspettano lo stesso periodo di tempo, continueranno a entrare in collisione. Si utilizza l'algoritmo **binary exponential backoff,** ovvero attesa binaria esponenziale. Supponiamo che la scheda di rete tenti per la prima volta di inviare un frame e che, mentre lo trasmette rilevi una collisione, allora sceglie dopo un tempo casuale, la prima o la seconda time slot a disposizione, ad esempio tra 10 o 10 minuti e 1/4. Se avessi di nuovo una collisione dopo questa scelta, il tempo nel quale vado a randomizzare crescerebbe esponenzialmente, arrivando così a quattro time slot, ecc. In questo modo è come se man mano aumentassi la probabilità che non avvenga la collisione. Dunque, si otterrà che dopo ***m collisioni*** la finestra di scelta sarà lunga $2^m - 1$, ovvero avrò un insieme $\{0, 1, 2, \dots, 2^m - 1\}$.

- T_{prop} = max prop delay between 2 nodes in LAN
- t_{trans} = time to transmit max-size frame

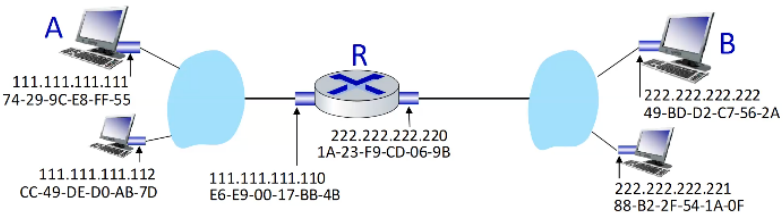
$$efficiency = \frac{1}{1 + 5t_{prop}/t_{trans}}$$

L'**efficienza** di questo meccanismo tende ad uno se il mezzo ha un ritardo di propagazione che tende a zero, dunque dipende dal tempo di propagazione. Equivalentemente, l'efficienza tende ad uno se il tempo per trasmettere un frame di grandezza massima tende ad infinito.

Una terza possibilità è creare dei turni, dove si parla di oggetti chiamati token per garantire che la comunicazione sia regolata.

Instradamento – Si pone attenzione alle reti locali commutate e all'instradamento dei frame a livello di collegamento che, a differenza di quello a livello di rete, non utilizza gli algoritmi di instradamento RIP e **OSPF**. Al posto degli indirizzi IP vengono usati gli indirizzi a livello di collegamento. In realtà, non sono i nodi ad avere indirizzi a livello di collegamento, ma i loro adattatori, ovvero le schede di rete. Un host o un router con più interfacce di rete avrà più indirizzi a livello di collegamento associati a esse, esattamente come accadeva per gli indirizzi IP. Mentre gli switch a livello di collegamento non hanno indirizzi a livello di collegamento associati alle loro interfacce che connettono a host e router. L'indirizzo di livello due si chiama **MAC**, è completamente diverso dall'indirizzo IP perché se questo è un indirizzo globale che indirizza host da una parte all'altra del mondo, il MAC invece gioca un ruolo locale indirizzando le unità nella sottorete. Per molte LAN, come Ethernet, l'indirizzo MAC è lungo 6 byte presentati in coppie di 4 bit che rappresentano numeri in esadecimale (sistema in cui si conta da 0 a 9 e poi si usano le lettere dell'alfabeto da A ad F), il che consente di avere 2^{48} possibili indirizzi perché ogni byte è composto da 8 bit per un totale di 48. Un dispositivo interconnesso a una generica rete locale è equipaggiato di due indirizzi: un indirizzo MAC della scheda di rete, ed è unico perché dipende dal produttore della scheda, ed un indirizzo IP. L'indirizzo MAC, come l'indirizzo IP, è unico; questa univocità dipende dal produttore della scheda ed è diversa rispetto a quella dell'altra tipologia di indirizzi. Se gli indirizzi IP hanno una struttura gerarchica, ovvero una parte per la rete e una parte per l'host, e devono essere cambiati quando l'host si sposta, ovvero quando cambia la rete al quale è collegato, l'indirizzo MAC rimarrà lo stesso indipendentemente dal luogo in cui il computer è utilizzato poiché dipende, ad esempio, dalla sua scheda Ethernet. *Per capire meglio:* è come se l'indirizzo IP fosse l'indirizzo postale di una persona e l'indirizzo MAC il codice postale.

ARP, dove ARP sta per address resolution protocol, è un protocollo per far avvenire l'accoppiamento del MAC address con l'indirizzo IP. Il protocollo ha il compito di creare nei router e negli host una tabella, definita **ARP table**, che contiene la corrispondenza tra indirizzi IP e MAC e un valore relativo al **TTL**, ovvero time-to-live (tempo di vita), che indica quando bisognerà eliminare una data voce dalla tabella. Se un utente host verso un router, o viceversa, volesse scoprire l'indirizzo MAC del destinatario del frame dovrebbe mandare in broadcast un **ARP request**, ovvero la domanda *"qual è l'indirizzo MAC associato al seguente indirizzo IP?"* La particolarità sta nel fatto che la richiesta è contenuta in un frame broadcast, mentre la risposta in un frame standard. In secondo luogo, ARP è **plug-and-play**, nel senso che la tabella ARP di un nodo si costruisce automaticamente e non deve essere configurata dall'amministratore del sistema.



Abbiamo due sottoreti collegate da un router, cosa succede quando un utente A deve mandare un frame ad un utente B del quale conosce solo l'indirizzo IP (in questo caso l'IP di destinazione è quello di B)? L'utente A va nella tabella di routing degli host, dopo aver applicato la maschera per quello specifico indirizzo di destinazione e cercato matching, se non esiste, capisce che l'instradamento non è diretto. Dunque,

capisce che il frame deve essere mandato al router di default che si affaccia sulla sua sottorete del quale conosce l'indirizzo IP, ma non l'indirizzo MAC. Allora A manda un ARP request *"chi ha questo indirizzo IP router?"* Con la risposta A può comporre il suo frame di livello due inserendo nell'intestazione l'indirizzo MAC di destinazione, ovvero quello del router. Il router lo "spacchetta" e analizza l'unità dati per scoprire e prendere l'IP di destinazione. Allora va nella tabella di routing e attraverso il matching scoprirà che il routing, questa volta, è diretto perché l'host destinatario fa parte della sottorete direttamente collegata al router. **NOTA BENE:** gli indirizzi IP non cambiano mai, mentre gli indirizzi MAC cambiano nei vari step della trasmissione.

Ethernet – Ethernet è uno standard di comunicazione di rete ampiamente utilizzato per collegare dispositivi locali in una LAN, rete di computer limitata a un'area geografica relativamente piccola. Dunque, Ethernet è un insieme di regole, protocolli e specifiche che definiscono come i dispositivi di rete devono comunicare tra loro per consentire la trasmissione e lo scambio di dati. In poche parole, è una connessione cablata tra i computer che forma gradualmente una rete locale (LAN).

Ethernet ha varie **topologie di rete**. Le prime LAN Ethernet usavano un cavo coassiale come bus per connettere i nodi, poi si è passati ad una topologia a stella che si serve di un doppino intrecciato in rame, all'inizio del 2000 la topologia a stella anziché appoggiarsi ad un hub centrale, l'ha sostituito con uno switch. Dunque, anziché utilizzare un unico cavo che collega tutti i vari terminali, per non incorrere in strutture instabili a livello di robustezza fisica, si è passati alla topologia **switched Ethernet**: si utilizza uno switch e ogni stanza ha il proprio cavo poiché le linee attraversano i muri.



I sei elementi del frame Ethernet



Sebbene un frame Ethernet trasporti un datagramma IP al suo interno, può trasportare anche altri tipi di pacchetti a livello di rete.

Preambolo: i frame Ethernet iniziano con un campo di otto byte utili per avvisare la scheda di rete dell'arrivo di un frame Ethernet.

Indirizzo di destinazione: la prima informazione utile subito dopo il preambolo è rappresentata dall'indirizzo MAC della scheda di rete di destinazione, in questo modo è possibile per la scheda di rete ricevente il frame capire subito se è di suo interesse leggerne o meno il contenuto, ovviamente ciò dipende se il MAC è suo o meno.

Indirizzo di sorgente: campo che include l'indirizzo MAC della scheda che trasmette il pacchetto.

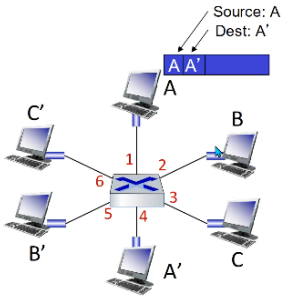
Campo dati: è il campo di payload, ovvero quello di lunghezza variabile che contiene i pacchetti IP.

CRC: campo che consente alla scheda di rete ricevente di rilevare la presenza di un errore nei bit del frame. È posto alla fine perché fisicamente deve essere utile per controllare i bit che la precedono, se sono stati intaccati da errori o meno.

Il cavo ethernet venne inizialmente concepito come un segmento di cavo coassiale, nelle installazioni moderne i nodi sono collegati ad uno switch con segmenti punto a punto, realizzati con doppini in rame o fibre ottiche, la fibra ottica non disperde a meno che sia arrotolata.

Il ruolo dello switch è ricevere i frame in ingresso e inoltrarli sui collegamenti in uscita. Lo switch stesso è **trasparente ai nodi**, cioè un nodo indirizza un frame a un altro nodo, piuttosto che indirizzarlo allo switch e invia il frame nella LAN, senza sapere che uno switch riceverà il frame e lo inoltrerà agli altri nodi. Gli switch sono dispositivi diversi da host e router, rispetto ad essi, sono caratterizzati solo dagli strati 1 e 2, mentre gli host da tutti gli strati e i router da 1, 2 e 3.

In sintesi, le loro funzioni fondamentali sono quella di filtraggio (**filtering**) e inoltro (**forwarding**). Infatti, innanzitutto bisogna stabilire se il frame debba essere inoltrato verso qualche interfaccia o scartato, dopodiché, se deve essere inoltrato, bisogna individuare l'interfaccia verso cui deve essere diretto. Affinchè queste funzionalità vengano implementate in un dispositivo del genere e, funzionino, è necessario fare riferimento alla tabella di commutazione, oppure **switch table**, composta da voci che contengono l'indirizzo MAC del nodo, l'indirizzo dello switch che conduce al nodo, ovvero la porta attraverso la quale quel dispositivo è collegato allo switch e, il TTL. Dunque, gli odierni switch sono dotati di una semplice intelligenza che permette di introdurre la definizione di plug-and-play, dato che la tabella viene configurata da sé, con un meccanismo self-learning, capendo in modo autonomo la configurazione della rete.



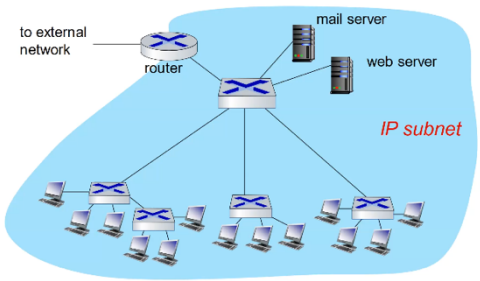
Supponiamo che un certo dispositivo A debba mandare ad A' un frame. Il frame giunge allo switch sull'interfaccia 1, non conoscendo l'interfaccia di A', lo manda in broadcast, ovvero a tutte le interfacce eccetto quella dalla quale l'ha ricevuto. In questo modo capisce che A sta sulla porta 1. Poiché i vari host mandano indietro il messaggio se non di loro interesse, lo switch capisce che A' si trova sulla porta 4, avendo trovato il match con il MAC address.

MAC addr	interface	TTL
A	*1*	*60*

Switch table (initially empty)

Quello che segue a destra è un esempio di una rete istituzionale connessa da quattro switch.

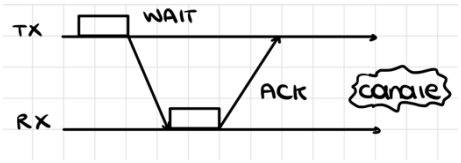
Small institutional network



Una comunicazione si definisce **inaffidabile** se vi sono errori sui bit del pacchetto trasmesso, se c'è mancanza di ordine dei bit, se non è garantito il corretto trasferimento dell'unità dati, ovvero non sono stati trasmessi tutti i bit che la compongono o, addirittura, nessuno. Esistono pertanto dei protocolli che cercano di risolvere questi problemi migliorando la comunicazione tra trasmettitore e ricevitore poiché tra essi è come se ci fosse una tenda che oscura ad entrambi la vista su ciò che accade nel corso della trasmissione.

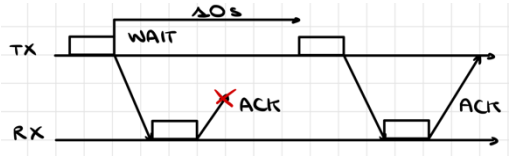
Il trasmettitore richiede un trasferimento affidabile delle informazioni, ovvero **rdt** (reliable data transfer protocol), d'altro lato il destinatario deve fare in modo da ricevere dati perfetti.

Caso 1: rdt2.0



Il trasmettitore invia un pacchetto attraverso il canale e si mette in attesa di un **feedback**, il ricevitore ottiene i dati e manda una "risposta". Le due tipologie di feedback sono positiva e negativa, nel primo caso si parla di un acknowledgment, ACK ovvero "OK", nel secondo caso di un NAK, ovvero "NO". Se si riceve un ACK significa che il trasmettitore è pronto a mandare altre unità informative.

Caso 2: rdt2.0



Se dal destinatario ricevessi un feedback negativo, ovvero un NAK, il protocollo crasherebbe perché dallo stato di blocco non potrebbe mandare nuovi pacchetti. Anziché "cadere" in questo loop infinito di attesa potrei evitare di inviare un NAK e anziché lasciare il trasmettitore in una attesa infinita, potrei avvalermi di un **temporizzatore**. Quest'ultimo funziona come un vero e proprio countdown che include il tempo di trasmissione del pacchetto, allo scadere del quale, se non è stato ricevuto l'ACK, lo si ritrasmette.

Sembrerebbe di aver risolto il problema, in realtà non è così perché l'ACK è un vero e proprio pacchetto, ovvero un messaggio, in quanto tale, potrebbe venir perso nel corso del tragitto dal ricevitore al trasmettitore, causando la duplicazione del pacchetto.

Per evitare questo errore si può scrivere nell'header del pacchetto una cifra per numerare le unità informative, in questo modo il ricevitore e il trasmettitore possono reciprocamente capire quanti e quali pacchetti sono stati inviati.

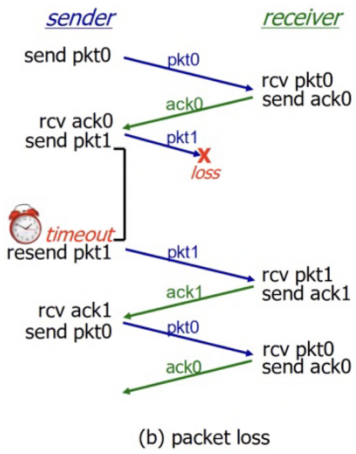
Questo protocollo per la trasmissione affidabile si chiama **Stop&Wait** perché è caratterizzato da due stati: stop e wait. È indubbiamente molto lento, però sicuro. Bisogna tuttavia aggiungere una regola al protocollo, ovvero la numerazione degli ACK per evitare che il trasmettitore, ricevuto il riscontro, spedisca lo stesso pacchetto. La numerazione non è relativa al pacchetto ricevuto correttamente dal destinatario, quanto piuttosto al pacchetto che il destinatario si aspetta di ricevere.

Per numerare i pacchetti si usa il modulo due, ovvero un unico bit: 0 o 1. Il primo pacchetto lo chiamo 0 e quando il destinatario manda l'ACK questo conterrà 1, dopo aver spedito il pacchetto 1, viene mandato l'ACK (0), ovviamente il pacchetto 0 non è quello di prima ma uno nuovo.

Consideriamo 3 casi:

1. Pacchetto 2 sbagliato
2. ACK (2) non arriva
3. ACK (2) arriva dopo il timeout

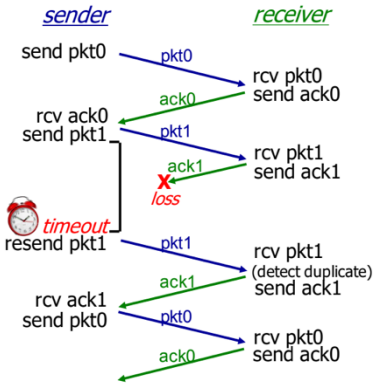
Caso 1



(b) packet loss

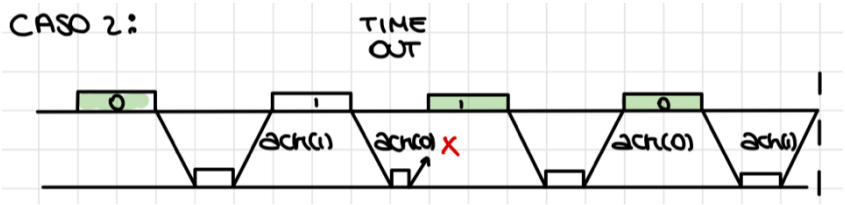
In questo caso se si volesse calcolare il **throughput**, che ricordiamo essere la quantità di dati trasmessi in una unità di tempo, ovvero il rapporto tra i bit trasmessi e il tempo totale di trasmissione dei pacchetti nel canale, sarebbe il rapporto tra il totale dei bit e il tempo totale di trasmissione: al numeratore abbiamo 3L/C perché i pacchetti effettivamente spediti sono 3 e, di questi mi interessa il tempo di trasmissione, mentre al denominatore il tempo di trasmissione dei 3 pacchetti + il RTT di ognuno dei 3 pacchetti (ricorda il RTT è il ritardo di propagazione andata-ritorno di ciascun pacchetto, dunque 3RTT) + timeout. In questo grafico spazio-tempo, il tempo è la lunghezza verticale, ovvero l'asse delle ordinate, mentre l'asse delle ascisse è lo spazio.

Caso 2

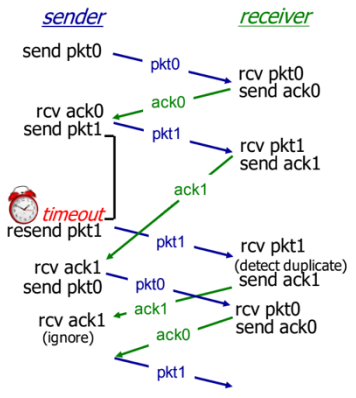


(c) ACK loss

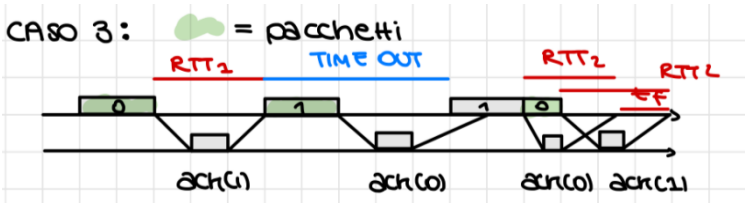
In questo caso se si volesse calcolare il **throughput** avremmo al numeratore sempre 3L/C, ovvero il tempo di trasmissione relativo ai tre pacchetti effettivamente spediti, mentre al denominatore sempre 3L/C + 3RTT + timeout + L/C. Dunque, ho aggiunto un L/C perché il timeout, rispetto a prima, è disegnato diversamente. Infatti, è come se avessi un pacchetto ripetuto.



Caso 3



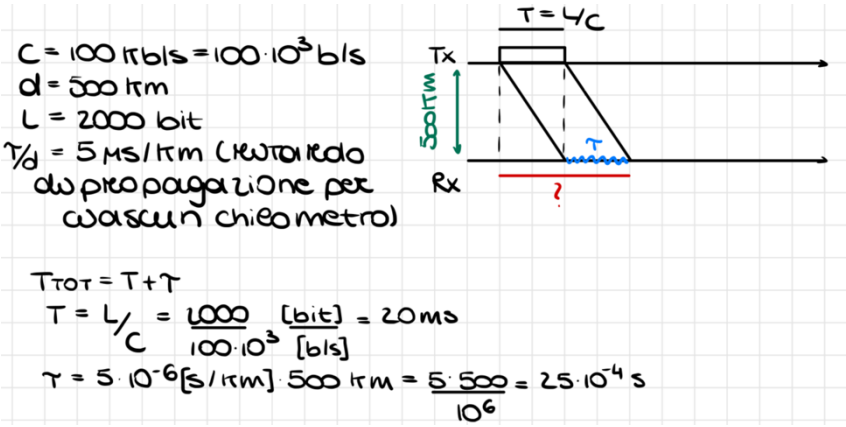
(d) premature timeout/ delayed ACK



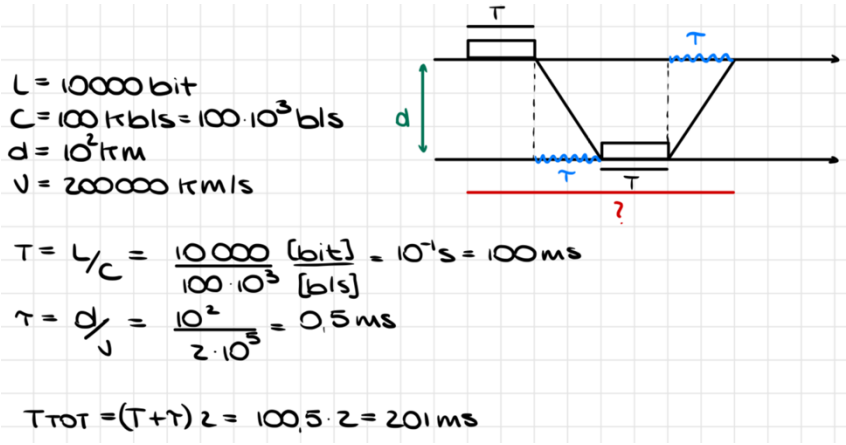
Al numeratore avrò sempre 3L/C mentre al denominatore 3L/C + 2RTT + timeout.

Esercizio 1.1 – un sistema trasmissivo della velocità di 100 [kb/s] presenta una lunghezza di 500 [km]. Si calcoli il tempo che intercorre fra la trasmissione del primo bit e la ricezione dell'ultimo bit di un pacchetto lungo 2000 [bit], assumendo che il ritardo sia di 5 [μs/km].

Da notare che viene fornito il valore del ritardo accumulato dal pacchetto ad ogni km percorso, per questo per conoscere il tempo di propagazione τ (ritardo nell'invio di un pacchetto) si moltiplica per il numero di km percorsi.



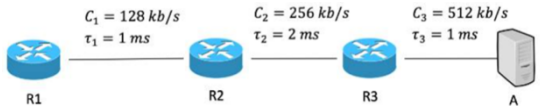
Esercizio 1.2 – un pacchetto di 10000 [bit] viene inviato dal nodo A alla velocità di 100 [kb/s] su un collegamento di 100 [km]. Il pacchetto viene ricevuto tutto in B e poi viene rimandato al mittente A alla stessa velocità di trasmissione. Si calcoli l'intervallo di tempo che intercorre fra la trasmissione del primo bit in A e la ricezione dell'ultimo bit, sempre in A, assumendo che la velocità del segnale sia di 200000 [km/s].



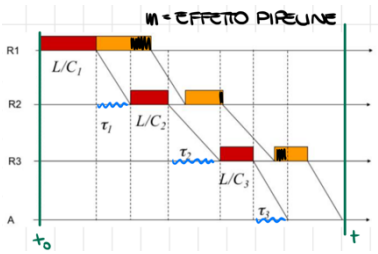


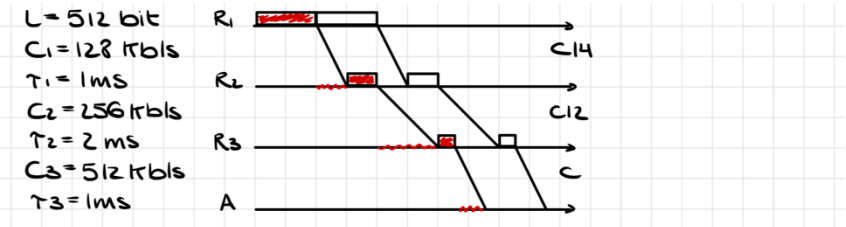
Da notare che v è la velocità del segnale e che si è definito il **RTT**, ovvero round trip time, il tempo impiegato dal pacchetto per raggiungere un host e tornare al punto di origine.

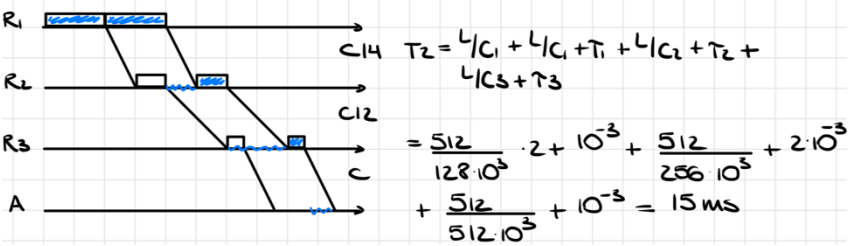
Esercizio 1.3 – si consideri la rete in figura. Al tempo t=0 la coda di uscita di R1 ha due pacchetti diretti ad A. Assumendo lunghezza dei pacchetti di L= 512 [bit], si indichi per ciascun pacchetto l'istante in cui viene completamente ricevuto a destinazione.



Bisogna notare che in questo caso entra in gioco l'**effetto pipeline**, infatti ci sono intervalli temporali durante i quali la trasmissione dei due pacchetti si sovrappone.

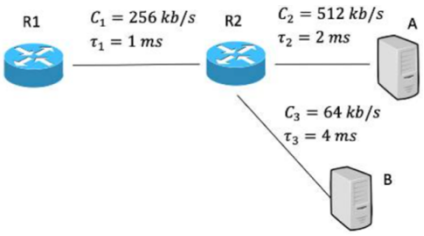


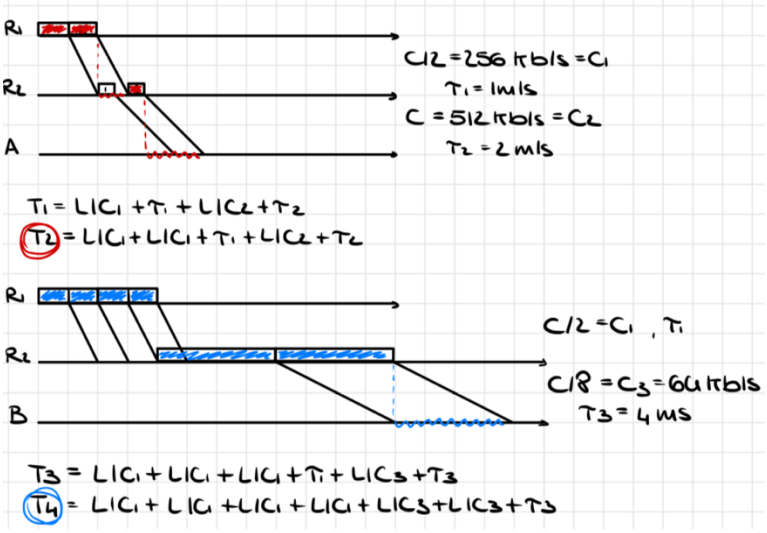




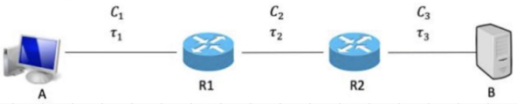
Se avessi dovuto trovare il valore del **throughput** in questo esercizio, avrei dovuto risolvere il rapporto tra la somma delle dimensioni dei due pacchetti inviati attraverso le tre interfacce, e il tempo totale di trasmissione, ovvero quello che va da t_0 a t: (512+512)/t_{TOT}.

Esercizio 1.4 – si consideri la rete in figura. Al tempo t=0 la coda di uscita di R1 ha 4 pacchetti diretti rispettivamente A, A, B, B. Assumendo lunghezza dei pacchetti di L=512 [bit], si indichi per ciascun pacchetto l'istante in cui viene completamente ricevuto a destinazione.

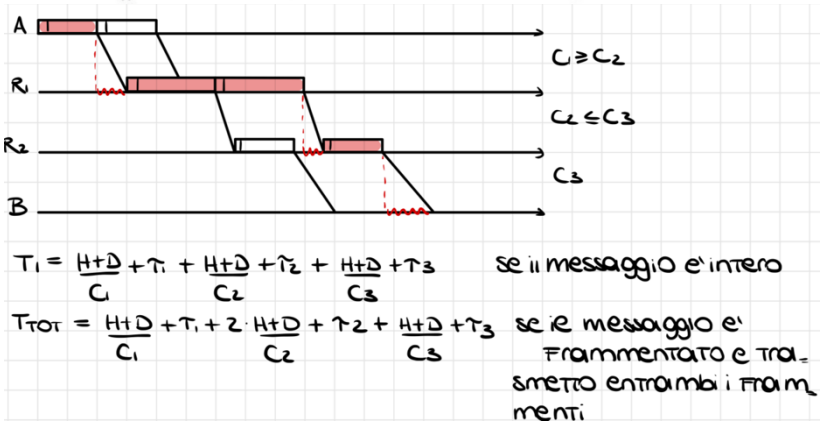




Esercizio 1.5 – si consideri la rete in figura.



a. Si calcoli in forma parametrica il tempo necessario a trasmettere un pacchetto da A a B (header H e dati D)
b. Si assume di dividere il pacchetto in due frammenti. Si calcoli in forma parametrica il tempo necessario per trasmettere tutti i frammenti. Si assuma $C_2 <= C_1 <= C_3$



c. Qual è il numero n di frammenti che minimizza il ritardo? Aumentando il numero di pacchetti migliora il tempo complessivo di trasferimento perché il parallelismo viene sfruttato al meglio e il ritardo diminuisce. Si può parlare dell'effetto chiamato ***trade off***: ogni volta che si genera un frammento aumenta l'effetto pipeline ma peggiora la dimensione totale dei dati da trasmettere perché ad ogni frammento si aggiungono i dati dell'header (peso maggiore).

$$T = \frac{h+D/n}{C_1} + \tau_1 + \frac{n(h+D/n)}{C_2} + \tau_2 + \frac{h+D/n}{C_3} + \tau_3 =$$

$$= \left(\frac{h}{C_1} + \tau_1 + \frac{D}{C_2} + \tau_2 + \frac{h}{C_3} + \tau_3\right) + \frac{D}{nC_1} + \frac{nh}{C_2} + \frac{D}{nC_3}$$

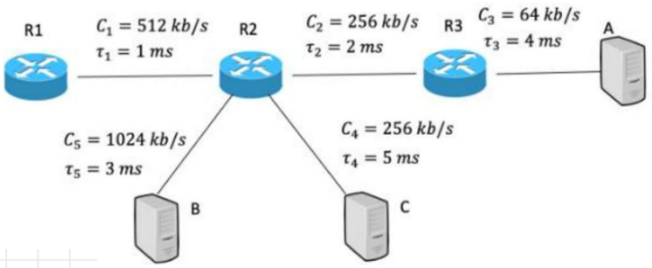
Troviamo il punto di minimo:

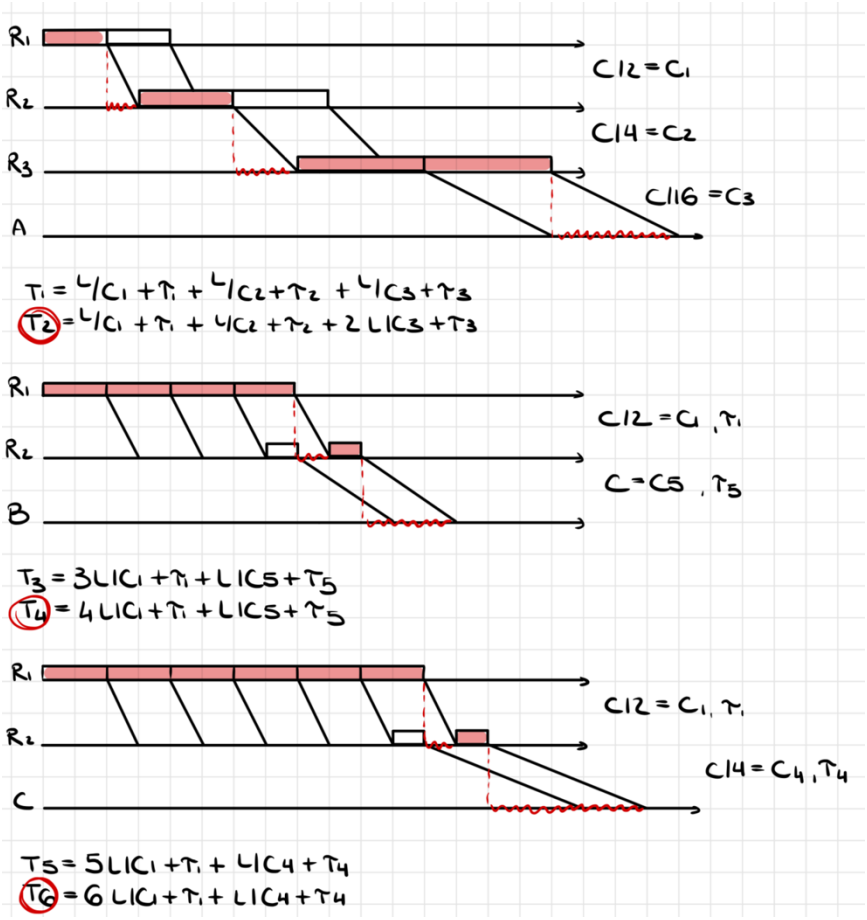
$$\frac{\partial T}{\partial n} = \frac{h}{C_2} - \frac{D}{n^2 C_1} - \frac{D}{n^2 C_3} = 0$$

$$n^* = \sqrt{\frac{C_2}{h}\left(\frac{D}{C_1} + \frac{D}{C_3}\right)}$$

d. Nel caso la lunghezza dell'header sia trascurabile, quale espressione assume il ritardo al crescere di n? Se non considerassi l'header la funzione precedente cambierebbe, il ritardo dipende dal collegamento che la capacità minima, torna il collo di bottiglia. La fascia di ritardo che ha pesato di più è quella con capacità C2, ovvero la capacità minima. La quantità di bit trasferiti è influenzata dal collo di bottiglia.

Esercizio 1.6 – si consideri la rete in figura. Al tempo t=0 la coda di uscita di R1 ha 6 pacchetti diretti rispettivamente A, A, B, B, C, C. assumendo lunghezza dei pacchetti di L=512[bit], si indichi per ciascun pacchetto l'istante in cui viene completamente ricevuto a destinazione.





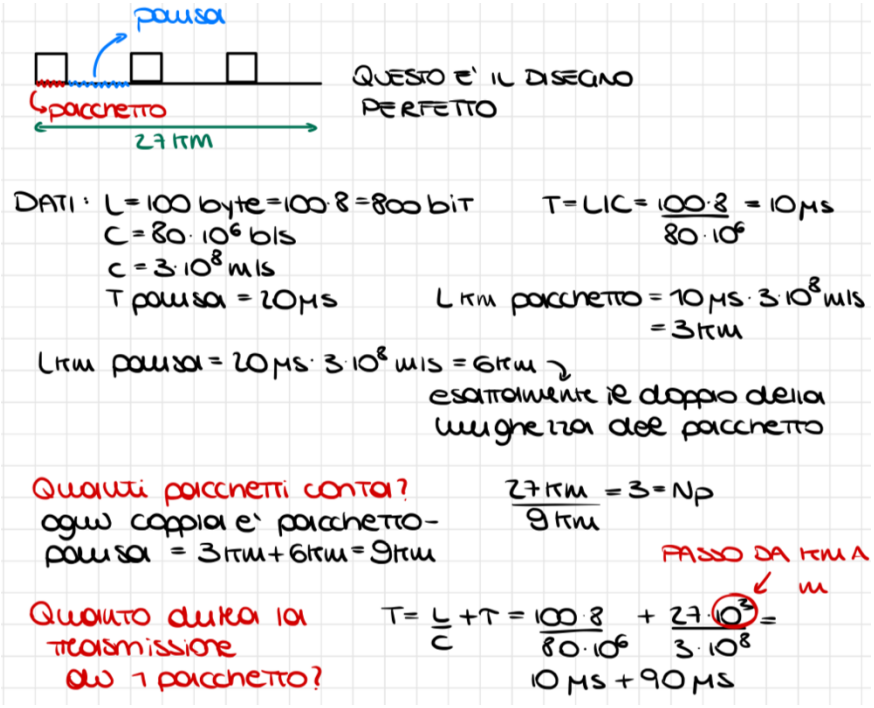
Ricorda che la coda di output dei pacchetti è diversa tra le diverse interfacce. Ciò significa che quando considero i pacchetti che vanno verso B, sull'interfaccia tra R1 e R2 non devo rappresentare le code dei primi due pacchetti perché sulle righe va messa la coda di output del canale successivo. Ecco perché sulla "riga" di R2 ho già rimpicciolito il pacchetto, quella è la dimensione che assumerà nella seconda interfaccia.

Esercizio 1.7 – quanti metri è "lungo" nel mezzo trasmissivo il segnale corrispondente ad un pacchetto di lunghezza L=75 byte durante la trasmissione su di un canale radio di capacità C=64 Mbit/s (velocità di propagazione pari alla velocità della luce nel vuoto)? Quanto dura la sua trasmissione? Per quanto riguarda gli esercizi di trasmissione su una generica interfaccia abbiamo tre componenti: lunghezza del pacchetto L, velocità di trasmissione C, velocità di propagazione che dipende dal mezzo, in questo caso la velocità della luce $c=3*10^8$m/s.

$$T = L/C = \frac{75 \cdot 8\ bit}{64 \cdot 10^6\ bit/s} = 9{,}375\ \mu s$$

$$\text{lunghezza } \ell = T \cdot v = 9{,}375 \cdot 10^{-6}\ s \cdot 0{,}3\ 10^8\ km/s = 2{,}8125\ km$$

Esercizio 1.8 – quanti pacchetti di dimensione L=100 byte si trovano in "volo" durante la trasmissione su di un canale radio di capacità C=80 Mb/s (velocità di propagazione pari alla velocità della luce nel vuoto) e lunghezza fisica 27 km? Tra i vari pacchetti c'è un tempo di pausa, ovvero un tempo che intercorre tra la trasmissione dell'ultimo bit di un pacchetto e la trasmissione del primo bit del pacchetto successivo, che è pari a 20 µs. Quanti secondi dura la trasmissione del singolo pacchetto?



Bisogna innanzitutto stabilire quale sia il tempo di trasferimento di un pacchetto, espresso come rapporto L/C e, successivamente, la lunghezza fisica di ciascun pacchetto ricordando che velocità=spazio/tempo. Dopodiché, sapendo che tra un pacchetto ed un altro c'è un tempo di pausa, bisogna trovare la lunghezza fisica di ciascuna pausa.

Una volta trovata la "coppia" pacchetto-pausa bisogna cercare quante ne entrano nella lunghezza di 27 km del canale dividendo questa lunghezza per la lunghezza della coppia.

La trasmissione di ciascun pacchetto è data dalla somma del tempo di trasmissione e del tempo di propagazione τ, ricordando che quest'ultimo è dato dal rapporto tra la lunghezza del mezzo e la velocità del segnale, in questo caso la velocità della luce.

Regole da ricordare – quando si rappresenta graficamente la trasmissione di un pacchetto tra le interfacce non bisogna rappresentare i pacchetti diretti verso altre interfacce perché avranno una diversa **coda di output**, come per l'esercizio 1.6.

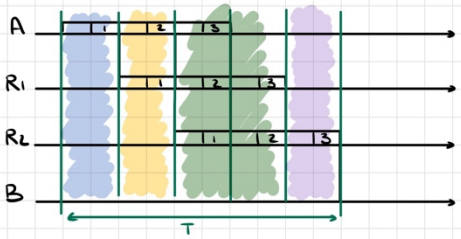
Preso un diagramma spazio-tempo che pone il tempo sull'asse delle ascisse e lo spazio sull'asse delle ordinate, la dimensione del pacchetto nel grafico (il rettangolo) esprime la durata della trasmissione dei dati lungo il canale, ovvero il **tempo di trasmissione** pari al rapporto L/C. L è la dimensione del pacchetto in bit, mentre C è il **bit-rate**, ovvero la capacità del canale trasmissivo, la quantità indicativa di dati espressi in bit che la rete può trasmettere in un secondo e si misura in bps (=b/s). Il **throughput**, invece, è la quantità reale di bit trasmessi per secondo. Ad esempio, nell'esercizio 3 si trova dividendo la quantità totale di bit trasmessi per il tempo totale di trasferimento.

Se il bit-rate aumenta il pacchetto è visivamente più corto perché il suo tempo di trasmissione, T=L/C, è inversamente proporzionale al bit-rate; se diminuisce è più lungo. L'ingrandimento, o la diminuzione, dei pacchetti tra due interfacce si disegna "prima". Il **ritardo di propagazione** τ, che si calcola come rapporto tra la lunghezza del mezzo d e la velocità di propagazione del segnale che dipende dal mezzo (es. velocità della luce), influenza l'inclinazione della linea che tiene conto del tragitto del pacchetto. Infatti, esprime il tempo impiegato dal segnale che trasferisce i bit a propagarsi lungo l'interfaccia. Più τ è alto più sarà inclinata. Se non è espresso il valore di τ ma solo quello di C, più C è alto più la linea sarà retta.

CASO GENERALE

Analizziamo il caso di 3 interfacce: un terminale A, 2 router, un terminale B. Ipotizzando di non avere ritardo di propagazione, un messaggio, frammentato in 3 pacchetti con lunghezza fissa e un campo utile "PAYLOAD" di dimensione L, lo dobbiamo trasmettere. Per calcolare di quanti pacchetti abbiamo bisogno per trasmettere il messaggio di dimensione M faccia-mo: $\lceil \frac{M}{L} \rceil$ = bit del quale prendiamo la parte superiore, ovvero avendo 2,7, i pacchetti saranno 3.

Ognuno dei 3 pacchetti aurà un'intestazione.
Non avere ritardo di propagazione significa che i pacchetti vengono mandati istantaneamente.



ciascuna interfaccia ha capacità C

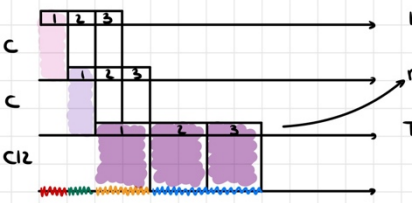
Si osservano i tratti in cui non si è verificato l'effetto pipeline e poi gli istanti in parallelo presi una volta sola!

$T = \frac{L}{C} + \frac{2L}{C} + \frac{L}{C} + \frac{L}{C} + 3\tau$ (2 cose contempor.)

se ci fosse il ritardo di propagazione si prenderebbe una volta per ogni interfaccia

Oppure si considerano i due istanti, il primo e l'ultimo con pacchetti singoli e poi nei 3 istanti con effetto pipeline i pacchetti ripetuti 1 volta sola: $L/C + 3L/C + L/C$.

CASO 1: CAMBIO VELOCITÀ SU UN'INTERFACCIA (L'ULTIMA)

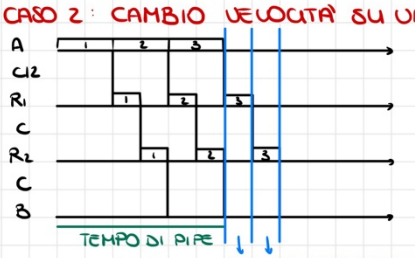


l'espansione di questo livello perché trasmettiamo sull'ultima interfaccia

$T = \frac{L}{C} + \frac{2L}{C/2} + \frac{L}{C} + \frac{L}{C/2}$

oppure $T = \frac{L}{C} + \frac{3L}{C/2} + \frac{L}{C}$ poiché abbiamo considerato il primo pacchetto quando viaggia sulle interfacce più veloci e, tutti e 3 sull'interfaccia più lenta.

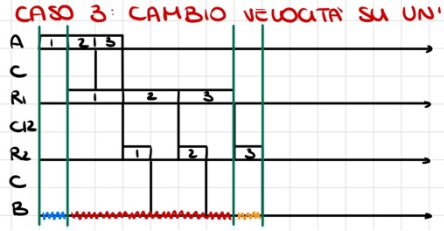
CASO 2: CAMBIO VELOCITÀ SU UN'INTERFACCIA (PRIMA)



$T = \frac{3L}{C/2} + \frac{L}{C} + \frac{L}{C}$

QUI NON C'È PIPELINE E DUNQUE PRENDO IL PACCHETTO 3 SINGOLARMENTE

CASO 3: CAMBIO VELOCITÀ SU UN'INTERFACCIA (CENTRO)



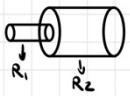
nella prima interfaccia si può vedere l'effetto dell'accodamento

$T = \frac{L}{C} + \frac{3L}{C/2} + \frac{L}{C}$ perché ho 3 pacchetti e pipeline

Si parla di collo di bottiglia quando c'è un fattore, nel calcolo del tempo di trasmissione, che rallenta T. Dunque ovunque e qualunque sia la capacità più lenta questa causa il collo di bottiglia.

ie Throughput è pari a $\lceil \frac{M}{T} \rceil$ bit

il collo di bottiglia qui è causato da $3L/C/2$



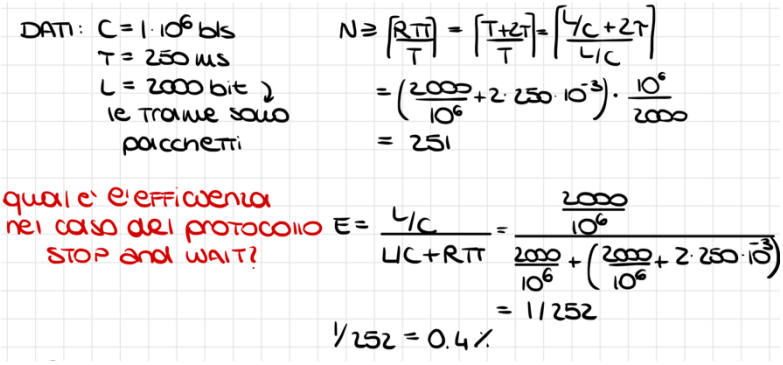
In questo caso ho supposto il tempo di propagazione pari a 0, altrimenti a T andrei aggiunto tanti τ quante sono le interfacce attraversate, indipendentemente dal numero di pacchetti.

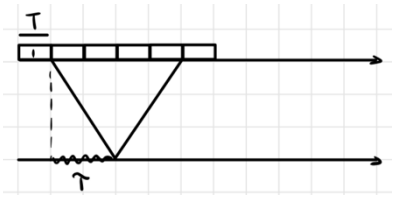
Se avessi avuto 10 pacchetti, in questo caso:
$T = \frac{L}{C} + \frac{10L}{C/2} + \frac{L}{C} + 3\tau$

e se i τ fossero stati diversi: $+\tau_1 + \tau_2 + \tau_3$

Esercizio 3.1 – si consideri un canale via satellite della capacità di 1 Mb/s. Considerando che il tempo di propagazione attraverso un satellite geostazionario richiede 250 ms, si chiede di dimensionare la minima finestra di trasmissione di un protocollo Go-Back-N (con time-out) in modo che sia consentita la massima efficienza temporale del canale quando vengano trasmesse trame di 2000 bit in assenza di errori. Si suppongano gli ACK trascurabili. Si calcoli poi la massima efficienza trasmissiva che si avrebbe nel caso in cui il meccanismo ARQ fosse di tipo Stop&Wait.

Innanzitutto bisogna ricordare che la trasmissione è inefficiente se non si riuscisse a coprire l'intervallo stabilito dal RTT, ovvero round trip time, il tempo impiegato dal pacchetto per raggiungere un host e tornare al punto di origine. Ecco perché per trovare la dimensione minima della finestra di trasmissione W_s (=N) bisogna dividere il RTT per il tempo di trasmissione di un singolo pacchetto: $W_s >= RTT/T$, dove $T=L/C$.

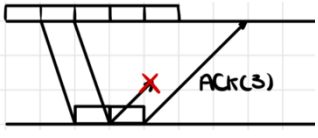


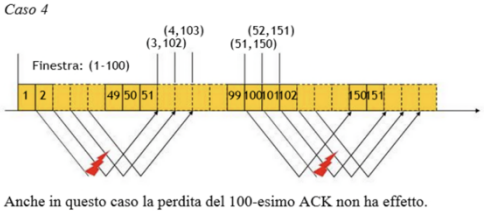


La prima domanda in pratica chiedeva quante fossero le unità informative che compongono la minima finestra di trasmissione in modo da avere efficienza 1. Ricorda che la trasmissione è inefficiente se **NT<RTT**, mentre è continua se **NT>=RTT**. In questo caso, a causa del ritardo di propagazione, il prodotto banda-ritardo è molto alto e pertanto l'efficienza molto bassa.

Esercizio 3.2 – un sistema che utilizza un protocollo Go-Back-N presenta *un ritardo di propagazione pari a 24 volte il tempo di trasmissione* di un pacchetto e viene usato per inviare un file di 1000 pacchetti. Si assuma la durata del timeout pari al tempo di trasmissione dell'intera finestra di pacchetti. Ipotizzando che tutti i pacchetti ricevuti correttamente siano riscontrati (*tempo di trasmissione del riscontro = tempo di trasmissione del pacchetto*), si calcoli il numero di pacchetti trasmessi inutilmente (errati o corretti ma scartati dal ricevitore), nel caso in cui la finestra sia lunga N=100 quando:

a. Si sbaglia il primo pacchetto del file: se il temporizzatore è impostato pari alla finestra di trasmissione significa che devono essere prima trasmessi tutti i pacchetti della finestra, poi il primo può essere ritrasmesso. Dunque sono stati trasmessi inutilmente 99 pacchetti.
b. Si sbagliano il primo e il 100esimo pacchetto del file: essendo la finestra di trasmissione sempre composta da 100 pacchetti (il 100esimo è incluso), si presenta la situazione precedente.
c. Si sbaglia l'ACK del primo pacchetto del file: l'ACK(2) relativo al primo pacchetto, ovvero quello che chiede la trasmissione del secondo, si perde. Nonostante ciò, il secondo pacchetto viene inviato e di esso non si perde l'ACK, che in questo caso chiederà l'invio del pacchetto 3. Poiché il protocollo Go-Back-N utilizza l'ACK cumulativo, una volta ricevuto l'ACK(3), il trasmettitore sa che il primo pacchetto è stato ricevuto con successo.
d. Si sbaglia l'ACK del primo e del 100esimo pacchetto del file. (Guarda l'immagine del caso 4)

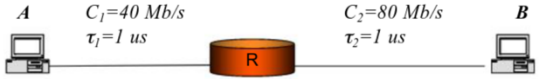




Anche in questo caso la perdita del 100-esimo ACK non ha effetto.

Il testo dice che la finestra di trasmissione è lunga 100 pacchetti ma, il ritardo di propagazione occupa lo spazio di 24 pacchetti, ciò significa che il primo riscontro arriva dopo il 24esimo pacchetto.

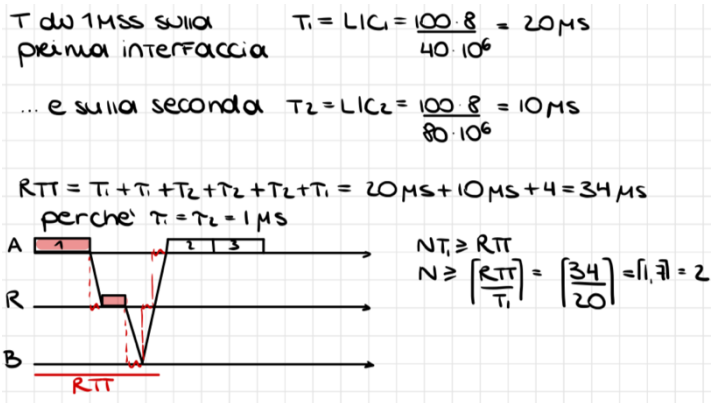
Esercizio 3.3 – si consideri la rete in figura in cui tra l'host A e l'host B sia attivata una connessione TCP. Si calcoli l'istante di tempo dall'attivazione della connessione oltre il quale la trasmissione sul link 1 risulta continua, supponendo



i. header trascurabili
ii. link bidirezionali e simmetrici
iii. RCWND= 4000[byte] e SSTHRESH= 400[byte]
iv. dimensione segmenti MSS= 100[byte]
v. dimensione ACK = dimensione segmenti per apertura della connessione = trascurabile

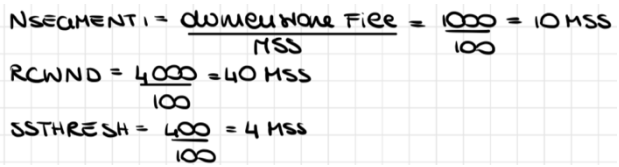
Quanto tempo occorre per trasferire un file da 1[kbyte] sulla connessione TCP sopra specificata (dall'istante di trasmissione del primo segmento all'istante di ricezione dell'ACK dell'ultimo segmento)? ***È sempre utile trasformare tutti i dati in modo che dipendano da MSS!***

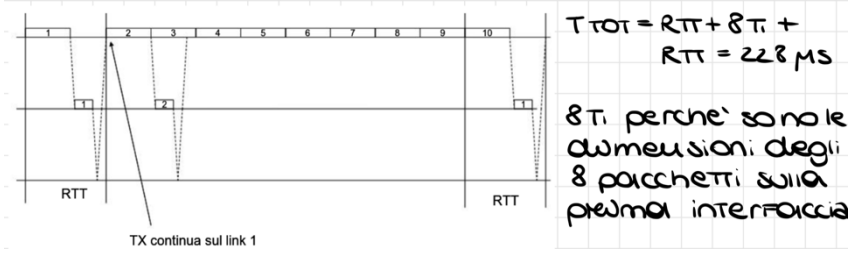
Poiché stiamo considerando una connessione TCP, bisogna analizzarne tutte le fasi e, non essendoci ulteriori specificazioni, ipotizziamo di stare analizzando un TCP Tahoe, pertanto, con la fase slow start. Ci troviamo a dover spedire prima un MSS e poi comincia la fase di crescita esponenziale fino alla ssthresh, da cui comincia la fase additiva lineare.



Innanzitutto si calcola il tempo totale impiegato per trasmettere un MSS attraverso le due interfacce con il relativo riscontro, ovvero il cosiddetto RTT. Una volta calcolato il tempo impiegato da un MSS per essere trasmesso è possibile stabilire il numero minimo di pacchetti necessari per riempire il RTT ed assicurare una trasmissione continua: NT>=RTT.

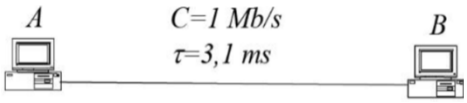
Calcoliamo il numero di segmenti da trasmettere lungo il canale, dividendo la dimensione del file per la grandezza di un MSS. Avendo stabilito che ci troviamo nella fase slow start e che per avere una trasmissione continua bisogna aspettare solo un RTT perché dopo aver spedito il primo pacchetto, a causa della crescita esponenziale, ne invierò 2 e avrò quindi già rispettato la condizione NT>=RTT; per calcolare il tempo totale di trasmissione di un file di 1[kbyte] basterà sommare il primo RTT con il tempo di trasmissione di 8 pacchetti (da 2 a 9) e l'ultimo RTT, ovvero quello del decimo pacchetto.





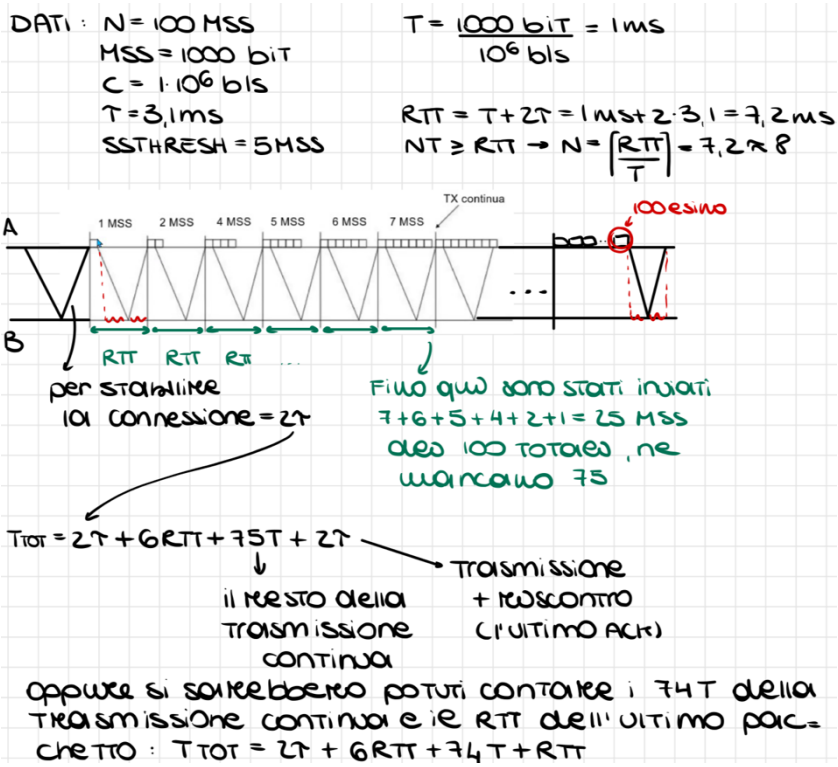
Se la rcwnd fosse stata composta da 4 segmenti l'evoluzione non sarebbe cambiata perché si trasmette sempre il minimo tra rcwnd e cwnd.

Esercizio 3.4 – si consideri il collegamento in figura tra i due host A e B. A deve trasferire una sequenza di 100 segmenti di lunghezza massima usando TCP. Si calcoli il tempo necessario supponendo:



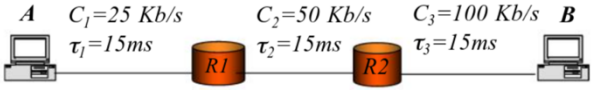
i. MSS= 1000[bit]
ii. Lunghezza degli header di tutti i livello trascurabile
iii. La connessione venga aperta da A e la lunghezza dei segmenti di apertura della connessione sia trascurabile
iv. La lunghezza degli ACK sia trascurabile
v. SSTHRESH sia pari a 5 MSS

Ricorda: col protocollo TCP, prima di cominciare ad inviare pacchetti nel canale, è necessario stabilire la connessione tra trasmettitore e ricevitore attraverso uno scambio di SYN, SYN/ACK e ACK (nel caso http i dati iniziano ad arrivare dopo 2RTT).



Sto considerando una connessione **TCP Tahoe**, ciò significa che si manda innanzitutto un MSS col relativo riscontro, dopodiché, trovando nella fase slow start, inizia l'incremento esponenziale fino al momento in cui si raggiunge il valore "limite" della ssthresh, in corrispondenza del quale comincia la fase additiva lineare. Bisogna notare che la trasmissione sarà discontinua finché non sarà raggiunta una finestra di trasmissione, rcwnd, di dimensione *almeno* pari al rapporto tra il RTT e il tempo di trasmissione T. In questo caso il tempo di trasmissione di un pacchetto è pari ad 1ms, mentre 1RTT è pari a 7,2ms ≈ 8ms, ciò significa che, essendo T=1ms, la trasmissione sarà continua solo a partire dall'invio di 8 pacchetti.

Esercizio 3.5 – all'istante 0 viene attivata una connessione TCP tra l'host A e l'host B. Si calcoli l'istante di tempo oltre il quale la trasmissione sul link 1 risulta continua, supponendo

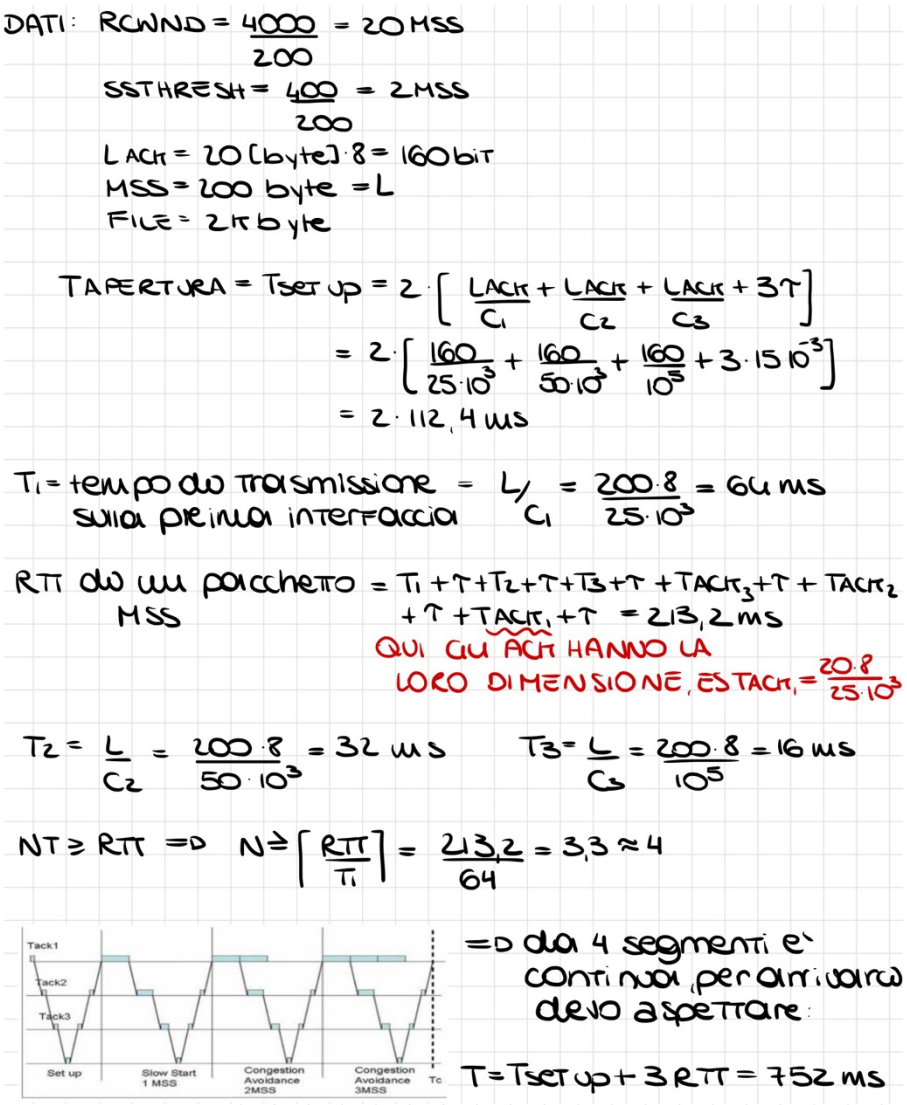


i. Header trascurabili
ii. Link bidirezionali e simmetrici
iii. RCWND= 4000[byte] e SSTHRESH= 400[byte]
iv. Dimensione segmenti MSS= 200[byte]
v. Dimensione ACK = dimensione segmenti per apertura della connessione = 20 [byte]
vi. Connessione aperta dal terminale A

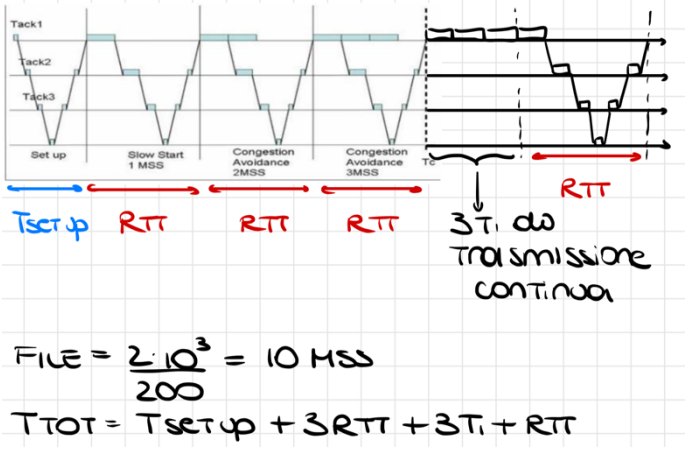
Quanto tempo occorre per trasferire un file da 2[kbyte] (dall'istante di trasmissione del primo segmento all'istante di ricezione dell'ACK dell'ultimo segmento)?

In apertura vengono mandati dei segmenti di dimensione 20 [byte] con i relativi ritardi di propagazione sulle tre interfacce. Il tempo di trasmissione di questi segmenti per aprire la connessione si calcola come somma dei loro tempi di trasmissione sulle varie interfacce, con i tempi di propagazione, in questo modo si trova quanto tempo si impiega per raggiungere l'host B, dopodiché si moltiplica per due perché tornano all'host A.

Sapendo che la ssthresh vale 2 MSS, dopo aver mandato due pacchetti (fase di incremento esponenziale), inizia la fase additiva lineare. Per sapere dopo quanto tempo si può parlare di trasmissione continua bisogna innanzitutto stabilire la dimensione minima della finestra di trasmissione, ovvero NT>=RTT, dopodiché vedere quanti RTT ci vogliono per arrivare a quel valore.



Per sapere quanto tempo ci vuole per trasmettere un file di 2[kbyte], innanzitutto bisogna trovare a quanti MSS corrisponde (10), dopodiché si considera il tempo di set up della connessione, 3RTT di trasmissione non continua e così si arriva a 1+2+3= 6 MSS, ne mancano 4 di MSS, ovvero $3T_1$ + il RTT dell'ultimo segmento.



Esercizio 3.6 – una connessione TCP è usata per trasmettere un file da 39.5 [kbyte] utilizzando i seguenti parametri: MSS= 500[byte], RTT= 500[ms], time-out pari a 2 RTT. Si assuma che le *condizioni iniziali* delle finestre siano: RCWND= 12[kbyte], SSTHRESH= 8[kbyte], CWND= 500[byte]. E che si verifichi un *errore sulla connessione* all'istante 3s, ovvero tutti i segmenti in trasmissione vengano persi, e al tempo 4,5s il ricevitore segnali RCWND= 2[kbyte]. Si tracci l'andamento nel tempo di CWND, RCWND e SSTHRESH. Si calcoli il *tempo* di trasmissione del file utilizzando *multipli di RTT* come base temporale.

In esercizi di questo tipo solitamente si assume di ricevere un ACK ogni RTT, pertanto si misura il tempo in base ai RTT che notificano degli eventi come l'aggiornamento del valore della congestion window (*cwnd*), ovvero gli MSS inviati. A meno che venga specificato, si considera di studiare un protocollo **TCP Tahoe** caratterizzato unicamente dalla fase di slow start.

Dopo aver stabilito i valori di cwnd, rcwnd e ssthresh, si parte dall'istante di tempo t=0 con l'invio di 1 pacchetto di lunghezza MSS. Avendo come unità di misura un RTT, ovvero 0,5s, ogni mezzo secondo viene aggiornata la congestion window sulla base dell'incremento esponenziale e si avrà:

Istante di Tempo (t)	Segmenti Inviati (MSS)
0s	1
0,5s	2
1s	4
1,5s	8
2s	16

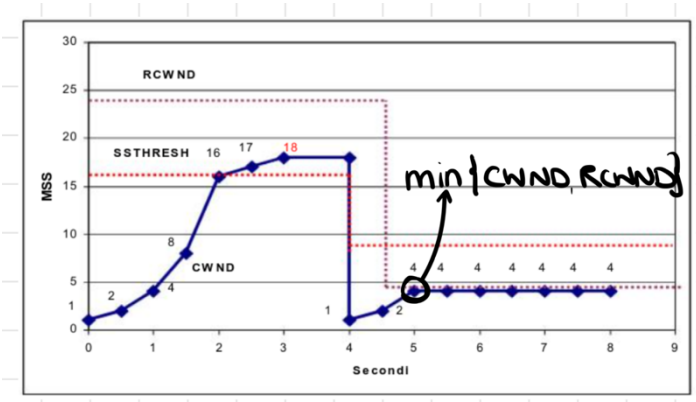
Arrivati a questo punto ci si trova in corrispondenza del valore limite rappresentato dalla ssthresh che ci costringe a proseguire con la fase additiva lineare fino all'evento di errore, in corrispondenza di 3s, quando tutti i pacchetti fino a quel momento trasmessi vanno persi:

t= 2,5s -> 17MSS
t= 3s -> 18MSS

Una volta persi 18+17+16+8+4+2+1 = 66MSS parte il **time-out** che sarà pari, come da traccia, a 2 RTT (= 0,5s*2 = 1s), graficamente rappresentato da un segmento orizzontale fino ad arrivare a 4s perché durante quegli istanti il trasmettitore resta in ascolto. *Se non fosse stata specificata la lunghezza del time-out sarebbe bastato ricordarsi che deve essere almeno pari al RTT affinché permetta al trasmettitore di capire l'esito della trasmissione.*

L'ultimo valore della congestion window è settato a 18, pertanto, la nuova ssthresh da impostare proprio pari alla sua metà sarà di dimensione 9MSS. In questo momento il protocollo TCP Tahoe torna alla fase di **slow start** (*se avessimo considerato TCP Reno, in questo momento, avremmo cominciato ad inviare una quantità di pacchetti pari alla nuova ssthresh con fase additiva lineare*).

Dunque si parte con l'incremento esponenziale fino a che il valore della cwnd raggiunga il limite della nuova ssthresh. In corrispondenza dell'istante di tempo 4,5s la rcwnd si impone essere pari a 2[kbyte], ovvero 4MSS. Poiché ogni volta che si trasmettono pacchetti è come se si effettuasse la scelta **minima tra {cwnd, rcwnd}**, non potendo spedire più pacchetti di quelli che la receiver window consente, l'andamento resterà costante a 4MSS finché non verranno spediti tutti i 79 pacchetti, ricordando che i primi 66 erano andati persi.



DATI: MSS = 500 byte
RTT = 500 ms = 0,5s
TIMEOUT = 2 RTT = 2·0,5 = 1s
rcwnd = 12·10³ byte / 500 byte = 24 MSS
cwnd = 500 byte / 500 byte = 1 MSS
SSTHresh = 8·10³ byte / 500 byte = 16 MSS

TOT PACCHETTI INVIATI = DIMENSIONE FILE / MSS = 39,5·10³ / 500 = 79 MSS

al tempo 4,5s : rcwnd = 2·10³ / 500 = 4 MSS

Esercizi capitolo 4

All'interno della rete principale è possibile creare reti più piccole, attraverso la pratica di subnetting che permette di ottimizzare l'utilizzo degli indirizzi IP e migliorare la gestione e la sicurezza delle reti. In questo modo ogni sottorete avrà il proprio intervallo di indirizzi IP e la propria maschera di sottorete, un parametro che definisce la dimensione della sottorete IP a cui appartiene un host, ovvero definisce l'intervallo di indirizzi IP della sottorete.

Dunque, per definire la sottorete avremo il suo indirizzo IP seguito dalla notazione decimale relativa alla sua maschera, ovvero il range di indirizzi che identificano la sottorete. Per esprimere questo intervallo, la maschera utilizza la notazione CIDR: **a.b.c.d/x**, dove **x** è la maschera, il numero di bit dal più al meno significativo (da sinistra a destra) che compongono l'indirizzo di rete. Infatti, tutti gli indirizzi IP di una sottorete condividono un prefisso, ovvero il net ID, messo in evidenza dalla maschera, così come la porzione relativa all'host specifico. Come l'indirizzo IP di livello quattro (IPv4), la maschera di sottorete è composta da 32 bit suddivisi in 4 byte, dei quali un certo numero impostati ad 1 per indicare la rete, gli altri a 0 per indicare la parte dedicata agli host.

Per trovare il range di indirizzi che identificano la sottorete è possibile calcolare quanti host sono rappresentabili, infatti su 32 bit totali, **x** sono dedicati alla rete e i restanti **32 – x** agli host. Dunque, saranno rappresentabili $\mathbf{2^{(32-x)}}$ host, quelli realmente utilizzabili sono $\mathbf{2^{(32-x)} - 2}$ perché bisogna escludere l'indirizzo di rete e l'indirizzo broadcast.

Quando il sistema operativo riceve da un programma la richiesta di inviare un pacchetto IP ad un certo indirizzo IP destinatario, per prima cosa calcola l'AND logico fra la maschera di sottorete e il proprio indirizzo IP, e lo confronta con l'AND logico tra la maschera di sottorete e l'indirizzo IP di destinazione. Se il risultato è identico, ovvero il prefisso è lo stesso, allora significa che i due host appartengono alla stessa sottorete e il pacchetto può essere inviato nella rete locale.

Ad esempio, abbiamo indirizzo IP: 192.168.32.97, maschera di sottorete: 255.255.255.224, ovvero 192.168.32.97/27 e chiediamo di connetterci all'indirizzo IP 192.168.32.130. Prima di tutto trasformiamo in notazione binaria gli indirizzi IP e la maschera di sottorete:

```
192.168.032.097 = 11000000.10101000.00100000.01100001
192.168.032.130 = 11000000.10101000.00100000.10000010

255.255.255.224 = 11111111.11111111.11111111.11100000
```

Allora il livello IP calcolerà:

```
11000000.10101000.00100000.01100001 AND      (192.168.032.097)
11111111.11111111.11111111.11100000 =        (255.255.255.224)
-----------------------------------
11000000.10101000.00100000.01100000          (192.168.032.096)
```

Ora ripetiamo l'operazione con l'IP di destinazione:

```
11000000.10101000.00100000.10000010 AND      (192.168.032.130)
11111111.11111111.11111111.11100000 =        (255.255.255.224)
-----------------------------------
11000000.10101000.00100000.10000000          (192.168.032.128)
```

I risultati 192.168.32.96 e 192.168.32.128 indicano due sottoreti differenti, e quindi le macchine appartengono a sottoreti differenti.

Ricorda le regole dell'operazione **AND** e **OR** e **XOR**:

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Inoltre bisogna ricordare che quando negli esercizi si definisce un indirizzamento IP di tipo *classfull* è sottointesa la richiesta di specificare la classe alla quale si fa riferimento. Per far ciò basta prendere l'indirizzo IP fornito dall'esercizio e convertire in binario il primo byte per osservarne i primi bit.

Se il primo bit è posto a 0 avremo la **classe A** con relativa maschera di sottorete 255.0.0.0, ciò implica la dicitura in formato CIDR /8 e che avremo 24 bit per la parte host. Dunque, 1 byte per la net ID e 3 byte per l'host ID.

Se i primi due bit sono impostati a 10 avremo la **classe B** con la relativa maschera di sottorete 255.255.0.0, ciò implica la dicitura in formato CIDR /16 e che avremo 16 bit per la parte host. Dunque, 2 byte per la net ID e 2 byte per l'host ID.

Se i primi tre bit sono impostati a 110 avremo la **classe C** con la relativa maschera di sottorete 255.255.255.0, ciò implica la dicitura in formato CIDR /24 e che avremo 8 bit per la parte host. Dunque, 3 byte per la net ID e 1 byte per l'host ID.

Subnetting nelle reti IPv4 – L'Internet Protocol nella versione 4 si avvale di due indirizzi speciali, quali l'indirizzo di rete e l'indirizzo broadcast, rispettivamente il primo e l'ultimo della sottorete ottenuta con l'operazione di subnetting. Quando si ha una rete e la si scompone in N sottoreti, ognuna controindicata da un ordine che parte da 0 fino a N-1 (#0, #1, …, #N-1), ciascuna sottorete ha i propri indirizzo di rete e broadcast.

Rete	Rete (binario)	Indirizzo broadcast
192.168.5.0/26	11000000.10101000.00000101.**00**000000	192.168.5.63
192.168.5.64/26	11000000.10101000.00000101.**01**000000	192.168.5.127
192.168.5.128/26	11000000.10101000.00000101.**10**000000	192.168.5.191
192.168.5.192/26	11000000.10101000.00000101.**11**000000	192.168.5.255

Ad esempio, abbiamo l'indirizzo IP 192.168.5.0 e, la maschera di sottorete /26 equivalente a 255.255.255.192. Immaginando di voler creare 2^2 sottoreti, ovvero quattro, con una maschera di quel tipo si deduce che i bit dedicati alla parte host-ID sono sei e dunque gli host rappresentabili $2^6 = 64$. Ciascuna delle quattro sottoreti conterrà 64 indirizzi, comprendenti indirizzo di rete e broadcast.

Da cui si deduce che per la sottorete #3 l'ultimo host disponibile sarà proprio 192.168.5.254.

Per questi esercizi è utile ricordare:

$2^1 = 2$	$2^2 = 4$	$2^3 = 8$	$2^4 = 16$	$2^5 = 32$	$2^6 = 64$	$2^7 = 128$	$2^8 = 256$	$2^9 = 512$	$2^{10} = 1024$	$2^{11} = 2048$	$2^{12} = 4096$

Esercizio 4.1

Nel sistema di indirizzamento IP classfull, si consideri l'indirizzo della rete 129.16.0.0

a) *Quante sottoreti /22 possono essere ricavate dalla rete base, assumendo che un identificatore di subnet può anche essere costituito da tutti 0 o tutti 1?*
Innanzitutto, quando si parla di indirizzamento IP classfull bisogna stabilire a quale classe appartiene convertendo il primo byte in binario per cercare la corrispondenza dei primi bit. Il numero 129 in binario equivale a 10000001, i primi due bit sono 10 e dunque la classe è la B, con la relativa maschera di sottorete 255.255.0.0, in formato CIDR /16. La domanda richiede il numero di **blocchi di classe B**, ovvero le sottoreti che si ottengono passando da una maschera /16 ad una /22, perciò mancano 22-16 = 6 bit. Questi permettono di ottenere $2^{(22-16)} = 2^6 = 64$ possibili sottoreti, includendo indirizzo di rete e indirizzo broadcast.

b) *La sottorete 129.16.248.0/22 è la sottorete #62 della rete base.*
129.16.248.0/22 è una delle possibili 64 sottoreti, in particolare vogliamo sapere il suo numero d'ordine, ovvero quale tra le 64 sia nello specifico. Le 64 opzioni si ottengono da tutte le possibili combinazioni di 0 e 1 dei 6 bit aggiunti ai 16 per avere maschera /22. Sapendo che 129.16.248.0 = 10000001.00010000.**111110**00.00000000 dove la parte sottolineata è equivalente alla maschera /22 e i bit rossi sono i 6 dai quali si ottengono 64 sottoreti. Convertendo quei 6 bit in decimale si trova l'ordine della sottorete, infatti 111110 = 62.

c) *Si partizioni ulteriormente la sottorete 129.16.248.0/22 in N sottoreti /n che permettano di indirizzare almeno 64 host ognuna (a questi host-ID adiacenti a partire dal valore più piccolo possibile). Qual è la lunghezza del prefisso di sottorete/n? Quante sottoreti N con prefisso /n è possibile creare?*
Avendo la sottorete 129.16.248.0/22, di questi 32 bit totali, 22 sono adibiti al prefisso della sottorete, mentre i restanti 10 a tutte le possibili rappresentazioni di host, ovvero 2^{10} =1024. Per organizzare 1024 host in N sottoreti di 64 host ciascuna bisogna ricordare che vanno aggiunti per ciascuna sottorete gli indirizzi speciali, ecco perché ogni sottorete sarà composta da 66 indirizzi. Per trovare la **maschera**: **1)** si trovano i bit necessari per rappresentare 66 indirizzi, $2^6 = 64$ indirizzi sono insufficienti, mentre con 7 bit ottengo $2^7 = 128$ indirizzi; **2)** se 7 bit sono dedicati all'host, 32 – 7= 25 saranno dedicati al net ID, la maschera sarà /25. Per trovare quante **sottoreti**: passando da una maschera /22 ad una /25 si possono ricavare $2^{(25-22)} = 2^3 = 8$ sottoreti dalla rete base, con 3 bit variabili.
Si scriva in formato decimale la maschera delle sottoreti /n. Abbiamo stabilito che la sottorete è /25 in formato CIDR, ciò significa che 25 bit della maschera sono impostati ad 1 e i restanti 7 a 0: 11111111.11111111.11111111.10000000. Questo numero in notazione decimale equivale a scrivere: 255.255.255.128.

d) *Si scrivano in formato decimale e binario:*
   1. *L'indirizzo broadcast della sottorete /n #0.* L'indirizzo broadcast è sempre l'ultimo indirizzo IP della sottorete, in questo caso abbiamo l'indirizzo della sottorete di ordine zero, #0, pari a 129.16.248.0/25, poiché gli indirizzi da rappresentare sono 66 abbiamo visto che serve una maschera /25 con 7 bit per la parte host-ID. Perciò in realtà permette di rappresentare 128 indirizzi a partire da 0, si arriverà fino a 127 (l'ultimo). Pertanto, l'indirizzo broadcast della sottorete /25 #0 sarà 129.16.248.127/25.

      In realtà sarebbe bastato prendere l'indirizzo IP della sottorete di ordine 0, convertirlo in bit e cambiare gli ultimi 7 bit, ovvero quelli che la maschera riserva alla parte host ID, impostandoli ad 1 perché l'indirizzo broadcast, l'ultimo della sottorete, ha questa

struttura. 10000001.00010000.11111000.0**0000000** ->
10000001.00010000.11111000.0**1111111** = 129.16.248.127

Oppure…
NOTA BENE: la regola per trovare l'indirizzo broadcast è la seguente, si prende l'indirizzo IP scritto in binario e la maschera, quest'ultima la si inverte bit a bit, ovvero gli 1 diventano 0, e viceversa. Dopodiché si trova l'indirizzo broadcast con l'operazione dell'OR logico. Mentre l'indirizzo di rete è corrispondente all'AND logico tra i bit dell'indirizzo IP e la maschera. In questo caso avremo:

Indirizzo IP	10000001.00010000.11111000.00000000
Maschera	00000000.00000000.00000000.01111111
Operazione OR	
Indirizzo Broadcast	10000001.00010000.11111000.01111111

2. *L'indirizzo dell'ultimo host, quello dall'indirizzo più alto, della sottorete /n #3.*
   1) Troviamo l'indirizzo della sottorete di ordine 3, se si è passati dalla maschera /22 alla /25 significa che le 8 possibili sottoreti sono ottenute dalle combinazioni di 0 e 1 degli ultimi due bit del terzo byte e il primo del quarto byte: 129.16.248.0 = 10000001.00010000.111110**00.0**0000000. La sottorete di ordine 3 avrà quei tre bit impostati in modo da indicare l'ordine specifico, dunque si usano per scrivere 3: se mi trovo #0 avrò 00.0, #1 avrò 00.1, #2 avrò 01.0 e #3 avrò 01.1, il numero 3 in binario si scrive come 11 e, dovendolo scrivere con 3 bit mi darà 01.1.
   2) L'ultimo indirizzo IP della sottorete di ordine 3 è l'indirizzo di broadcast che imposta i 7 bit dell'host ID ad 1 (10000001.**00**010000.111110**01.1**1111111), a noi interessa prendere il penultimo, ovvero l'ultimo assegnabile ad un host. Pertanto, l'ultimo indirizzo host utilizzabile è quello precedente all'indirizzo broadcast, ovvero 10000001.**00**010000.111110**01.1**1111110 che si traduce in 129.16.249.254/25.

   Avremmo potuto anche ragionare in un altro modo: se l'indirizzo della sottorete di ordine 3 è 10000001.**00**010000.111110**01.1**0000000 = 129.16.249.128, sapendo che ognuna di queste 8 sottoreti contiene 128 indirizzi assegnabili, sarebbe bastato considerare i successivi 128 e arrivare a 129.16.249.255, ma sapendo che l'ultimo è l'indirizzo broadcast, selezionare il precedente: 129.16.249.254.

Esercizio 4.2

Quando un operatore una volta ricevuto un indirizzo deve generare quattro sottoreti usando maschere a lunghezza variabile che meglio matchano il set di host che si aspetta, deve cercare di sprecare il minor numero possibile di indirizzi a disposizione. Dunque avendo quattro sottoreti con un numero di host differente tra l'una e l'altra, ***secondo la regola, bisogna partire dalla sottorete più grande****. In questo caso avendo rete I con 92 host, rete II con 18 host, rete III con 5 host e rete IV con 4 host, si parte dalla prima per poi continuare scendendo in ordine decrescente.*

Bisogna riempire la tabella trovando Subnet ID, ovvero l'indirizzo della sottorete, la sua maschera, il numero di host richiesti e quelli disponibili. Sapendo che l'indirizzo IP ricevuto dall'operatore è 201.184.237.0 = 11001001.10111000.11101101.00000000, ovvero un indirizzo di classe C con maschera di default /24 = 255.255.255.0. Ricorda che nella classe, per ciascun prefisso di rete è possibile rappresentare 256 indirizzi perché 8 sono i bit dell'host ID $2^{32-24} = 2^8 = 256$

Rete I – la prima sottorete deve contenere 92+2= 94 indirizzi, perché ciascuna sottorete ha i propri indirizzo di rete e broadcast.

1. *Trovare i bit necessari per rappresentare 94 indirizzi:* 2^7= 128, dovremo riservare 7 bit all'host ID.
2. *Trovare la maschera di sottorete:* se 7 sono i bit dell'host ID, allora 32-7= 25 sono i bit della net ID. In questo modo avremo la maschera /25.
3. *Assegnabili:* gli indirizzi assegnabili sono 126 perché di quei 128 bisogna scartare i due speciali.
4. *Richiesti:* 92.
5. *Prefisso:* essendo il primo blocco a dover essere riempito, si utilizza l'indirizzo di rete fornito all'operatore, ovvero 201.184.237.0. Sapendo di dover riservare 128 indirizzi, l'ultimo indirizzo del range di questo primo blocco sarà 201.184.237.127.

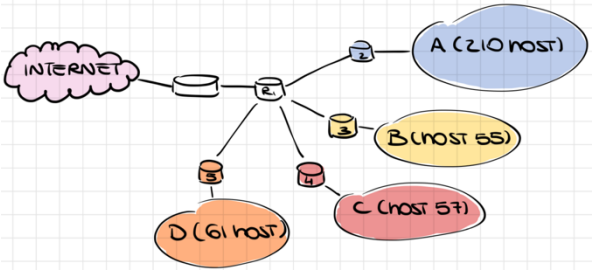
Rete II – la seconda sottorete deve contenere 18+2= 20 indirizzi, perché ciascuna sottorete ha i propri indirizzo di rete e broadcast. Dunque avremo 2^5= 32, dovremo riservare 5 bit all'host ID e avremo così 32-5= 27 bit per la net ID. In questo modo avremo la maschera /27. Gli indirizzi assegnabili sono 30 perché di quei 32 bisogna scartare i due speciali. Host richiesti: 18. Prefisso: se prima sono arrivata fino a 201.184.237.127, potendo rappresentare 32 indirizzi e, sapendo che ciascun blocco di classe C può contenere al massimo 256 indirizzi, 201.184.237.128 sarà il primo indirizzo di rete disponibile. Aggiungendo i 32 indirizzi disponibili si arriverà a 201.184.237.159.

Rete III – la terza sottorete deve contenere 5+2= 7 indirizzi, perché ciascuna sottorete ha i propri indirizzo di rete e broadcast. Dunque avremo 2^3= 8, dovremo riservare 3 bit all'host ID e avremo così 32-3= 29 bit per la net ID. In questo modo avremo la maschera /29. Gli indirizzi assegnabili sono 6 perché di quegli 8 bisogna scartare i due speciali. Host richiesti: 5. Prefisso: se prima sono arrivata fino a 201.184.237.159, potendo rappresentare 8 indirizzi e, sapendo che ciascun blocco di classe C può contenere al massimo 256 indirizzi, 201.184.237.160 sarà il primo indirizzo di rete disponibile. Aggiungendo i 8 indirizzi disponibili si arriverà a 201.184.237.167.

Poi c'è la quarta sottorete ma il ragionamento è lo stesso dei casi precedenti.

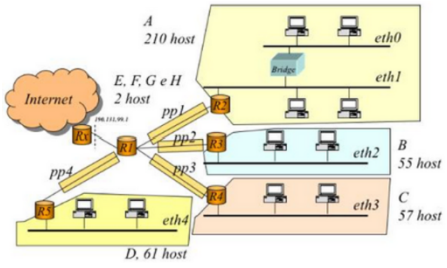
Rete	Subnet ID	/n	Host richiesti	Indirizzi assegnabili
I	201.184.237.0	25	92	126
II	201.184.237.128	27	18	30
III	201.184.237.160	29	5	6
IV	201.184.237.168	29	4	6

Esercizio 4.3



Si parte da un indirizzo IP con maschera, ovvero 195.56.78.0/23. Di conseguenza avremo 11000011.00111000.01001110.00000000 e maschera 11111111.11111111.11111110.00000000. Se è classfull allora è di classe C perché i primi bit sono 110. *L'obiettivo dell'esercizio è distribuire gli indirizzi IP a disposizione tra le sottoreti Ethernet e i link punto-punto, partendo sempre da quelle più capienti.*

Qual è la maschera più efficiente da poter assegnare ad un link punto-punto, ovvero ad un collegamento tra due host? In una rete devo avere sempre due indirizzi liberi (broadcast e di rete), sapendo che i link Point-to-Point legano due host/router, ci sarà bisogno di quattro indirizzi perché i due restanti vanno assegnati ai dispositivi collegati. Le possibili combinazioni devono essere 4 e, dunque avrò $2^2 = 4$, per cui ho bisogno di 2 bit per la parte host-ID. Il che mi porta alla maschera /30.



Ora bisogna assegnare gli indirizzi alle sottoreti: la eth0 dovrà avere 150 host, la eth1 invece 60, eth2 55, eth3 57, eth4 61. In questo disegno la eth0 e la eth1 sono collegate da un oggetto di livello 2 che si chiama *bridge*; quindi, le due sottoreti sono un'unica sottorete. In conclusione, dobbiamo considerare un'unica sottorete da 210 host.

Rete A – si parte dalla sottorete più grande, in questo caso quella composta da 210 host a cui vanno aggiunti i 2 indirizzi speciali.

1. Trovare i bit necessari per rappresentare 210 indirizzi: $2^8 = 256$, dovremo riservare 8 bit all'host ID.
2. Trovare la maschera di sottorete: se 8 sono i bit dell'host ID, allora 32-8= 24 sono i bit della net ID. In questo modo avremo la maschera /24.
3. Prefisso: essendo il primo blocco a dover essere riempito, si utilizza l'indirizzo di rete fornito, ovvero 195.56.78.0. Sapendo di dover riservare 256 indirizzi, l'ultimo indirizzo del range di questo primo blocco sarà 195.56.78.255.

Rete D – la seconda sottorete da indirizzare è la eth4 con 61+2= 63 indirizzi, perché ciascuna sottorete ha i propri indirizzo di rete e broadcast. Dunque avremo $2^6 = 64$, dovremo riservare 6 bit all'host ID e avremo così 32-6= 26 bit per la net ID. In questo modo avremo la maschera /26. Se prima sono arrivata fino a 195.56.78.255, potendo rappresentare 64 indirizzi e, sapendo che ciascun blocco di classe C può contenere al massimo 256 indirizzi, 195.56.79.0 sarà il primo indirizzo di rete disponibile. Aggiungendo i 64 indirizzi disponibili si arriverà a 195.56.79.63.

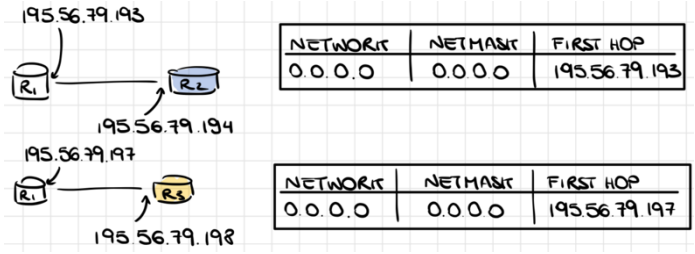
Rete C – la terza sottorete da indirizzare è la eth3 con 57 host e due indirizzi speciali: $2^6 = 64$, dunque ci troviamo ancora nella sottorete con maschera /26. Il primo indirizzo disponibile sarà 195.56.79.64 e dovendo aggiungere 64 indirizzi si arriverà all'ultimo indirizzo del blocco 195.56.79.127.

Rete B – la quarta sottorete da indirizzare è la eth2 con 55 host e due indirizzi speciale: $2^6 = 64$, dunque ci troviamo ancora nella sottorete con maschera /26. Il primo indirizzo disponibile sarà

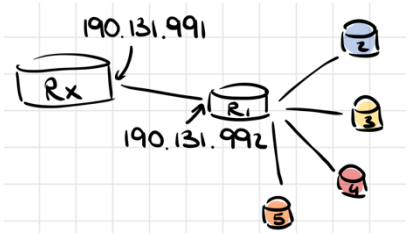
195.56.79.128 e dovendo aggiungere 64 indirizzi si arriverà all'ultimo indirizzo del blocco 195.56.79.191.

Dopodiché si possono rappresentare i link punto-punto: ad esempio tra R1 e R2 avrò, come detto prima, una maschera /30 e dunque dopo aver rappresentato gli ultimi 64 di prima arrivo al primo indirizzo disponibile 195.56.79.192/30. Tra R1 e R3 avrò sempre maschera /30 e dopo aver rappresentato i 4 del link punto-punto precedente arriverò a 195.56.79.196/30. Tra R1 e R4 partirò con l'indirizzo 195.56.79.200/30 e, tra R1 e R5 con l'indirizzo 195.56.79.204/30.

Quando abbiamo due host/router legati da un link Point-to-Point la maschera specifica /30 permetterà la rappresentazione di 4 indirizzi di cui due speciali. L'assegnazione dei due rimanenti ai router R_1 e R_n è indifferente, l'unica cosa che bisogna ricordare è che compilando la tabella di routing del router R_n il primo hop (collegamento diretto) riferito all'indirizzo di default 0.0.0.0 con maschera 0.0.0.0 è quello del router opposto, ovvero R_1. La tabella di routing di R_1 oltre all'indirizzo di default 0.0.0.0, avrà gli indirizzi di rete di tutti i blocchi, in questo caso di classe C.



Sulla base dei range trovati ad esempio se a R1 si assegna 195.56.79.193 e a R2 195.56.79.194, nella tabella di routing di R2 dovrò inserire in corrispondenza dell'indirizzo di default 0.0.0.0 con maschera 0.0.0.0 come hop successivo sempre quello "a sinistra" ovvero 195.56.79.193.

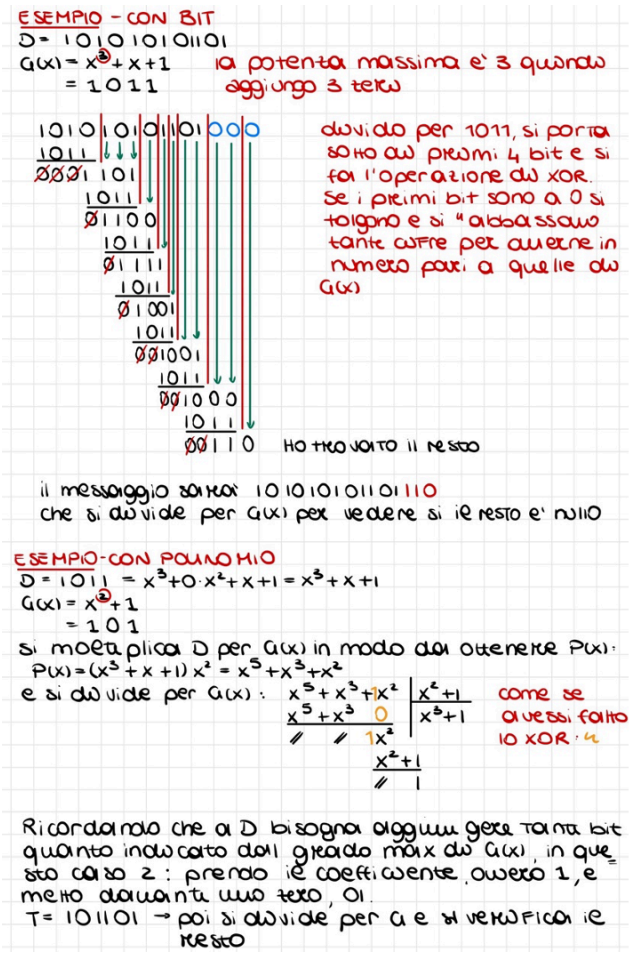


Questa è la tabella di routing di R2:

Network	Netmask	First hop
195.56.78.0	255.255.255.0	195.56.79.193
195.56.79.0	255.255.255.192	195.56.79.197
195.56.79.64	255.255.255.192	195.56.79.202
195.56.79.128	255.255.255.192	195.56.79.206
0.0.0.0	0.0.0.0	190.131.99.1

Ricorda l'indirizzo di rete ha i bit dell'ultimo byte posti a zero ed è contrassegnato da un numero pari, mentre l'indirizzo di broadcast da uno dispari. Indirizzo di rete: 195.56.79.**64**/26. Indirizzo broadcast 195.56.79.**127**/26.

Esercizio 6.1 – Controllo a ridondanza ciclica (CRC)



Nello strato di collegamento, ovvero a livello 2, esiste un algoritmo attraverso il quale poter rilevare gli errori di trasmissione dei frame in grado di compromettere l'integrità dei dati stessi: CRC, il quale si avvale di codici polinomiali. In questo modo si tratterà il flusso di dati come un numero binario da dividere per un altro numero binario, questa volta fisso, definito generatore, ed utilizzare il resto di questa divisione come valore di CRC.

1. Si prende il numero binario rappresentante i dati da trasmettere D, ricordando che ogni cifra (0 o 1) è coefficiente di una potenza del polinomio, e il generatore G. Se quest'ultimo si trova sotto forma di polinomio se ne ricavano i coefficienti, ovvero i bit, e si costruisce la stringa binaria.
2. Si guarda la potenza massima del generatore e si aggiungono infondo a D tanti zeri quanto indicato dal grado della potenza massima del numero binario del generatore.
3. Si divide il numero binario dei dati D, come stabilito in precedenza, per G per trovare il resto. Questo andrà sostituito agli zeri aggiunti in precedenza infondo a D, in questo modo si è stabilito il messaggio che il trasmettitore effettivamente trasmetterà al ricevitore.
4. Il ricevitore per verificare la presenza o meno di errori di trasmissione, ovvero i cosiddetti flip, deve dividere il numero binario ricevuto per G. Se il resto è composto da bit tutti posti a 0 allora non ci sono stati errori, viceversa sì.

In realtà l'algoritmo si può applicare anche senza effettuare l'operazione di modulo 2, ovvero usando i polinomi rispettivi di D e G. In questo caso si prende il polinomio di D, che avrà grado massimo pari ad n (il numero di bit che lo compongono) -1, costruito usando i bit come coefficienti e leggendo da destra la sequenza binaria ponendo i termini a x^0, x^1, x^2, ecc. Preso il generatore G, fornito in precedenza, si guarda la potenza massima e la si moltiplica per il polinomio D per ottenere p(x). Ora si può effettuare il rapporto tra p(x) e g(x) (polinomio di G), ottenendo un quoziente q(x) e un resto r(x). Tale divisione viene svolta con la lunga divisione dell'aritmetica modulo 2, ovvero non si fanno riporti: ricordiamo che nell'aritmetica modulo 2 la somma e la sottrazione sono eseguibili utilizzando la funzione logica XOR. Il messaggio inviato dal trasmettitore sarà m(x) = p(x) – r(x), o equivalentemente la stringa binaria D con alla fine aggiunti i bit corrispondenti di r(x). Ricorda, p(x) = q(x)*g(x) + r(x). Dunque, m(x) = q(x)*g(x) + r(x) – r(x) = q(x)*g(x). Una volta ricevuto, il ricevitore verifica la presenza o meno di errori di trasmissione effettuando la divisione tra m(x) e g(x), se il resto è nullo non ce ne sono stati.

NOTA BENE: detto T(x) il polinomio associato al frame trasmesso, il polinomio associato al frame ricevuto può essere espresso come T(x) + E(x), dove E(x) avrà coefficiente 1 per ogni bit che è stato modificato da errori trasmissivi. Un errore passerà inosservato solo se T(x) + E(x) sarà divisibile per G(x), inoltre essendo per definizione T(x) divisibile per G(x), ovvero T(x)/G(x) = 0, l'errore passerà inosservato se E(x) sarà divisibile per G(x).

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Esercizio – Si applichi alla stringa P= 1110 il meccanismo di generazione di una stringa binaria lato emettitore con CRC ottenuto attraverso un polinomio generatore $G(x) = x^3 + x + 1$.

Si derivi: **1)** la stringa binaria T emessa lato emettitore, **2)** una stringa d'errore E1 che sommata a T non dia errore in ricezione (E1 deve essere diversa da E = 0001011), **3)** una stringa d'errore E2 che sommata a T dia errore in ricezione.

ESERCIZIO 6.1

1) $P = 1110 = x^3+x^2+x$
$G(x) = x^3+x+1 = 1011$

- Si moltiplica per x^3: $(x^3+x^2+x)x^3 = x^6+x^5+x^4$
- Si divide per $G(x)$:

$$\begin{array}{r|l} x^6+x^5+x^4 & x^3+x+1 \\ \hline x^6+x^4+x^3 & x^3+x^2 \\ \hline \quad x^5+x^3 & \\ \hline \quad x^5+x^3+x^2 & \\ \hline \quad\quad x^2 & \end{array}$$

RESTO = 100 = $x^2+0\cdot x+0$

- Dovrei aggiungere 3 bit: 1110100 = T

2) Se $T(x) = Q(x)\cdot G(x)$ basta trovare un nuovo $T'(x)$ che sia divisibile per $G(x)$. Inevitabilmente se si trova un polinomio $T'(x)$ divisibile per $G(x)$ il resto sarà 0 ma, rispetto a T conterrà errore di trasmissione, essendo diverso da T.
Avendo $T' = T + E_1$ con la differenza si trova l'errore
XOR Basta scegliere un quoziente di grado $\leq G(x)$

$T'(x) = Q'(x)\cdot G(x) = (x^3+x)(x^3+x+1) = x^6+\cancel{2x^4}+x^3+x^2+x = 1001110$

$T' - T = E_1 \rightarrow$

$$\begin{array}{r} 1001110 \\ 1110001 \\ \hline 0111111 \end{array}$$

3) Ad esempio a $T'(x) = x^6+x^3+x^2+x$ togliendo x non sarebbe divisibile per $G(x)$ e avremo trovato e' errore: $T'' = x^6+x^3+x^2$.

$$\begin{array}{r} 1001100 \\ 1110001 \\ \hline 0111101 = E_2 \end{array}$$

Esercizi Risposte Multiple – Appello 15 gennaio

Considerando un algoritmo di controllo dell'errore a livello di collegamento basato su **CRC** con polinomio generatore x^3+x^2+1, se l'header di una trama di livello di collegamento è di 3 bit e il campo payload è di 5 bit, *quale delle seguenti affermazioni descrive correttamente la lunghezza totale della trama?* Il controllo di errore basato su CRC utilizza tutta la dimensione del frame, dunque il pacchetto è "grande" 8 bit, a cui bisogna aggiungere un numero di bit pari alla potenza massima del polinomio generatore, ovvero 3, così ottenendo una trama lunga 3+5+3 = 11 bit.

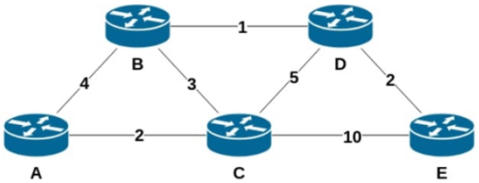
Nel contesto del protocollo **CSMA/CD**, se un dispositivo di rete, una NIC, ha sperimentato 8 collisioni consecutive, viene generato un numero che …*sarà scelto randomicamente nella finestra {0-255} e definirà il periodo di attesa che il dispositivo deve osservare prima di tentare nuovamente l'invio, seguendo una strategia di backoff esponenziale per ridurre le collisioni future*. Infatti si sceglie un tempo randomico per tentare nuovamente la trasmissione delle unità informative da una finestra che ha come estremo inferiore 0 e superiore 2^m-1, dove m sono le collisioni.

Si consideri una LAN commutata rappresentata da una topologia a stella, con 4 host ed uno switch Ethernet come centro stella. Ciascun host è collegato all'omonima porta dello switch (es. H1 alla porta 1). Considerando l'**invio di un** frame da H1 ad H4, allora: *se l'associazione tra indirizzo MAC di H4 e la relativa porta di uscita non è nota allo switch, allora il frame viene rilanciato su tutte le porte di uscita tranne quella dove si trova H1*.

Con riferimento alle funzioni svolte dal piano di controllo, indicare quale delle seguenti affermazioni è **errata**:

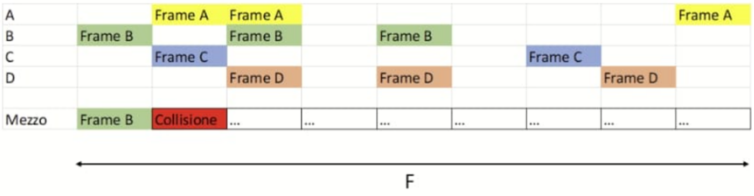
a. Lavora su scale temporali molto maggiori rispetto al piano dei dati (**ERRATA**, *la velocità di elaborazione dati dipende dall'algoritmo scelto e dai dati*)
b. Una delle funzioni principali svolte dal piano di controllo è l'indirizzamento (CORRETTA, *il control plane svolge la funzione di indirizzamento nella gestione delle reti, il controllo degli indirizzi IP è una delle sue principali responsabilità. Coinvolge l'assegnazione e la gestione degli indirizzi IP per i dispositivi sulla rete*)
c. È realizzato in software, mediante lo sviluppo di algoritmi di ottimizzazione (CORRETTA, *ricorda SDN*)
d. Internet utilizza un piano di controllo distribuito (CORRETTA, *il controllo è distribuito tra molteplici dispositivi e protocolli che lavorano insieme per instradare il traffico, gestire le connessioni e garantire che i dati vengano consegnati correttamente ai loro destinatari*)

Con riferimento alla figura mostrata sotto, indicare il contenuto del vettore delle distanze del nodo B al tempo 0, cioè quando la fase di scambio dei D(v) non è ancora iniziata. *Praticamente chiede come viene riempita la tabella dell'***algoritmo di Dijkstra** *al passo 0, considerando B come nodo origine.* I nodi adiacenti sono A a distanza 4, C a distanza 3, D a distanza 1, mentre E non è adiacente e dunque a distanza infinita: [A=4; B=0; C=3; D=1; E=inf].



Lo schema di indirizzamento di livello 2 completamente non gerarchico delle **LAN IEEE 802** (*Ricorda:* questo schema di indirizzamento si basa sugli indirizzi MAC che vengono assegnati in modo univoco ai dispositivi di rete e sono utilizzati per l'identificazione su una rete locale. Gli indirizzi MAC sono composti da 48 bit, spesso rappresentati in notazione esadecimale (ad esempio, 00:1A:2B:3C:4D:5E)) ha il principale **vantaggio** di:

a. Garantire l'univocità degli indirizzi delle interfacce delle stazioni ed eliminare ogni necessità di configurazione di indirizzi (**CORRETTA**, *gli indirizzi MAC, per la loro univocità, consentono di identificare in modo univoco le interfacce di rete delle stazioni all'interno di una LAN. Questo elimina la necessità di configurare manualmente gli indirizzi MAC e semplifica la gestione della rete. Inoltre, essendo univoci, gli indirizzi MAC evitano collisioni di indirizzi, garantendo che ogni dispositivo abbia il suo identificatore unico nella rete*)
b. Rendere univoca l'identificazione dell'applicazione della stazione sulla LAN
c. Rendere particolarmente efficiente l'instradamento delle trame MAC nelle tabelle degli switch



Si considerino i 4 nodi A, B, C e D che trasmettono in modalità accesso multiplo **Slotted ALOHA**. Data l'evoluzione temporale delle trasmissioni riportata in figura, completando quello che accade sul mezzo condiviso, *qual è la portata utile normalizzata nella finestra temporale F?* La portata utile normalizzata si trova dividendo il numero di pacchetti trasmessi correttamente per gli slot di tempo in cui è suddiviso il canale: 4/9 = 0,44. Perché ci sono B nella prima finestra, C nella settima, D nell'ottava e A nell'ultima.

Quale delle seguenti NON è una caratteristica di un **algoritmo di routing di tipo link state**?

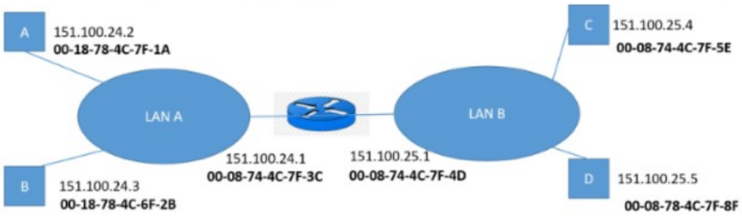
a. Utilizza un approccio di tipo globale
b. Un nodo scambia con i suoi vicini informazioni relative al costo di raggiungibilità delle varie destinazioni
c. *La scelta dei percorsi è basata sulla minimizzazione di una funzione di costo lineare* (La C **NON** è una caratteristica di un algoritmo di questo tipo perché il costo computazionale, ad esempio, per l'algoritmo di Dijkstra è $O(n^2)$)
d. Offre prestazioni migliori (scalabilità e tempi di convergenza) rispetto ad un approccio di tipo distance vector

Con riferimento al piano di controllo di una rete IP, indicare quale delle seguenti affermazioni è **corretta**:

a. Lavora su scale temporali molto maggiori rispetto al piano dei dati
b. Internet utilizza un architettura basata sul piano di controllo centralizzato
c. *Una delle funzioni più importanti svolte dal piano di controllo è l'indirizzamento* (**CORRETTA**)
d. Un router memorizza i percorsi (sequenza di nodi attraversati) trovati all'interno di specifiche tabelle

Qual è la principale differenza tra **accodamento** e **perdita** nel contesto del packet switching?

a. L'accodamento si riferisce alla temporanea memorizzazione del pacchetto in attesa di essere trasmesso, mentre la perdita si verifica quando i pacchetti vengono eliminati a causa di una congestione eccessiva nella rete (**CORRETTA**)
b. L'accodamento si verifica quando il rate di arrivo del pacchetto eccede la capacità del collegamento in uscita per un periodo limitato di tempo, mentre la perdita si verifica quando un pacchetto subisce danni irreparabili e non può essere consegnato correttamente
c. L'accodamento è il processo di instradamento dei pacchetti da un nodo a un altro dietro l'altro, mentre la perdita è la probabilità che i pacchetti vadano persi durante la trasmissione a causa di interferenze



Si consideri la seguente architettura di rete con indirizzi IP e MAC come riportati in figura. Quale delle seguenti risposte (ARP Reply) segue un ARP Request del tipo: *"Who has address 151.100.24.3? Tell 151.100.24.2"* **151.100.24.3 is at 00-18-78-4C-6F-2B**

Ricorda: se l'host fosse appartenuto ad una LAN diversa da quella di A, ovvero la connessione non fosse stata diretta, non ci sarebbe stata una REPLY.

Specificare l'**indirizzo broadcast della** rete 74.18.128.0/19. Innanzitutto si converte l'indirizzo in binario e si ottiene **01001010.00010010.10000000.00000000**, dove blu sono i bit della net ID e arancioni quelli riservati per l'host ID. Essendo l'indirizzo broadcast l'ultimo della sottorete, ovvero quello che imposta i bit dell'host ID ad 1: **01001010.00010010.10011111.11111111** che equivale a scrivere 74.18.159.255.

Si assuma che un router, la cui **routing table** è riportata sotto, riceva un pacchetto con indirizzo IP di destinazione pari a 10.71.164.37. Indicare attraverso quale interfaccia verrà rilanciato il pacchetto. (10.71.164.37 = 00001010.01000111.10100100.00100101)

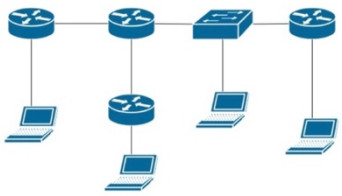
Destination Prefix	Out Interface
10.71.0.0/16	1
10.71.128.0/17	2
10.71.128.0/18	3
10.71.128.0/19	4

Si convertono in bit l'indirizzo dato e gli indirizzi di ciascuna interfaccia, poi si mettono in evidenza i bit riservati per la parte net ID e host ID, così da mostrare bene la maschera per ciascuna interfaccia. Così si ottiene:

a. 10.71.0.0/16 = **00001010.01000111**.00000000.00000000, di cui l'ultimo sarà 10.71.255.255
b. 10.71.128.0/17 = **00001010.01000111.1**0000000.00000000, di cui l'ultimo sarà 10.71.255.255
c. 10.71.128.0/18 = **00001010.01000111.10**000000.00000000, di cui l'ultimo sarà 10.71.191.255
d. 10.71.128.0/19 = **00001010.01000111.100**00000.00000000, di cui l'ultimo sarà 10.71.159.255

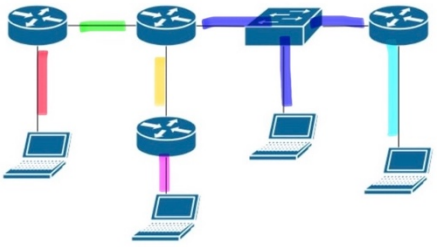
Da cui si vede già che la quarta interfaccia non può includere l'indirizzo di interesse perché non ho abbastanza indirizzi a disposizione, mi fermerei a 10.71.159.255. A questo punto bisogna scegliere l'interfaccia il cui indirizzo di rete ha più bit in comune con l'indirizzo dato. In questo caso, avendo 10.71.164.37 = 00001010.01000111.10100100.00100101, si trova match sia con /16 che con /17 e /18 ma, l'indirizzo di rete con più bit in comune è quello con maschera /18. Dunque il pacchetto verrà rilasciato sull'interfaccia 3.

Dato lo schema di figura, indicare il numero di reti LAN presenti



- a. 6
- b. 7
- c. 5
- d. 8

Per ogni uscita di un router, i dispositivi tondi, viene creata una rete diversa. Ad esempio, il router all'estrema sinistra ha due reti perché ci sono due uscite diverse. Lo switch, i dispositivi quadrati, si può ignorare perché serve solo da collegamento ma non spezza la rete. In tutto le sottoreti sono 6.



La *trasformata di Fourier* del segnale x(t) = sinc(3t) è 1/3rect(f/3). Ricorda che sinc si chiama seno cardinale.

La *trasformata di Fourier tempo discreto* (Discrete Time Fourier Transform, DTFT) *di* x_n è continua e periodica.

Dato il segnale x(t) = sinc2(t/2) *qual è il numero minimo di campioni che devono essere presi in 10 secondi per soddisfare il teorema del campionamento?* 20

Indicare quale tra i seguenti è un protocollo di routing?

a. ICMP
b. **BGP** (**CORRETTA**)
c. ARP
d. DHCP

In una rete **CSMA/CD** la massima distanza tra due stazioni è di 2000 metri. *Qual è il minimo tempo di trasmissione di una trama se la velocità di propagazione sul mezzo è di* $4*10^4$ *m/s?* Ricordando che la relazione da verificare è che L/R >= RTT e che, in questo caso, RTT è pari a 2d/v, si ottiene: T>= (2*2000) / ($4*10^4$), ovvero T>= 0.1s

Nell'indirizzamento in Internet la maschera di sottorete:

a. Distingue tra indirizzi in forma binaria e in rappresentazione "dotted"
b. Serve a rendere noto ad ogni sistema in che modo il suo indirizzo IP è suddiviso tra net ID e host ID (**CORRETTA**)
c. Specifica la classe di indirizzamento che è stata adottata

Con riferimento alla funzione di verifica di errore, si consideri il seguente codice polinomiale (**CRC**) x^5+x^4+x+1. Questo polinomio *è in grado di rilevare stringhe di errore a raffica di lunghezza 5*.

Si supponga di avere un file di dimensione 40 kbyte e di utilizzare una compressione con **rapporto di compressione** R=8. Se in una line che ha velocità di X Mbit/s il file viene trasferito in meno di 0,8 ms, *qual è la velocità X della linea?* Sapendo che il rapporto di compressione si calcola come il rapporto tra la lunghezza dei dati originali e la lunghezza dei dati compressi in bit, si avrà:

L = 40 kbyte
L compressi = Lc
R = 8
C = X Mbit/s
T = Lc/C = 0,8 ms

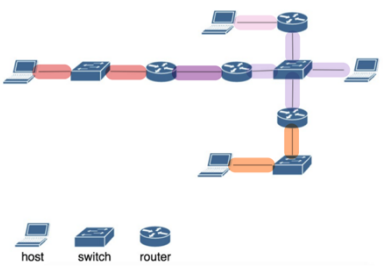
R = L/Lc si può anche scrivere R = L/Lc tanto C si semplifica reciproco

$\frac{L}{C} = R \cdot \frac{Lc}{C} \rightarrow \frac{L}{C} = R \cdot T \rightarrow \not{C} \cdot \frac{L}{\not{C}} = R \cdot T \cdot C$

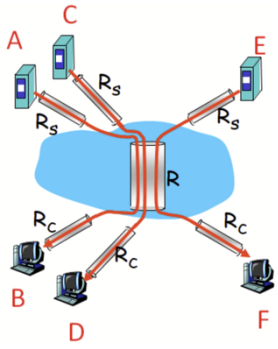
$\rightarrow C = \frac{L}{RT} = \frac{40 \cdot 10^3 \cdot 8}{8 \cdot 0{,}8 \cdot 10^{-3}} = 50 \cdot 10^6$ bit/s = 50 Mbit/s

Con riferimento alla topologia fisica mostrata in figura, determinare il numero di sotto-reti

SONO 5



Esercizi Risposta Multipla – Test Intermedi Matricole Pari



Si consideri la seguente configurazione di rete contre comunicazioni che si dividono una banda R. Assumendo che la comunicazione tra E ed F ottenga sul canale condiviso una banda pari a R/2 e le altre due si dividono equamente la banda rimanente (R/2 + (R/2)/2 + (R/2)/2). Siano R_s=1Mbit/s, R_c=2Mbit/s e R=20Mbit/s. *Quale delle tre comunicazioni ha il throughput maggiore?* Nessuna delle tre comunicazioni ha throughput maggiore perché bisogna ricordare che questo è rappresentato, per ciascuna comunicazione tra due host, come il min(R_c, R_s). Se tutte le comunicazioni hanno gli stessi valori di R_c e R_s avremo tre throughput identici.

In una procedura di recupero d'errore Go-Back-N si adotta una finestra in emissione che ha larghezza 3. Il trasmettitore A ha sempre pacchetti da trasmettere. Ricordando la convenzione per cui il ricevitore con l'ACK chiede il prossimo numero di sequenza che si aspetta di ricevere e, supponendo che la finestra di emissione abbia in un dato istante limite inferiore uguale a 4, *come reagisce A ricevendo da B un ACK con numero 6?* Aggiorna il limite inferiore della finestra a 6 e trasmette i pacchetti 7 e 8, non trasmette il numero 5 perché l'ACK è cumulativo e avverte il trasmettitore che i pacchetti fino al 6 sono stati ricevuti con successo.

Quanti pacchetti di dimensione 50 byte possiamo trovare contemporaneamente in "volo" su un mezzo trasmissivo lungo 20 Km che trasmette a bit rate C=60 Mbit/s e la cui velocità di propagazione è pari a quella della luce nel vuoto ($3*10^8$ m/s)?

L = 50 byte = 50·8 bit
mezzo trasmissivo = 20 km
C = 60 Mbit/s
v = $3 \cdot 10^8$ m/s

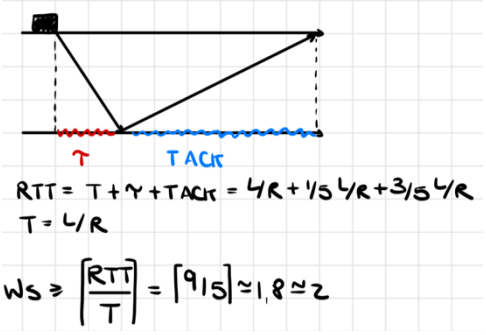
$T = L/C = \frac{50 \cdot 8}{60 \cdot 10^6} = 6{,}6$ μs

Quanto km occupa un pacchetto per essere trasmesso?
$S = v \cdot T = 3 \cdot 10^8 \cdot 6{,}6 \cdot 10^{-6} = 2 \cdot 10^3$ m = 2 km

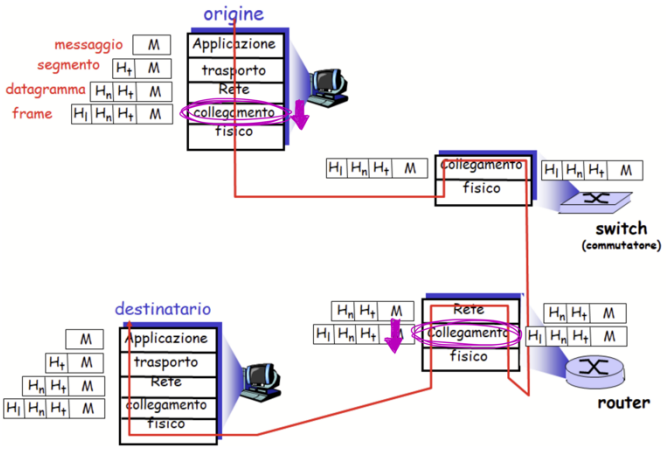
Quanti pacchetti trasmetto in 20 km?
$\frac{20}{2} = 10 = N_p$

Un'applicazione residente nell'host A scambia dati con due altre applicazioni residenti in host distinti B e C, in quale modo l'host A distingue le due connessioni TCP utilizzate? TCP utilizza diverse socket per le varie connessioni, ciascuna di esse è identificata da quattro parametri: indirizzo IP di origine, numero di porta di origine, indirizzo IP di destinazione e numero di porta di destinazione. Dunque, gli indirizzi IP degli host B e C sono segni distintivi.

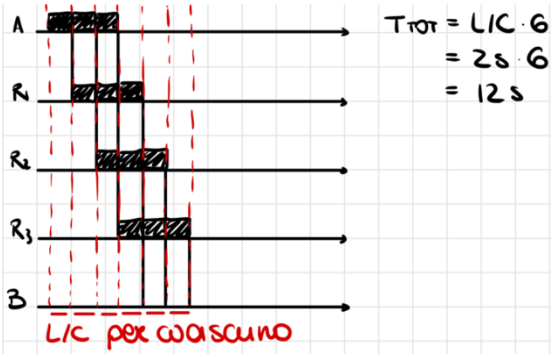
Si consideri un protocollo di controllo d'errore di tipo Go-BACK-N. Sia: 1) ritardo di propagazione pari a 1/5 del tempo di trasmissione di un pacchetto; 2) il tempo di trasmissione dei riscontri pari a 3/5 del tempo di trasmissione di un pacchetto e l'elaborazione nei nodi trascurabile. *Quanto deve essere la dimensione minima della finestra di emissione N affinchè, se non si verificano errori, il trasmettitore sia in grado di tramettere pacchetti continuamente (senza soluzione di continuità)?* Sappiamo che la finestra di emissione W_s, costituita da N pacchetti, deve essere maggiore o al massimo uguale alla parte intera del rapporto tra il RTT e il tempo di trasmissione di un pacchetto. Questo perché per inviare pacchetti continuamente devo almeno riuscire a coprire tutto il RTT.



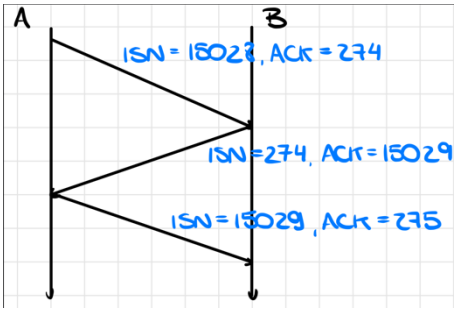
Per quale motivo nei protocolli di trasferimento è necessario aggiungere i numeri di sequenza? Per permettere a chi riceve sia di accorgersi di eventuali pacchetti duplicati che di pacchetti persi.



Si consideri il seguente schema, *quante volte un datagramma viene incapsulato in un frame nel percorso di un messaggio M dall'origine alla destinazione?* Accade due volte perché sia nell'origine che nel router si passa dal livello di rete a quello di collegamento (si scende).



Si considerino due terminali interconnessi attraverso una catena di tre nodi intermedi che trasmettono, nella modalità a pacchetto, un file suddiviso in 3 pacchetti. Considerando trascurabili i tempi di propagazione e di accodamento, se la trasmissione di un pacchetto su un'interfaccia richiede 2 secondi, *quanto tempo si impiega per trasferire tutto il file?* Ricorda che nell'ultima interfaccia i pacchetti non vanno disegnati, in questo modo si mettono in evidenza il fatto che per il primo 4 tempi di trasmissione, essendoci l'effetto pipeline, si considera solo il primo pacchetto, e poi il secondo ed il terzo. L'obiettivo è trovare un modo per coprire la linea temporale occupata nella trasmissione del pacchetto, ecco come si conta.



Un'entità TCP A apre la procedura di instaurazione di una connessione con un Initial Sequenze Number (ISN)=15028; la controparte B replica con un ISN=274. *Quali sono i contenuti dei due campi Sequence Number (SN) e Acknowledgment Number (AN) nel segmento emesso dall'entità A per concludere la procedura (terzo segmento nella procedura di instaurazione della connessione)?* Ricorda che l'ISN è il numero randomico della sequenza che un host invia per aprire la connessione, mentre il SN è il numero della sequenza che l'host sta inviando.

Un router, la cui routing table è riportata sotto, riceve un pacchetto il cui indirizzo di destinazione è 10.0.1.171. *Indicare su quale interfaccia di uscita viene rilanciato il pacchetto.*

Prefisso di sottorete	**Interfaccia di uscita**
10.0.0.0/8	1
10.0.1.0/24	2
10.0.1.0/25	3
10.0.1.128/27	4

Sicuramente non sarà la prima perché il prefisso di rete è completamente diverso, né tanto meno l'ultima perché sono solo 5 i bit riservati alla parte host-ID, il che implica la possibilità di rappresentare solo 2^5=32 host e raggiungere l'indirizzo IP 10.0.1.160. *L'interfaccia corretta è la 2* perché avendo 32-24 = 8 bit per la parte host-ID si possono rappresentare 256 host. Sarà pertanto possibile arrivare all'indirizzo che stiamo considerando.

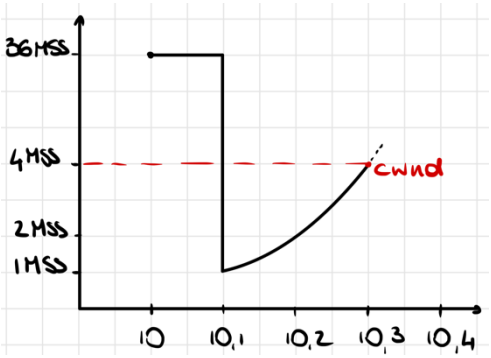
In una rete caratterizzata dal prefisso 151.168.0.0/23, *quanti host possono essere indirizzati al massimo (escludendo indirizzo broadcast e di rete)?* Avendo a disposizione 32-23 = 9 bit per la parte host-ID, è possibile rappresentare 2^9 -2 = 510 host.

In uno schema di indirizzamento Classless sia dato il seguente indirizzo di rete IP 151.100.64.0/18, *qual è l'indirizzo broadcast di questa sottorete?* Ricorda che l'indirizzo broadcast è l'ultimo indirizzo di una sottorete, ovvero quello che imposta ad 1 tutti i bit riservati alla parte host-ID. Per trovarlo basta scrivere l'indirizzo IP in codice binario e impostare gli ultimi 32-18 = 14 bit ad 1, poi tornare nel formato dotted. 10010111.11001000.01**000000.00000000** → 10010111.11001000.01**111111.11111111** = 151.100.127.255

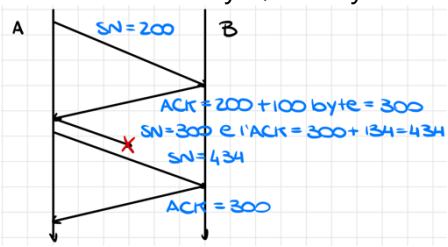
Indicare quanti blocchi di classe C sono aggregati dal prefisso 192.168.64.0/21. Se il prefisso in questione è di classe C significa che avrà maschera /24, di conseguenza tutte le possibili sottoreti, ovvero i blocchi di classe C, che si ottengono passando da una maschera /21 ad una maschera /24 saranno $2^{(24-21)} = 2^3 = 8$.

Due sistemi terminali stanno scambiando messaggi utilizzando una connessione TCP versione Tahoe. Al tempo T=10 secondi, istante in cui si verifica un evento di RTO (Retransmission Time Out), la congestion window ha una dimensione pari a 36 MSS. Assumendo che l'RTT è costante e pari a 100 millisecondi, *indicare la dimensione della congestion window all'istante T1=10,3 secondi.*

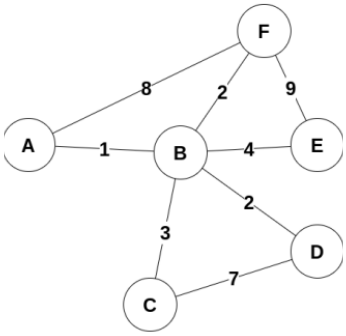
Ricorda che la versione Tahoe di TCP è quella costituita dalla fase slow start. Pertanto, immaginando di trovarci all'istante di tempo 10 secondi raggiungiamo il picco pari ad una cwnd di 36 MSS, l'evento di perdita di pacchetti porta a ricominciare da capo la fase additiva. Prima che ciò accada, sapendo che ogni evento si verifica ogni RTT = 100 ms = 0,1 s, e che il time out è $\geq RTT$, allora ci saranno 100 ms di time out per poi scendere ad 1MSS e ricominciare la fase slow start prima con l'incremento esponenziale e poi quello lineare. Essendo nel picco la cwnd pari a 36 MSS, la nuova ssthresh sarà pari a 36/2 MSS. Dunque, la cwnd al tempo T1 = 10,3 secondi sarà pari a 4MSS.



L'host A invia tre segmenti TCP all'host B, aventi lunghezza rispettivamente 100 Byte, 134 Byte e 78 Byte. Supponendo che il secondo segmento si perda, *indicare il valore assunto dall'ultimo dei messaggi di ACK mostrato nella figura.* Ricorda che in casi come questo bisogna somma il numero di sequenza alla dimensione in byte del segmento stesso. Dunque, l'ACK sarebbe 300 perché bisogna reinviare quello relativo all'ultimo segmento che non è stato ricevuto.



Si consideri il grafo in figura. Con riferimento all'esecuzione dell'algoritmo di Dijkstra con radice nel nodo A, *si indichi quale nodo viene visitato all'iterazione 4.*



#Passo	Lista	B	C	D	E	F
0	A	1, A	inf	inf	inf	8, A
1	AB	/	4, B	3, B	5, B	3, B
2	ABD	/	10, D	/	inf	inf
3	ABDC	/	/	/	inf	inf
4	ABDCF	/	/	/	12, F	/
5	ABDCFE					

Riassumendo il procedimento di compilazione della tabella:

Al **passo 0** viene visitato il nodo origine A, si cercano i percorsi più brevi per raggiungere i nodi adiacenti, ovvero B ed F, mentre i nodi che non sono direttamente collegati ad A si pongono ad infinito. Qual è il percorso più breve per raggiungere il nodo B passando per A? La distanza è lunga 1. Qual è il percorso più breve per raggiungere il nodo F passanddmo per A? La distanza è lunga 8. Dopodiché si sceglie il minimo percorso della riga, in questa è proprio 1, allora si inserisce B nella lista dei nodi visitati.

Al **passo 1** viene visitato il nodo B appena aggiunto alla lista, lasciando vuote le caselle relative ai nodi già visitati, i nodi adiacenti a B sono C, D, E ed F. Qual è il percorso più breve per raggiungere il nodo C passando per B? La distanza è lunga 3, allora partendo dal nodo A sarebbe 1+3 = 4. Qual è il percorso più breve per raggiungere il nodo D passando per B? La distanza è lunga 2, allora partendo dal nodo A sarebbe 1+2 = 3. Questo meccanismo si ripete per E ed F. Nello scegliere il minimo percorso della riga c'è un "doppione", ovvero sia per il nodo D che per il nodo F il percorso è pari a 3 e il nodo precedente è B, si sceglie quello visitato prima: D. Questo si inserisce nella lista dei nodi visitati.

Al **passo 2** viene visitato il nodo D appena aggiunto alla lista, come prima si lasciano vuote le caselle dei nodi già visitati e notando che i nodi adiacenti sono B e C (E ed F non sono adiacenti, si pongono ad infinito). Qual è il percorso più breve per raggiungere il nodo C passando per D e partendo da A? La distanza è pari a 10, infatti bisogna passare per B, il che porta ad avere 1+2+7 e come nodo precedente D. Si sceglie il minimo percorso, ovvero 10 passando per D e, si aggiunge C alla lista.

Al **passo 3** viene visitato il nodo C appena aggiunto, non considerando i nodi già visitati, i nodi rimanenti E ed F non sono adiacenti a C così si pongono ad infinito le distanze. Si è arrivati al punto di stallo perché non è possibile scegliere un percorso minore. A questo punto si ripercorre la tabella in corrispondenza dei nodi ancora da visitare, quindi E ed F, per scegliere il valore minore escludendo ovviamente le distanze infinite. In questo caso si devono percorrere le colonne di E ed F, i valori utili si trovano al passo 1: (5, B) per il nodo E e (3, B) per il nodo F (è un caso che occupino la stessa riga, avrebbero potuto trovarsi anche in righe diverse, in quel caso il ragionamento sarebbe stato lo stesso: trovare i primi valori utili e valutare quale dei due sia minore). Si sceglie il nodo F, ovvero quello a distanza minore.

Al **passo 4** si visita il nodo F perché essendo quello a distanza minore, è l'ultimo ad essere inserito in lista. Qual è il percorso minore partendo da A e arrivando ad E, passando per F? Quello che segue l'andamento A -> B ->F -> E, per cui si avrà 1+2+9= 12.

Al **passo 5** si inserisce il nodo E in lista, l'ultimo rimasto.

Come trovare il percorso più breve? Si scorre per ogni nodo la sua colonna e si seleziona la distanza minore percorsa. Nella tabella sono evidenziati, quindi riassumendo:

A->B collegamento diretto che vale 1
A->C, si nota come osservando la colonna di C il valore minore sia (4, B), per raggiungere C bisogna passare per B, quindi A->B->C
A->D, il valore minore della colonna è (3, B), allora A->B->D
A->E, il valore minore è 5, B , allora A->B->E
A->F, il valore minore è 3,B , allora A->B->F

Esame 15 gennaio – Traccia 1

Abbiamo un ISP con il relativo indirizzo IP 193.32.0.0 = 11000001.00100000.00000000.00000000, da cui è evidente sia un indirizzo di classe C con relativa maschera "di default" /24. Passando da una maschera /12 ad una /24 otteniamo la suddivisione della rete in $2^{(24-12)}=2^{12}$=4096 blocchi di classe C, ovvero sottoreti appartenenti a quella classe. Anziché avere un'unica sottorete contenente 4096 blocchi, ne voglio ottenere quattro: A con 2048 blocchi, B con 1024 blocchi, C con 512 blocchi e D con 128 blocchi. La regola impone di affidare gli indirizzi IP partendo dalle sottoreti contenenti il maggior numero di blocchi.

Quali sono i bit variabili per la rappresentazione dei 4096 blocchi? Essendo la classe da appartenenza dell'indirizzo IP dell'ISP la C, vuol dire che la parte net ID sarà composta di 24 bit, mentre la parte host ID da 8 bit. Nella parte net ID 12 rappresentano la rete ID mentre 24-12 = 12 i 4096 blocchi, sono quelli dunque che variano: 193.32.0.0 = 11000001.0010**0000.00000000**.00000000.

Blocco A – Per rappresentare 2048 blocchi, di quei 12 dedicati alla subnet ID ne utilizzo solo 11 poiché 2^{11}=2048. Essendo il blocco A il primo ad essere riempito, avrà lo stesso indirizzo dell'ISP, ovvero 193.32.0.0. La maschera del blocco A è determinata dai bit della parte di rete che non sono variabili, in questo caso 24-11 = 13, ecco perché indirizzo e maschera della sottorete A saranno 193.32.0.0/13. Il numero di host rappresentabili nel blocco A si trova moltiplicando il numero di host che ciascuna sottorete di classe C può ospitare, ovvero 2^8= 256, per il numero di sottoreti del blocco, ovvero 2048, a cui tolgo gli indirizzi speciali che sono due: 256*2048 – 2= 524286. Per sapere quale sia il primo indirizzo utile nella sottorete da poter assegnare all'interfaccia del router due basta considerare che bisogna scartare il primo perché rappresentante dell'indirizzo di rete, di conseguenza, il successivo di 193.32.0.0 è 193.32.0.1.

Per trovare l'indirizzo IP dell'ultima delle 2048 sottoreti del blocco A bisogna considerare di porre ad 1 tutti i bit variabili che sono 11: 11000001.00100**111.11111111**.00000000, ovvero 193.39.255.0. Per conoscere proprio l'ultimo indirizzo IP dobbiamo trovare l'ultimo host rappresentabile, ovvero quando si pongono gli ultimi 8 bit (host ID) dell'ultima delle 2048 sottoreti ad 1: 11000001.00100**111.11111111**.11111111, ovvero 193.39.255.255.

Blocco B – Per rappresentare 1024 blocchi, di quei 12 dedicati alla subnet ID ne utilizzo solo 10 poiché 2^{10}=1024. Avendo concluso tutte le rappresentazioni della sottorete 193.39.255.0, ovvero essendo arrivati a 193.39.255.255, la successiva sarà 193.40.0.0. La maschera di sottorete è determinata dai bit della parte di rete che non sono variabili, in questo caso 24-10 = 14, ecco perché indirizzo e maschera della sottorete B saranno 193.40.0.0/14. Il numero di host rappresentabili nel blocco B si trova moltiplicando il numero di host che ciascuna sottorete di classe C può ospitare, ovvero 2^8= 256, per il numero di sottoreti del blocco, ovvero 1024, a cui tolgo gli indirizzi speciali che sono due: 256*1024 – 2= 262142. Per sapere quale sia il primo indirizzo utile nella sottorete da poter assegnare all'interfaccia del router due basta considerare che bisogna scartare il primo perché rappresentante dell'indirizzo di rete, di conseguenza, il successivo di 193.40.0.0. è 193.40.0.1.

Per trovare l'indirizzo IP dell'ultima delle 1024 sottoreti del blocco B bisogna considerare di porre ad 1 tutti i bit variabili che sono 10: 11000001.001010**11.11111111**.00000000, ovvero 193.43.255.0. Per conoscere proprio l'ultimo indirizzo IP dobbiamo trovare l'ultimo host rappresentabile, ovvero

quando si pongono gli ultimi 8 bit (host ID) dell'ultima delle 1024 sottoreti ad 1: 11000001. 00101011.11111111.11111111, ovvero 193.43.255.255.

Blocco C – Per rappresentare 512 blocchi, di quei 12 dedicati alla subnet ID ne utilizzo solo 9 poiché 2^9=512. Avendo concluso tutte le rappresentazioni della sottorete 193.43.255.0, ovvero essendo arrivati a 193.43.255.255, la successiva sarà 193.44.0.0. La maschera di sottorete è determinata dai bit della parte di rete che non sono variabili, in questo caso 24-9 = 15, ecco perché indirizzo e maschera della sottorete C saranno 193.44.0.0/15. Il numero di host rappresentabili nel blocco C si trova moltiplicando il numero di host che ciascuna sottorete di classe C può ospitare, ovvero 2^8= 256, per il numero di sottoreti del blocco, ovvero 1024, a cui tolgo gli indirizzi speciali che sono due: 256*512 – 2= 131070. Per sapere quale sia il primo indirizzo utile nella sottorete da poter assegnare all'interfaccia del router due basta considerare che bisogna scartare il primo perché rappresentante dell'indirizzo di rete, di conseguenza, il successivo di 193.44.0.0. è 193.44.0.1.

Per trovare l'indirizzo IP dell'ultima delle 512 sottoreti del blocco C bisogna considerare di porre ad 1 tutti i bit variabili che sono 9: 11000001.00101101.11111111.00000000, ovvero 193.45.255.0. Per conoscere proprio l'ultimo indirizzo IP dobbiamo trovare l'ultimo host rappresentabile, ovvero quando si pongono gli ultimi 8 bit (host ID) dell'ultima delle 512 sottoreti ad 1: 11000001. 00101101.11111111.11111111, ovvero 193.45.255.255.

Blocco D – Per rappresentare 128 blocchi, di quei 12 dedicati alla subnet ID ne utilizzo solo 7 poiché 2^7=128. Avendo concluso tutte le rappresentazioni della sottorete 193.45.255.0, ovvero essendo arrivati a 193.45.255.255, la successiva sarà 193.46.0.0. La maschera di sottorete è determinata dai bit della parte di rete che non sono variabili, in questo caso 24-7 = 17, ecco perché indirizzo e maschera della sottorete D saranno 193.46.0.0/17. Il numero di host rappresentabili nel blocco D si trova moltiplicando il numero di host che ciascuna sottorete di classe C può ospitare, ovvero 2^8= 256, per il numero di sottoreti del blocco, ovvero 1024, a cui tolgo gli indirizzi speciali che sono due: 256*128 – 2= 32766. Per sapere quale sia il primo indirizzo utile nella sottorete da poter assegnare all'interfaccia del router due basta considerare che bisogna scartare il primo perché rappresentante dell'indirizzo di rete, di conseguenza, il successivo di 193.46.0.0. è 193.46.0.1.

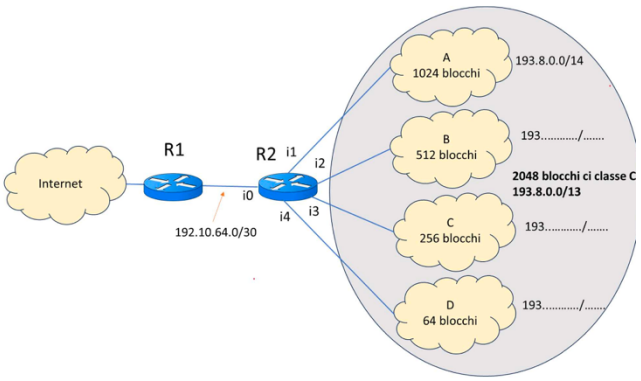
Per trovare l'indirizzo IP dell'ultima delle 128 sottoreti del blocco D bisogna considerare di porre ad 1 tutti i bit variabili che sono 10: 11000001.00101110.01111111.00000000, ovvero 193.46.127.0. Per conoscere proprio l'ultimo indirizzo IP dobbiamo trovare l'ultimo host rappresentabile, ovvero quando si pongono gli ultimi 8 bit (host ID) dell'ultima delle 2048 sottoreti ad 1: 11000001. 00101110. 01111111.11111111, ovvero 193.46.127.255.

Per trovare il numero di blocchi non ancora assegnati dei 4096 totali basta sommare quelli rappresentati dalle quattro sottoreti A,B,C e D: 4096 – (2048+1024+512+128) = 384.

Tabella di routing di R2 – Quando abbiamo un link Point-to-Point stiamo collegando due host, abbiamo perciò bisogno di indirizzo di rete, indirizzo broadcast e altri due indirizzi per i due host: 2^2 = 4, allora la maschera sarà /30. Dunque il salto, passaggio successivo partendo dall'indirizzo di default che è 0.0.0.0 con maschera 0.0.0.0, è l'altro indirizzo IP disponibile, ovvero 192.10.64.2, sull'interfaccia I0. Vanno inseriti tutti gli indirizzi IP delle sottoreti relative ai gruppi rimanenti, ovvero B, C e D e le maschere /14 /15 /17 scritte nei formati dotted. Le interfacce si ricavano dal

disegno e i salti successivi sono “local” perché ci si trova nella stessa sottorete. Infine la sottorete 192.10.64.0 la cui sottorete è /30, con l’interfaccia I0.

Esame 15 gennaio – Traccia 2



Indirizzo IP dell'ISP: 193.8.0.0/13 = **11000001.00001**000.00000000.00000000. Essendo un indirizzo di classe C ha maschera di default /24, la differenza è di 24-13 = 11 bit con i quali si ottengono 2^{11}= 2048 blocchi di classe C, i quali si organizzano in quattro sottoreti: A 1024, B 512, C 256, D 64. Ecco gli 11 bit variabili: **11000001.00001000.00000000**.00000000.

Blocco A – per ottenere 1024 blocchi servono 10 di quei bit variabili, infatti 2^{10} = 1024. Ed essendo il primo blocco a dover essere riempito perché di dimensione maggiore avrà stesso indirizzo IP dell'ISP: 193.8.0.0. La maschera è determinata dai bit della parte di rete non variabili, in questo caso 24-10 = 14. Il numero di host rappresentabili si trova moltiplicando il numero di host che un blocco di classe C può rappresentare, ovvero 2^8= 256, per il numero di sottoreti del blocco, ovvero 1024, e sottraendo al numero ottenuto i due indirizzi speciali (indirizzo di rete e broadcast): 256*1024 – 2= 262142. Per sapere quale sia il primo indirizzo utile nella sottorete da poter assegnare all'interfaccia del router R2 basta scartare il primo, ovvero l'indirizzo di rete, in questo caso scartando 193.8.0.0, avrò 193.8.0.1. L'indirizzo IP dell'ultima delle 1024 sottoreti del blocco A si trova ponendo ad 1 i dieci bit variabili: 11000001.00001011.11111111.00000000, ovvero 193.11.255.0. L'ultimo di quest'ultima sottorete si trova ponendo ad 1 i bit dell'host ID: 11000001.00001011.11111111.11111111, ovvero 193.11.255.255.

Blocco B – per ottenere 512 blocchi servono 9 di quei bit variabili, infatti 2^9 = 512. Le rappresentazioni del blocco A si concludono con l'indirizzo 193.11.255.255, il successivo, ovvero il primo disponibile è 193.12.0.0. La maschera è determinata dai bit della parte di rete non variabili, in questo caso 24-9 = 15. Il numero di host rappresentabili si trova moltiplicando il numero di host che un blocco di classe C può rappresentare, ovvero 2^8= 256, per il numero di sottoreti del blocco, ovvero 512, e sottraendo al numero ottenuto i due indirizzi speciali (indirizzo di rete e broadcast): 256*512 – 2= 131070. Per sapere quale sia il primo indirizzo utile nella sottorete da poter assegnare all'interfaccia del router R2 basta scartare il primo, ovvero l'indirizzo di rete, in questo caso scartando 193.12.0.0, avrò 193.12.0.1. L'indirizzo IP dell'ultima delle 512 sottoreti del blocco B si trova ponendo ad 1 i nove bit variabili dell'indirizzo 193.12.0.0= 11000001.00001100.00000000.00000000, ottenendo 11000001.00001101.11111111.00000000 = 193.13.0. L'ultimo di quest'ultima sottorete si trova ponendo ad 1 i bit dell'host ID: 11000001.00001101.11111111.11111111, ovvero 193.13.255.255.

Blocco C – 2^8 = 256, i bit di interesse sono 8 tra quelli variabili. Il primo disponibile per identificare la rete è 193.14.0.0 con maschera di 24-8 = 16. Il numero di host è pari a 256*256 – 2= 65534. Il primo indirizzo utile da poter assegnare all'interfaccia del router R2, scartando l'indirizzo di rete, è 193.14.0.1. Avendo che 193.14.0.0 = 11000001.00001110.00000000.00000000 con gli 8 bit evidenziati variabili, l'ultimo indirizzo dei 256 blocchi si trova ponendo ad 1 i bit rossi per l'indirizzo dell'ultimo blocco e ad 1 i bit dell'host ID: 11000001.00001110.11111111.11111111 = 193.14.255.255.

Blocco D – 2^6 = 64, i bit di interesse sono 6 tra quelli variabili. Il primo disponibile per identificare la rete è 193.15.0.0 con maschera di 24-6 = 18. Il numero di host è pari a 256*64 – 2= 16382. Il primo indirizzo utile da poter assegnare all'interfaccia del router R2, scartando l'indirizzo di rete, è

193.15.0.1. Avendo che 193.15.0.0 = 11000001.00001111.00**000000**.00000000 con gli 6 bit evidenziati variabili, l'ultimo indirizzo dei 64 blocchi si trova ponendo ad 1 i bit rossi per l'indirizzo dell'ultimo blocco e ad 1 i bit dell'host ID: 11000001. 00001111. 00**111111**.11111111 = 193.15.255.255.

Sottorete	Indirizzo/Maschera sottorere	Numero di host indirizzabili nella sottorete	Indirizzo IP del router R2 sulla sottorete
A	193.8.0.0/14	262142	I1=193.8.0.1
B	193.12.0.0/15	131070	I2=193.12.0.1
C	193.14.0.0/16	65534	I3=193.14.0.1
D	193.15.0.0/18	16382	I4=193.15.0.1

TABELLA 1

I blocchi non ancora assegnati sono 2048 – (1024+512+256+64) = **192**.

Tabella di routing di R2 – Tra i due router R1 e R2 c'è un link Point-to-Point, abbiamo bisogno di quattro indirizzi di rete perché ne assegneremo uno per ciascun router e due sono quelli speciali: 2^2 = 4, ovvero 2 bit, e pertanto una maschera /30. L'indirizzo di default 0.0.0.0 ha maschera 0.0.0.0 e il salto successivo è il router R1 con indirizzo 192.19.64.2, visto che il router R2 sull'interfaccia I0 ha indirizzo192.19.64.1.

Vanno inseriti tutti gli indirizzi IP delle sottoreti relative ai gruppi rimanenti, ovvero B, C e D e le maschere /15, /16, /18 scritte nel formato dotted. Le interfacce si ricavano dal disegno e i salti successivi sono "local" perché ci si trova nella stessa sottorete, infatti A, B, C e D sono blocchi di sottoreti che fanno parte della grande sottorete del router R2.

Infine la sottorete192.19.64.0 la cui maschera è /30, con interfaccia I0.

Sottorete	Maschera sottorete	Next Hop	Interfaccia
0.0.0.0	0.0.0.0	192.10.64.2	I0
A=193.8.0.0	255.252.0.0	local	I1
B=193.12.0.0	255.254.0.0	local	I2
C=193.14.0.0	255.255.0.0	local	I3
D=193.15.0.0	255.255.192.0	local	I4
192.10.64.0	255.255.255.252	local	I0

TABELLA ROUTING di R2

Esame 15 gennaio – Traccia 3

Dobbiamo trovare le possibili rappresentazioni delle sottoreti e non degli host, di conseguenza conviene innanzitutto capire la classe alla quale appartiene l'indirizzo IP dell'Internet Service Provider: 193.16.0.0 = **110**00001.00010000.00000000.00000000 è un indirizzo di classe C, ciò vuol dire che la parte host-ID è composta da 8 bit, ovvero l'ultimo byte che, di conseguenza, non posso toccare quando scorro lungo le sottoreti. La maschera /14 = 11111111.11111100.00000000.00000000 = 255.252.0.0 che stabilisce il range di bit che devo scorrere per rappresentare tutte le sottoreti: 193.16.0.0 = **11000001.000100**00.00000000.00000000 . Ma la maschera relativa alla classe C è /24, e dunque 24-14 = 10 sono i blocchi, ovvero le sottoreti di indirizzi di classe C contigui che voglio rappresentare: **11000001.00010000.00000000**.00000000 .

A sono 512 blocchi, ovvero 512 sottoreti, il che equivale a 2^9, ovvero 9 bit. Essendo la prima sottorete parto dall'indirizzo dell'ISP, ovvero 193.16.0.0 = 11000001.0001000**0.00000000**.00000000 e devo variare i bit rossi per rappresentare tutte le 512 sottoreti, che sono proprio gli ultimi 9 che precedono gli 8 bit dedicati agli host che non possono essere toccati; quindi, l'ultima combinazione li setta tutti ad 1. Dunque avrò 11000001.0001000**1.11111111**.00000000 = 193.17.255.0 . Avendo stabilito che ogni sottorete di classe C può rappresenta 2^8 host, ovvero 256 host, per trovare il totale relativo al blocco A dovrò fare 256 * 512 = 131072 – 2 (gli indirizzi speciali) = 131070.

Abbiamo un indirizzo IP di classe C 193.16.0.0 con la relativa maschera di "default" /24 che impone a tutti gli indirizzi IP di questo tipo di dedicare 24 bit alla parte di rete e, 8 bit alla parte host. In questo caso l'ISP che stiamo considerando ha a disposizione 1024 sottoreti, ovvero 2^{10}, infatti ha maschera /14. In questo caso io voglio dividere le 1024 sottoreti in ulteriori 4 blocchi di sottoreti, ognuno di questi avrà la sua maschera. Il blocco A deve contenere 512 di quelle 1024 sottoreti, quindi conterrà 2^9 = 512 sottoreti. Ciò vuol dire che all'interno della parte di bit di rete, che sono 24, 9 sono dedicati al subnet-ID. Quindi per trovare la maschera di sottorete: $2^{(24-x)} = 2^9$ = 512 che sono le sottoreti del blocco A. Fare 24 – 9 = 15 mi porta ad avere la maschera per le sottoreti del blocco A: /15.

B sono 256 blocchi, ovvero 256 sottoreti, il che equivale a 2^8, ovvero 8 bit. Avendo concluso tutte le rappresentazioni del 193.17.255.0, ovvero del "17" perché nella classe C posso rappresentare al massimo 2^8 = 256 host, ovvero da 0 a 255, passo al successivo: 193.18.0.0 . E arrivo fino a rappresentare tutte le 256 sottoreti: 193.18.255.0 . Avendo stabilito che ogni sottorete di classe C può rappresenta 2^8 host, ovvero 256 host, per trovare il totale relativo al blocco B dovrò fare 256 * 256 = 65536 – 2 (gli indirizzi speciali) = 65534.

C sono 128 blocchi, ovvero 128 sottoreti, il che equivale a 2^7, ovvero 7 bit. Come prima vado da 193.19.0 fino a 193.19.127.0 . Avendo stabilito che ogni sottorete di classe C può rappresenta 2^8 host, ovvero 256 host, per trovare il totale relativo al blocco B dovrò fare 256 * 128 = 32768 – 2 (gli indirizzi speciali) = 32766.

D sono 32 blocchi, ovvero 32 sottoreti, il che equivale a 2^5, ovvero 5 bit. Come prima scorro di altri 32, partendo da 193.19.128.0, fino ad arrivare a 193.19.159.0 . Avendo stabilito che ogni sottorete di classe C può rappresenta 2^8 host, ovvero 256 host, per trovare il totale relativo al blocco C dovrò fare 256 * 32 = 8192 – 2 (gli indirizzi speciali) = 8190.

Dunque, avrò in A la maschera /15, in B la maschera /16 (24-8), in C la maschera /17 (24-7) e in D la maschera /18 (24-5).

Riassumendo

A: **193.16.0.0** -> 193.17.255.0

B: **193.18.0.0** -> 193.18.255.0

C: **193.19.0.0** -> 193.19.127.0

D: **193.19.128.0** -> 193.19.159.0

La terza domanda chiede di stabilire qual è il primo indirizzo affidabile ad un host guardando l'indirizzo IP di ciascun blocco. Sapendo che l'indirizzo che imposta a zero i bit riservati alla parte host, ovvero il primo, è l'indirizzo di rete, il primo disponibile è quello che imposta gli 8 bit dell'ultimo byte in questo modo 00000001. Infatti, in questo caso, avendo indirizzi di classe C, la parte host corrisponde agli 8 bit dell'ultimo byte. Così avremo 193.16.0.1 . Per la stessa logica il primo indirizzo utilizzabile del gruppo B sarà 193.18.0.1, per il gruppo C 193.19.0.1 e, per il gruppo D 193.19.128.1 .

La quarta domanda chiede quanti blocchi, ovvero sottoreti, di classe C tra i 1024, non sono ancora stati assegnati: 1024 – 512 – 256 – 128 – 32 = 96.

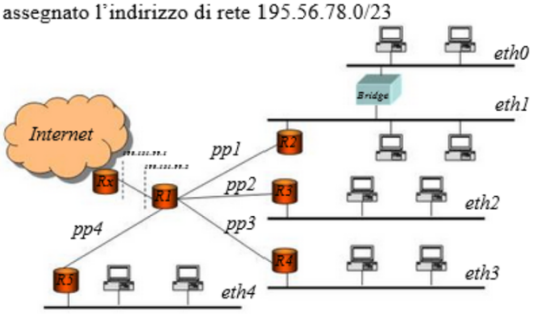
Tabella di routing di R2

Quando abbiamo un link Point-to-Point significa che stiamo collegando due host, abbiamo perciò bisogno di indirizzo di rete, indirizzo broadcast e altri due indirizzi per i due host: $2^2 = 4$. Ecco perché la maschera sarà /30. Dunque, il salto, passaggio successivo partendo dall'indirizzo di default che è 0.0.0.0 con maschera 0.0.0.0, è l'altro indirizzo IP disponibile, ovvero 192.10.64.2, sull'interfaccia I0.

Vanno inseriti tutti gli indirizzi IP delle sottoreti relative ai gruppi rimanenti, ovvero B, C e D e le maschere /15 /16 /17 /18 scritte nei formati dotted. Le interfacce si ricavano dal disegno e i salti successivi sono "local" perché ci si trova nella stessa sottorete.

Infine, la sottorete 192.10.64.0 la cui sottorete è /30 e si affaccia sull'interfaccia I0.

Alla rete in figura è assegnato l'indirizzo di rete 195.56.78.0/23



Le reti devono contenere almeno un numero di *host* (**includendo anche le interfacce dei router**) pari a eth0: 150, eth1: 60, eth2: 55, eth3:57, eth4: 61. I collegamenti "pp" sono collegamenti punto-punto (ottenuti ad esempio con giga-ethernet full duplex) e necessitano di due indirizzi IP. Suddividere la rete in sottoreti indicando per ognuna indirizzo e *netmask* (sia per le LAN *ethernet* che per i collegamenti punto-punto). Assegnare alle interfacce dei *router* degli indirizzi compatibili con quelli delle reti a cui sono collegate. Scrivere tabelle di *routing* consistenti per tutti i *router*.

L'indirizzo di rete 195.56.78.0 corrisponde a 11000011.00111000.01001110.00000000, con maschera /23, ovvero 11111111.11111111.11111110.00000000 = 255.255.254.0. Notando che il bridge è un oggetto di livello due che unisce due sottoreti avremo: rete I 150+60 (R_1- R_2), rete II 55 (R_1- R_3), rete III 57 (R_1- R_4) e rete IV 61 (R_1- R_5). Ricordando che si riempie sempre prima la rete composta dal maggior numero di host, partiamo dalla rete I.

Rete I – il totale degli host da rappresentare è pari a 212 perché vanno aggiunti i due speciali (indirizzo di rete e broadcast). Dunque ci sarà bisogno di 8 bit, infatti 2^8= 256. Per cui l'indirizzo di rete è quello assegnato alla rete di partenza, ovvero 195.56.78.0, mentre la maschera si ottiene come 32-8= 24, infatti 8 sono i bit dell'host ID. Il range va da 195.56.78.0 a 195.56.78.255.

Rete IV – la rete subito dopo rappresentabile è quella contenente 61 host a cui vanno aggiunti i due speciali, per rappresentare 63 indirizzi saranno necessari 6 bit poiché 2^6= 64. La maschera necessario è di tipo 32-6= 26, /26. Il primo indirizzo di rete disponibile è 195.56.79.0, partendo dal quale si arriverà fino a 195.56.79.63.

Rete III – i 59 indirizzi necessari si ottengono come combinazione di 6 bit per l'host ID, infatti 2^6= 64. Il primo indirizzo disponibile sarà 195.56.79.64 con maschera /26, fino a raggiungere l'indirizzo 195.56.79.127.

Rete II – i 57 indirizzi necessari si ottengono come combinazione di 6 bit per l'host ID, infatti 2^6= 64. Il primo indirizzo disponibile sarà 195.56.79.128 con maschera /26, fino a raggiungere l'indirizzo 195.56.79.191.

Point-to-Point – tra R_1- R_2 sono richiesti due indirizzi da destinare ai router in aggiunta ai due speciali, ciò si traduce nella necessità di disporre di 4 indirizzi, 2^2= 4, e di una maschera /30. Il primo indirizzo disponibile è 195.56.79.192 mentre l'ultimo sarà 195.56.79.195. Tra R_1- R_3 altri 4 per coprire il range 195.56.79.196 - 195.56.79.199. Tra R_1- R_4 altri 4 per coprire il range 195.56.79.200 - 195.56.79.203. Tra R_1- R_5 altri 4 per coprire il range 195.56.79.204 - 195.56.79.207.

Network	Netmask	First Hop
195.56.78.0	255.255.255.0	195.56.79.193
195.56.79.0	255.255.255.192	195.56.79.197
195.56.79.64	255.255.255.192	195.56.79.201
195.56.79.128	255.255.255.192	195.56.79.205
0.0.0.0	0.0.0.0	190.131.99.1